

UNIVERSAL THIRSTY FAMILY / CANONICAL FIELD MANUAL

Thirsty-Lang UTF 101 (Universal Thirsty Family)

The Canonical End-to-End Reference

ONE FAMILY. SIX TIERS. ONE GOVERNED CURRENT.



SOFTWARE	DOCUMENT	SOURCE SET
v0.8.6	2026-08-02	05193bc67744ed39

Language and runtime contract | policy and proof specification | security acceptance | deployment | release evidence | source traceability

Canonical principle: missing is not false; invalid is not false; unresolved is not evidence.

Contents

The PDF outline and this table of contents are generated from the maintained source headings.

Canonical Orientation and End-to-End Guide	10
1. Document authority and scope	10
1.1 Authority legend	10
1.2 Source hierarchy	10
1.3 Published boundary	11
2. The mental model	11
3. Install and prove provenance	12
3.1 Isolated installation on PowerShell	12
3.2 Verify the installed artifact	12
4. Fifteen-minute core path	13
Step 1: create hello.thirsty	13
Step 2: run it	13
Step 3: add state and repetition	13
Step 4: inspect other core entry points	13
5. Fifteen-minute governed path	14
Step 1: create governed_hello.thirsty	14
Step 2: create tutorial.tarl	14
Step 3: run under the policy	14
Step 4: prove the denial path	14
Step 5: inspect static obligations without executing effects	14
Step 6: understand the production difference	15
6. The exact six-tier family	15
7. Tier 1 language bridge	15
7.1 Modules and modes	15
7.2 Bindings and mutation	16
7.3 Functions, lambdas, and contracts	16
7.4 Control flow	16
7.5 Fountains, members, and subscripts	17
7.6 Error and asynchronous structure	17
7.7 Types and implementation limits	17
7.8 Expression behavior	17
8. Modules, standard library, and project files	18
8.1 Imports	18
8.2 Sensitive capability map	18
8.3 Package manager scope	19
9. All seven installed CLIs	19
9.1 thirsty	19
9.2 tarl	19
9.3 tarl-lsp	20

9.4 shadow-thirst	20
9.5 tscg	20
9.6 tscg-b	20
9.7 thirst-of-gods	20
10. TARL policy fundamentals	21
10.1 Basic policy	21
10.2 Safe expressions	21
11. The authoritative context contract	22
11.1 One representation	22
11.2 Resolution is typed	23
11.3 Accepted value domain	23
11.4 Command-line evaluation	23
12. Schema, verdict, proof, and audit	23
12.1 Context schemas	24
12.2 Positive verdict admissibility	25
12.3 Proof binding	25
12.4 Registered sources	25
12.5 Durable audit	25
13. Hardened policy operation	26
13.1 Trust roots	26
13.2 Hardened CLI shape	26
13.3 Independent proof verification	26
13.4 Time and expiry	27
13.5 Revocation and replay	27
13.6 Broker and path guard	27
13.7 ESCALATE and quorum	27
14. Builds and governance loss	28
15. Higher tiers	28
15.1 Thirst of Gods	28
15.2 Shadow Thirst	29
15.3 TSCG	29
15.4 TSCG-B	29
16. Production, security, and release receipts	30
16.1 What the release evidence proves	30
16.2 What the release evidence does not prove	30
16.3 Continuous and release gates	30
17. Migration from 0.8.5 to 0.8.6	31
17.1 Upgrade the artifact	31
17.2 Convert caller data to the one accepted representation	31
17.3 Revalidate every policy family	31
17.4 Reissue proof and verification expectations	32
17.5 Preserve historical evidence	32
18. Troubleshooting	32
tarl or another CLI is not found	32

user.role denies even though the JSON contains an admin 32

Missing data makes an inequality or not expression deny 33

The schema is incomplete 33

tarl eval requests --now 33

An unsigned proof does not verify 33

A registered-source proof asks for two contexts 33

A proof verifies once and fails the second time 33

Hardened execution denies despite an ALLOW rule 33

No audit rows appear in hardened mode 33

A governed build refuses 33

TARL analysis reports that Z3 is unavailable 34

LLVM object, executable, assembly, or JIT build fails 34

--locked refuses to run 34

Strict or pure mode rejects an otherwise valid program 34

A governed imported file behaves differently from a core import 34

Docker behavior does not match the published package 34

19. Glossary 35

20. Source and test traceability 35

21. Validation commands 36

22. Where to continue 37

Language Family Specification 38

Overview 38

Tier 1 - Thirsty-Lang (Core) 38

 Keywords 38

 Operators 39

 Syntax Examples 39

 Types 40

 CLI Commands 40

Tier 2 - Thirst of Gods 40

 Keywords 41

 Syntax Examples 41

Tier 3 - T.A.R.L. (Thirsty's Active Resistance Language) 41

 Keywords 41

 Syntax 41

 Authoritative context contract 42

Tier 4 - Shadow Thirst 42

 Keywords 42

 Built-in Analyzers 43

Tier 5 - TSCG (Thirsty Symbolic Constraint Grammar) 43

 Symbols 43

 Operators 43

Tier 6 - TSCG-B (Thirsty Symbolic Constraint Grammar - Binary) 44

 Frame Format 44

License	44
-------------------	----

Tier 1 Source Grammar 45

Notation	45
1. Lexical grammar	45
1.1 Identifiers and member access	45
1.2 Numbers	45
1.3 Strings and escapes	46
1.4 Comments and whitespace	46
2. Program structure and types	46
3. Expressions	47
Exact precedence and associativity	48
4. Statements and blocks	48
5. Declarations and governed forms	49
5.1 Functions and contracts	49
5.2 Fountains	49
5.3 Error handling and cleanup	50
5.4 Data and interface declarations	50
5.5 Higher-tier marker forms accepted by Tier 1	50
6. Lexically reserved words	50
7. Source anchors	51

Feature and Tooling Reference 52

1. Language Core (thirsty, .thirsty)	52
1.1 Declarations	52
1.2 Statements	52
1.3 Expressions & operators	52
1.4 Built-in functions	53
1.5 Modes	53
2. Governance (the differentiator)	53
2.1 Design-by-contract	53
2.2 Capability gates	53
2.3 Cryptographic proofs	54
2.4 Temporal governance	54
2.5 Static parity (checker)	54
3. Semantic Verifiers	54
3.1 Shadow Thirst (shadow-thirst) - safe mutation & promotion	54
3.2 Thirst of Gods (thirst-of-gods) - deity contracts	54
4. T.A.R.L. - Policy Engine (tarl)	55
5. TSCG / TSCG-B - symbolic & binary tiers	55
6. Tooling & Workflow	55
7. Representative use cases	56

Governance and T.A.R.L. Runtime Model 57

Overview	57
Core Principles	57
T.A.R.L. Policy Language	57
Syntax	57
Example Policy	57
Conditions	58
Safe Evaluation	58
Authoritative context representation	58
Integration with Thirsty-Lang	58
Runtime Enforcement (the interpreter, today)	58
Static proof-obligation reporting	60
Extended governance surfaces	61
Runtime Evaluation	61
TarlRuntime	61
CLI Usage	61
Security Considerations	62
Future Enhancements	62

Production Deployment Checklist 63

1. Run hardened, fail-closed	63
2. One enforcement path (broker + path guard)	63
3. Context authority and registered-source transformations	63
4. Durable, cross-process replay & revocation state	64
5. External audit checkpoints	64
6. Trust-root key management	64
7. Secret custody (remains with the deployment)	65
8. Release gates (enforced in CI)	65
Verifying a deployment end to end	65

Offensive Threat Model C001-C073 67

Purpose	67
Security Objective	67
Existing Resistance Surface	67
Protected Assets	68
Trust Boundaries	68
Adversary Classes	69
Offensive Challenge Catalog	69
Mandatory Invariants	74
Offensive Test Suites	75
Current Defensive Evidence	77
Remaining Gaps	78
Proof-Obligation Behavior Contracts	78
Acceptance Bar For Hardened Runtime	78

Feature Status and Bootstrap Competence Register	80
Bootstrap Competence Register	80
Language core (Thirsty-Lang)	81
Governance (maximal)	81
Hardened runtime (offensive catalog C022-C073)	83
Semantic verifiers	85
T.A.R.L. policy engine	85
Stability	85
Security Policy	86
Supported Versions	86
Reporting a Vulnerability	86
How to Report	86
What to Include	86
Response Timeline	87
Disclosure Policy	87
Security-Relevant Components	87
Offensive Threat Model	87
Default DENY	87
Security Best Practices	87
Container Operations	89
Current Published Image	89
Quick Start	89
Verify the CLI and run the demo	89
Start an interactive REPL	89
Run a script from the current directory	89
Local Development	90
Docker Compose Services	90
Image Contents	90
Governance Example	91
Release Publishing	91
Local Helper Scripts	91
Troubleshooting	92
Image not found	92
Build fails while running tests	92
Mounted files are not writable	92
Confirm the published architectures	92
Contributor and Validation Workflow	93
Code of Conduct	93
How to Contribute	93
Reporting Issues	93

Submitting Changes	93
Development Setup	94
Development Workflow	94
Code Standards	94
Pull Request Guidelines	95
Governance	95
Questions?	95

Release Authentication and Provenance 96

Current 0.8.6 publication contract	96
Recommended future migration: Trusted Publishing	96
Optional detached OpenPGP signatures	97
TARL proof signing is separate	97
References	97

Architecture Whitepaper 98

Abstract	98
1. Motivation and threat setting	98
Mandatory invariants	98
2. Architecture: the Universal Thirsty Family	99
3. The language (Tier 1)	99
4. The governance model (Tier 3 in action)	100
4.1 Capability gates, fail-closed	100
4.2 Imports execute under the caller's governance	100
4.3 The hard floor	100
4.4 Parser fail-closed	100
5. T.A.R.L. - the policy engine and proof system	100
5.1 Policy language and evaluation	100
5.2 Temporal governance	101
5.3 Composition	101
5.4 Proof-carrying decisions	101
5.5 Independent verification - secure default, explicit unsigned mode	101
5.6 Static analysis (optional Z3 layer)	102
5.7 Audit, explanation, and tooling	102
6. Mutation governance (Tier 4) and integrity (Tiers 5-6)	102
7. Build outputs and governance loss	103
8. Security posture: covered vs. deferred	103
9. Engineering state (0.8.6)	104
10. Getting started	104
11. Conclusion	105

Operational Continuity and Release Evidence 106

2026-08-02 v0.8.6 Release Closure and Documentation Reconciliation	106
2026-08-02 Context Resolution Integrity Repair	106

2026-07-29 Sync and Release Fix Workstream 107

 What Was Read 107

 What Was Changed 108

 Verified 108

 Not Verified / Requires User Action 108

 Recommended Next Steps 108

 Open Issues 108

Baseline 108

Implemented 109

Files Modified 109

Tested 110

Not Yet Verified 111

Behavior Contracts 111

Open Issues 111

Resolved Validation Notes 111

Future Work 111

Safe to Continue 112

Document Provenance **113**

PART 01 / INTEGRATED-GUIDE

Canonical Orientation and End-to-End Guide

Maintained source: docs/THIRSTY_LANG_UTF_101.md



- **Canonical bridge manual for Thirsty-Lang 0.8.6**
- **Python:** 3.11 or later
- **Package:** `thirsty-lang`
- **License:** Apache-2.0

Release date: 2026-08-02

Thirsty-Lang is a governance-first programming language family. Its central execution question is not only "can this code run?" but also "should this action run under this policy, authority, context, and proof?"

This manual is the end-to-end bridge between a first successful program and the maintained language, policy, security, and operations references. It deliberately does not copy every grammar production or API member. The generated single-file PDF appends the detailed specifications and reference material behind this bridge.

Use this document to learn the system, select the correct execution path, and find the authoritative source or test for a claim.

1. Document authority and scope

1.1 Authority legend

Label	Meaning
Contract	Behavior implemented by active source and protected by deterministic tests.
Security contract	Fail-closed behavior whose test or threat-model evidence is part of the security boundary.
Published surface	Code, metadata, or commands included in the <code>thirsty-lang</code> 0.8.6 package.
Operational requirement	A control that an embedding or deployment must configure; installing the package alone does not satisfy it.
Repository-local	Material present in the repository but not included in the published package.
Historical	Evidence or behavior retained to explain a previous release. It does not override the current contract.
Pending acceptance	Implemented and released behavior whose independent constitutional acceptance is still open in the Competence Register.

1.2 Source hierarchy

When two descriptions differ, use this order:

- 1 The exact published artifact decides what a user of that artifact received.
- 2 Active source plus deterministic tests decides current implemented behavior.
- 3 `pyproject.toml`, package contents, and release workflows decide the shipped interface and provenance.

- 4 Maintained specifications and security documents explain that behavior.
- 5 Examples and tutorials demonstrate selected paths but do not expand the language or security contract.
- 6 Generated API pages are navigation aids. They do not override source or tests.

The primary maintained references are:

- [Language specification](#)
- [Exact grammar](#)
- [Feature and capability matrix](#)
- [Governance runtime model](#)
- [Feature status and Competence Register](#)
- [Offensive threat model](#)
- [Production deployment requirement](#)
- [Whitepaper](#)
- [Signing and publication model](#)
- [Release and shipping guide](#)
- [Security policy](#)
- [Operational continuity map](#)

1.3 Published boundary

The 0.8.6 wheel packages these namespaces:

TEXT

```
utf
utf.*
thirsty_lang
```

The canonical import is:

PYTHON

```
from utf import thirsty_lang
```

`import thirsty_lang` is a compatibility shim to the same implementation.

The top-level `governance/` tree and `src/psia` are **repository-local**. They are not included in the published 0.8.6 Python package and are not part of its installed runtime or security boundary. Their presence in a checkout must not be presented as a capability of a clean PyPI installation.

2. The mental model

The shortest accurate model is:

TEXT

```
source
-> lex, parse, and check
-> core or governed runtime
-> contracts and capability request
-> TARL policy evaluation
-> ALLOW, DENY, or ESCALATE
-> proof record
-> optional required durable audit
-> governed effect only after admission
```

The terms are intentionally separate:

- A **policy** states the decision rules.

- A **context** is the finite JSON input evaluated by those rules.
- A **schema** establishes that the evaluated context has the required shape and types.
- A **verdict** is `ALLOW`, `DENY`, or `ESCALATE`.
- A **proof** binds the decision to policy, context, schema, rule, time, and optional cryptographic identity.
- An **audit archive** durably persists proof records in a hash-linked chain.
- An **authority** identifies and, in hardened mode, cryptographically authenticates who may request the action.
- A **broker** is the single admission point for governed effects and external adapters.

These objects are related, but none substitutes for another. In particular, a proof object is not automatically a signed proof, and a proof is not durable audit evidence until an attached archive successfully stores it.

3. Install and prove provenance

3.1 Isolated installation on PowerShell

POWERSHELL

```
$ErrorActionPreference = "Stop"

py -3.11 -m venv .venv
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
.\.venv\Scripts\Activate.ps1

python -m pip install --upgrade pip
python -m pip install --no-cache-dir thirsty-lang==0.8.6
```

On POSIX shells:

BASH

```
python3.11 -m venv .venv
. .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install --no-cache-dir thirsty-lang==0.8.6
```

Development and analysis extras are explicit:

BASH

```
python -m pip install -e ".[dev,analysis]"
```

`analysis` adds Z3. It is not required for ordinary parsing, execution, or policy evaluation.

3.2 Verify the installed artifact

POWERSHELL

```
python -m pip show thirsty-lang
python -c "import importlib.metadata as m; print(m.version('thirsty-lang'))"
Get-Command thirsty, tarl, tarl-lsp, shadow-thirst, tscg, tscg-b, thirst-of-gods |
    Format-Table Name, Source
thirsty --version
tarl --help
```

If a console script is not on `PATH`, invoke the virtual-environment executable directly, for example:

POWERSHELL

```
.\.venv\Scripts\tarl.exe --help
```

For a source checkout:

BASH

```
git clone https://github.com/IAMSoThirsty/Thirsty-lang.git
cd Thirsty-lang
python -m pip install -e ".[dev,analysis]"
```

Do not treat an editable checkout as proof of what the PyPI artifact contains. Release verification uses a clean environment and the pinned published version.

4. Fifteen-minute core path

This path executes ordinary Tier 1 code. No policy or authority is required because the module is explicitly `core`.

Step 1: create `hello.thirsty`

THIRSTY

```
module hello: core

glass greet(name) -> String {
  return "hello, " + name + "!"
}

drink message = greet("Thirsty-Lang")
pour message
```

Step 2: run it

BASH

```
thirsty run hello.thirsty
```

Expected output:

TEXT

```
hello, Thirsty-Lang!
```

Step 3: add state and repetition

THIRSTY

```
module counter_demo: core

fountain Counter {
  drink count: Int = 0

  glass init(self) {
    self.count = 0
  }

  glass increment(self) -> Int {
    self.count = self.count + 1
    return self.count
  }
}

drink counter = new Counter()
times 3 {
  pour counter.increment()
}
```

Run and format it:

BASH

```
thirsty run counter.thirsty
thirsty fmt --check counter.thirsty
thirsty fmt counter.thirsty
```

Step 4: inspect other core entry points

BASH

```
thirsty repl
thirsty new sample-project --mode core
thirsty doctor
```

The core path demonstrates syntax and runtime behavior. It does not create a governed-execution claim.

5. Fifteen-minute governed path

This first governed example uses a plain local authority string so the policy flow is visible without provisioning keys. It is a tutorial path, not the production hardened profile.

Step 1: create governed_hello.thirsty

THIRSTY

```
module governed_hello: governed

glass greet(name) -> String
  requires length(name) > 0
  ensures length(result) > 0
{
  return "governed hello, " + name + "!"
}

drink message = greet("operator")
pour message
```

The function contract makes `greet` a governed function entry. `pour` also requests the `write` capability for standard output.

Step 2: create tutorial.tarl

TARL

```
policy governed_tutorial

when action == "greet" => ALLOW
when action == "write" => ALLOW
when true => DENY
```

Rules are first-match-wins. The final rule is an explicit default-deny control.

Step 3: run under the policy

BASH

```
thirsty run governed_hello.thirsty \
  --thirst-level governed \
  --policy tutorial.tarl \
  --authority local-tutorial
```

On PowerShell, backticks may replace the backslashes or the command can be entered on one line.

Step 4: prove the denial path

Copy the policy, remove the `write` grant, and run with the restrictive copy. The governed `pour` must fail closed with exit code 2. A denial is evidence that the gate executed; it is not a runtime crash.

Step 5: inspect static obligations without executing effects

BASH

```
thirsty prove governed_hello.thirsty \
  --policy tutorial.tarl \
  --emit-manifest

thirsty explain-denial governed_hello.thirsty \
  --policy tutorial.tarl
```

`thirsty prove` parses and checks the program and emits obligations. It does not run program side effects. Its report includes governed calls, sensitive standard-library calls, required actions and capabilities, contracts, context schema status, authority and proof requirements, policy/source hashes, audit requirements, and unresolved gaps.

Step 6: understand the production difference

The hardened path adds:

- a signed authority claim verified against an authority-issuer public key;
- Ed25519-signed decision proofs;
- an explicit context schema or a complete derived schema;
- a trusted clock for temporal authority;
- durable replay and revocation stores for independent verification;
- an explicitly attached audit archive when durable audit is required;
- a capability broker and path guard for all external effects.

Follow [PRODUCTION_DEPLOYMENT.md](#) for the complete operator checklist. `--hardened` alone does not create an audit database.

6. The exact six-tier family

The Universal Thirsty Family has six implemented tiers.

Tier	Name	Current role	Primary source
1	Thirsty-Lang	Lexer, parser, checker, interpreter, formatter, modules, package tools, contracts, builds, and primary CLI	<code>src/utf/thirsty_lang</code>
2	Thirst of Gods	Structural deity-contract validation over the real Tier 1 AST, then interpretation or transpilation	<code>src/utf/thirst_of_gods</code>
3	T.A.R.L.	Policy parsing, safe evaluation, schema admission, proof generation/verification, time, audit, broker, and escalation	<code>src/utf/tarl</code>
4	Shadow Thirst	Mutation analysis and <code>PROMOTE</code> , <code>FLAGGED</code> , or <code>REJECT</code> decisions	<code>src/utf/shadow_thirst</code>
5	TSCG	Symbolic constitutional grammar, parsing, canonical spacing, validation, and checksum	<code>src/utf/tscg</code>
6	TSCG-B	Binary frames for UTF-8 TSCG text with CRC32 and SHA-256 integrity	<code>src/utf/tscg_b</code>

The tiers form a family, not six interchangeable authorization gates. TARL and the capability broker perform policy admission. Shadow Thirst, TSCG, and TSCG-B contribute analysis or integrity services; they do not independently authorize a governed effect.

7. Tier 1 language bridge

The complete syntax belongs in [GRAMMAR.md](#) and [LANGUAGE_SPEC.md](#). This section supplies the working mental map.

7.1 Modules and modes

THIRSTY

```
module name: core
module name: governed
module name: strict
module name: pure
```

- `core` is ordinary execution.
- `governed` routes governed calls and capability-bearing effects through the configured governance path.
- `strict` rejects uninitialized bindings.
- `pure` rejects `pour` and `sip` I/O.

A governed parse error fails closed. Recovered statements are discarded rather than partially executed.

7.2 Bindings and mutation

THIRSTY

```
drink value = 1
drink mut counter = 0
let stable = "fixed"
next_value := 2
counter = counter + 1
```

`let` is immutable. `drink mut` and `:=` establish mutable bindings. Assignment mutates an existing mutable binding or supported member/subscript target.

7.3 Functions, lambdas, and contracts

THIRSTY

```
glass add(a, b) -> Int {
  return a + b
}

drink double = glass(x) -> Int {
  return x * 2
}
```

Contracts are executable checks:

THIRSTY

```
glass withdraw(amount) -> Int
  requires amount > 0
  ensures result >= 0
  invariant amount < 10000
{
  return amount
}
```

`requires` is checked before the body. `ensures` is checked after the body with `result` bound. `invariant` is checked at entry and exit. Method contracts are design-by-contract even in core mode; governed top-level calls additionally require policy and authority admission.

7.4 Control flow

Implemented control-flow families include:

THIRSTY

```
thirsty condition {
  pour "yes"
} hydrated {
  pour "no"
}

for item in items {
  pour item
}

refill condition {
  // while-like loop
}

refill (item in items) {
  // for-each form
}

refill (drink i = 0; i < 10; i = i + 1) {
  // C-style form
}

times 3 {
  pour "again"
}
```

The C-style `refill` form desugars to initialization plus a while-like loop; a declaration used as its counter is made mutable by the parser.

7.5 Fountains, members, and subscripts

THIRSTY

```
fountain Account {
    drink balance: Int = 0

    glass init(self) {
        self.balance = 0
    }

    glass credit(self, amount) {
        this.balance = this.balance + amount
        return this.balance
    }
}

drink account = new Account()
drink values = [10, 20, 30]
values[1] = account.credit(values[1])
```

Both `self` and `this` receiver conventions are supported. Lists, dictionaries, and strings support indexed reads; lists and dictionaries support assignment where appropriate. Bounds and invalid index types are checked.

7.6 Error and asynchronous structure

THIRSTY

```
spillage {
    drink result = cascade perform_work()
    return result
} error (problem) {
    pour problem
} cleanup {
    pour "cleanup"
} finally {
    pour "finished"
}
```

`spillage`, `error`, `cleanup`, `finally`, `throw`, and `cascade` provide the structured error and asynchronous vocabulary used by Tier 1 and validated more strictly by Thirst of Gods.

7.7 Types and implementation limits

Runtime/checker types include `Int`, `Float`, `Bool`, `String`, `Void`, `Any`, `Error`, `Quenched`, `Reservoir`, `Task`, `Result`, `Governed`, function types, and internal enum/struct/interface/generic forms.

Important 0.8.6 limits:

- An unannotated function return type is `Any`.
- A source type annotation currently consumes a single identifier; generic type strings accepted by the Python type API are not a general source syntax.
- Fountain inheritance or mixin syntax is not implemented.
- Enum variants do not carry parsed payload types.
- Interface parsing does not implement complete typed parameter signatures.
- `morph` does not parse a return annotation.
- Bare `!` is invalid; use `not`.
- Semicolons are generally optional, except where the grammar uses them as structural separators such as a C-style `refill` header.

7.8 Expression behavior

The implementation precedence, lowest to highest, is:

- 1 assignment

- 2 pipe and thirst
- 3 or
- 4 and
- 5 comparisons
- 6 + and -
- 7 *, /, %
- 8 arrow, ||, and ^
- 9 member access, calls, and subscripts

In Tier 1, ^ and || require Boolean operands. The pipe tokens |> and bare | are both recognized. Consult the grammar for the exact parse forms.

8. Modules, standard library, and project files

8.1 Imports

THIRSTY

```
import "thirst::json" as json
import "thirst::fs" as fs
```

The standard library currently registers 16 namespaces:

TEXT

```
thirst::time      thirst::crypto    thirst::reservoir
thirst::fs        thirst::path      thirst::json
thirst::http      thirst::env       thirst::process
thirst::log       thirst::test     thirst::collections
thirst::net       thirst::sqlite   thirst::yaml
thirst::toml
```

Global built-ins are:

TEXT

```
length  contains  split    abs
min      max      push     pop
size     get       flood    condense
evaporate strain  transmute distill
```

8.2 Sensitive capability map

Governed mode wraps the functions currently declared in SENSITIVE_STDLIB_CAPABILITIES:

Namespace	Mapped actions
thirst::fs	read_file, exists, list_dir as read; write_file, mkdir, remove as write
thirst::http	get, post, put, delete as network
thirst::net	tcp_connect, tcp_listen, udp_send as network
thirst::env	get, all as read; set as write
thirst::process	run, exit as execute; args, pid as read
thirst::log	info, warn, error, debug as write
thirst::test	describe, it as write
thirst::sqlite	connect, execute, close as write; query as read

This table is a boundary, not a claim that every callable in all 16 namespaces is mediated. For example, current time and crypto functions are not listed in this map. A production embedding must inspect or extend the map when its threat model treats additional functions as sensitive.

8.3 Package manager scope

`thirsty add`, `thirsty lock`, and `thirsty audit` manage local manifest and lock integrity. Version 0.8.6 records `thirsty-registry://`-style resolution identities and deterministic hashes; it does not download packages from a live remote dependency registry. Do not treat a generated lockfile as supply-chain verification of externally fetched code.

`thirsty run --locked` requires the local lock integrity check to pass.

9. All seven installed CLIs

The following is the exact 0.8.6 command family. Use each command's `--help` for all flags and positional arguments.

9.1 `thirsty`

Subcommands:

TEXT

```
run
repl
fmt
new
build
prove
explain-denial
govern
add
audit
lock
doctor
lsp
docs
```

Notes:

- `audit` here audits dependency-lock integrity. TARL archive audit is under `tarl audit`.
- `lsp` launches the TARL policy language server over stdio or TCP. It is not a separate Thirsty source-language LSP.
- `prove` and `explain-denial` are static and do not execute effects.
- `docs` generates the repository documentation site surface; this maintained manual and its manifest-driven PDF remain the canonical integrated guide.

9.2 `tarl`

Subcommands:

TEXT

```
eval
parse
verify
lint
audit
keygen
revoke
explain
test
analyze
```

`tarl audit` has three subcommands:

TEXT

```
verify-chain
checkpoint
query
```

tarl analyze modes are exactly:

TEXT

```
coverage
shadows
conflicts
equiv
refines
```

tarl keygen roles are exactly:

TEXT

```
authority-issuer
proof-signer
time-authority
```

9.3 tarl-lsp

tarl-lsp has no subcommands. It runs the TARL JSON-RPC language server over standard input/output. Its command-line options are `--help` and `--version`.

9.4 shadow-thirst

Subcommands:

TEXT

```
check
visualize
```

9.5 tscg

Subcommands:

TEXT

```
parse
canonical
checksum
validate
list
```

9.6 tscg-b

Subcommands:

TEXT

```
encode
decode
stream
```

`encode` writes a binary frame. `decode` accepts a hexadecimal frame argument or binary standard input. `stream` decodes concatenated frames from hexadecimal input or standard input.

9.7 thirst-of-gods

Subcommands:

TEXT

```
run
check
transpile
```

`transpile --target` accepts exactly `thirsty` or `js`.

10. TARL policy fundamentals

TARL is Thirsty's Active Resistance Language. It evaluates explicit rules and produces one of three verdicts:

TEXT

```
DENY < ESCALATE < ALLOW
```

DENY is the default. **ESCALATE** is not an implicit **ALLOW**; promotion requires the separately configured quorum path.

10.1 Basic policy

TARL

```
policy access_control v1

when user.role == "admin" => ALLOW
when action == "delete" and resource == "critical" => ESCALATE
when true => DENY
```

Rule syntax is:

TEXT

```
when <condition> => ALLOW|DENY|ESCALATE [for: <duration>]
```

Policy support also includes:

- **EXTENDS** and **RESTRICTS** relationships;
- **INCLUDE** by name or file with optional alias;
- **STOP**;
- **valid_from**, **valid_until**, **supersedes**, and **on_expiry** metadata;
- succession with **if_unresolved_after** and **revert_to**;
- policy sets using **UNION**, **INTERSECT**, or **MAJORITY** plus a default verdict.

Malformed rules, dates, and durations are rejected. **on_expiry** cannot grant **ALLOW**.

10.2 Safe expressions

TARL expressions support:

- **and**, **or**, and **not**;
- comparisons and strict membership;
- arithmetic and unary minus;
- nested dotted paths;
- list/set-like literals;
- registered sources;
- safe functions and quantifiers;
- explicit temporal built-ins.

Safe functions include **MATCHES**, **STARTS_WITH**, **ENDS_WITH**, **CONTAINS**, **ELAPSED_SINCE**, **LEN**, **LOWER**, and **UPPER**.

TARL's expression algebra is deliberately stricter than Tier 1 runtime truthiness:

- Boolean operators require resolved Boolean operands.
- Both Boolean operands are validated even when one would decide the result.
- Every supplied **ALL** or **ANY** element is validated.

- Empty quantifier collections provide no authorization evidence and fail closed.
- Integers and floats may compare numerically; strings never coerce to numbers.
- Non-finite numeric operands or results fail closed.
- Missing or malformed paths cannot enter comparisons, membership, functions, or ordinary Boolean algebra.

11. The authoritative context contract

11.1 One representation

TARL policy expressions use dotted **paths**. Caller context uses nested JSON.

Policy:

TARL

```
when user.role == "admin" => ALLOW
```

Authoritative context:

JSON

```
{
  "user": {
    "role": "admin"
  }
}
```

Representation identity:

TEXT

```
tarl.context.nested-json.v1
```

Normalization:

TEXT

```
algorithm: identity
version: 1
path model: nested objects
silent conversion: none
```

A flat dotted key is rejected:

JSON

```
{
  "user.role": "admin"
}
```

A mixed representation is also rejected, even if both values are equal:

JSON

```
{
  "user.role": "admin",
  "user": {
    "role": "admin"
  }
}
```

A contradictory mixed representation is a representation conflict, never a choice between competing values:

JSON

```
{
  "user.role": "guest",
  "user": {
    "role": "admin"
  }
}
```

11.2 Resolution is typed

Every path lookup has one of four states:

TEXT

```
RESOLVED(value)
MISSING
TYPE_ERROR
REPRESENTATION_CONFLICT
```

Only **RESOLVED** carries an ordinary expression value. The other states short-circuit policy evaluation to a fail-closed decision with an explicit reason.

This distinction is constitutional:

TEXT

```
missing is not false
invalid is not false
unresolved is not evidence
```

A resolved Boolean false remains valid and distinguishable:

JSON

```
{
  "feature": {
    "enabled": false
  }
}
```

11.3 Accepted value domain

Caller contexts and registered-source values must be finite JSON values:

- objects with string keys;
- arrays;
- strings;
- Booleans;
- integers and finite floating-point values;
- null.

Duplicate object keys, NaN, infinity, cycles, tuples, sets, and arbitrary Python objects are rejected. Callers cannot provide reserved temporal, internal, or `source: *` fields.

11.4 Command-line evaluation

BASH

```
tarl parse access.tarl

tarl eval access.tarl \
  --context '{"user":{"role":"admin"}}' \
  --json
```

The following must deny because the representation is invalid:

BASH

```
tarl eval access.tarl \
  --context '{"user.role":"admin"}' \
  --json
```

12. Schema, verdict, proof, and audit

These four layers are intentionally independent.

Layer	Question answered	Failure meaning
Context schema	Did the exact evaluated representation contain the required fields and types?	Policy rules do not run; fail closed with DENY or configured ESCALATE .
Verdict	Which rule decided this request?	No matching authoritative grant means no effect.
Proof	Can the decision be bound to policy, context, schema, rule, time, and signer?	The verdict may be diagnostic but is inadmissible for a consumer requiring those bindings.
Audit archive	Was the proof durably appended to a hash-linked history?	If persistence is configured as required, failure downgrades the decision to DENY .

12.1 Context schemas

Schema kinds are exactly:

TEXT

```
string
int
float
number
bool
list
dict
```

Boolean is not accepted as `int` or `number`, despite Python's internal type relationship.

An explicit schema may use the full canonical form:

JSON

```
{
  "status": "explicit",
  "representation": {
    "id": "tarl.context.nested-json.v1",
    "path_model": "nested-objects",
    "normalization": "none"
  },
  "on_violation": "DENY",
  "fields": [
    {
      "name": "user.role",
      "kinds": ["string"],
      "required": true
    },
    {
      "name": "risk",
      "kinds": ["number"],
      "required": false
    }
  ]
}
```

`on_violation` cannot be `ALLOW`. Unknown kinds, duplicate fields, unknown metadata, representation mismatch, or requested normalization are rejected.

The proof-obligation loader also accepts the documented compact field mapping, then normalizes it into the canonical schema model:

JSON

```
{
  "fields": {
    "user.role": "string",
    "risk": {
      "kind": "number",
      "required": false
    }
  }
}
```

When static schema derivation is incomplete, provide an explicit schema rather than loosening the evaluator. A differing policy-text override receives its own derived schema; the runtime does not silently reuse the base policy's derived schema.

12.2 Positive verdict admissibility

`tarl eval` can return a raw `ALLOW` for diagnostic use. A load-bearing broker or governed runtime accepts positive authority only when the explicit or complete derived schema passed and that result is bound into the proof for the exact representation evaluated.

This is the governing rule:

A positive policy verdict is inadmissible as advancement authority unless the exact context representation used for schema validation is the same representation evaluated by the policy engine and bound into the proof.

No conversion may occur silently.

12.3 Proof binding

A TARL proof records or binds, as applicable:

- policy hash;
- original and canonical/evaluated context hashes;
- context representation identity;
- normalization algorithm and version;
- conflict status;
- schema fingerprint, representation, and validation status;
- matched rule and condition;
- verdict and evaluation trace;
- evaluation time and expiry;
- signing algorithm, key ID, and signature.

HMAC is a symmetric compatibility mechanism. Ed25519 is the asymmetric attribution path and is required by hardened execution.

Proof verification requires signatures by default. `--allow-unsigned` is an explicit local-inspection option, not a production authorization mode.

12.4 Registered sources

Registered sources are the one explicit context transformation. Their values are injected under trusted top-level `source:<name>` fields. Callers may not submit those fields themselves.

A source-injected proof preserves both original and evaluated context hashes. Verification requires the exact original context and the exact evaluated context. Removing only valid top-level source additions must reproduce the original request.

12.5 Durable audit

`TarlAuditArchive` is SQLite-backed and hash-links records. It is not automatically created by `--hardened`.

A production embedding that requires durable audit must attach the archive to the same runtime and enable required persistence:

PYTHON

```
from utf.tarl.archive import TarlAuditArchive

with TarlAuditArchive("audit.db") as archive:
    runtime.set_archive(archive).set_require_audit(True)
    interpreter.attach_tarl(runtime)
    # Evaluate governed work while the archive remains open.
```

Checkpoint and verify the chain head:

BASH

```
tarl audit checkpoint --db audit.db --out audit-head.txt
tarl audit verify-chain --db audit.db --checkpoint audit-head.txt
tarl audit query --db audit.db --verdict DENY --json
```

Store the checkpoint outside the runtime host's silent rewrite authority. A locally re-linked truncated database cannot be detected without a trusted external head.

13. Hardened policy operation

13.1 Trust roots

Three Ed25519 roles are defined:

BASH

```
tarl keygen authority-issuer --key-id issuer-1 --out issuer.key
tarl keygen proof-signer --key-id signer-1 --out signer.key
tarl keygen time-authority --key-id time-1 --out time.key
```

Private files are written with restrictive permissions where the platform supports them. Prefer file flags over legacy hexadecimal key material on argv:

TEXT

```
--authority-key-file
--sign-proofs-file
--ed25519-key-file
```

13.2 Hardened CLI shape

After an authority issuer has minted a signed claim, the execution shape is:

BASH

```
thirsty run app.thirsty \
  --thirst-level governed \
  --hardened \
  --policy policy.tarl \
  --context-schema context-schema.json \
  --authority-token authority-token.json \
  --authority-key-file issuer.key.pub \
  --sign-proofs-file signer.key
```

Hardened mode fails closed unless the authority claim authenticates and every governed gate can create an Ed25519-signed proof.

13.3 Independent proof verification

BASH

```
tarl verify proof.json \
  --policy policy.tarl \
  --ed25519-key-file signer.key.pub \
  --ed25519-only \
  --context-file context.json \
  --max-age 300 \
  --now 2026-08-02T12:00:00Z \
  --replay-db replay.db \
  --revocation-store revocations.db \
  --json
```

For a registered-source proof, also provide:

TEXT

```
--evaluated-context-file evaluated-context.json
```

Verification checks proof structure, signature, policy identity, trace, context and schema coherence, temporal bounds, freshness, revocation, and replay according to the configured controls. A second verification against the same replay database is expected to fail as reuse.

13.4 Time and expiry

Temporal authority is explicit:

- CLI evaluation requires a timezone-aware `--now` for policy windows, time-bound verdicts, `CURRENT_*`, or `ELAPSED_SINCE`.
- A configured runtime clock that is missing, malformed, naive, or fails verification produces a denial. It does not fall back to host time.
- `valid_until` is an exclusive cutoff.
- A proof expires at the earlier of the effective policy cutoff or matched-rule duration.
- `on_expiry` cannot grant `ALLOW`.

Example:

BASH

```
tarl eval temporal.tarl \
  --context '{"user":{"role":"admin"}}' \
  --now 2026-08-02T12:00:00Z \
  --json
```

13.5 Revocation and replay

BASH

```
tarl revoke sha256:<policy-hash> \
  --store revocations.db \
  --reason "policy retired"

tarl revoke --store revocations.db --list --json
```

In-memory replay or revocation state is process-local and disappears on restart. Production verification uses durable stores shared by all relevant workers.

13.6 Broker and path guard

The `CapabilityBroker` is the single intended admission surface for in-language sensitive calls and out-of-language FFI, subprocess, MCP, agent, and tool adapters. An adapter that performs the effect without `broker.require(...)` is outside the governed boundary.

Filesystem operations should also use a `PathGuard` configured with allowed roots. It resolves canonical paths before the effect so traversal and symlink escapes fail closed.

13.7 ESCALATE and quorum

ESCALATE remains non-authorizing until `QuorumResolver` validates:

- the signed proof and exact policy/rule;
- the exact original request context;
- the exact evaluated context for registered-source proofs;
- passed authoritative schema binding;
- timezone-aware trusted time and signed expiry;
- maximum age and replay enforcement;
- distinct approver identities and key material;
- Ed25519 approvals over the digest of the complete proof artifact.

Approval count alone does not repair an invalid or stale proof.

14. Builds and governance loss

`thirsty build` targets are exactly:

TEXT

```
llvm-ir
llvm-obj
llvm-exe
llvm-asm
llvm-jit
js
wasm-pyodide
```

Version 0.8.6 build targets do not carry the governed Python runtime into the output. A governed module therefore refuses to build to these targets unless the user explicitly acknowledges the loss:

BASH

```
thirsty build governed_app.thirsty \
  --target js \
  --allow-governance-loss \
  --policy policy.tarl \
  --context-schema context-schema.json \
  --emit-manifest
```

The manifest records the disclosure. `--allow-governance-loss` is not a production-preservation feature; it is an explicit statement that the target does not provide the source runtime's governed-execution guarantee.

Additional target limits:

- `llvm-ir` is emitted directly.
- `llvm-obj`, `llvm-exe`, `llvm-asm`, and `llvm-jit` require appropriate LLVM or Clang tools such as `llc`, `clang`, or `lli`.
- JavaScript emission covers a practical subset rather than every interpreter behavior.
- `wasm-pyodide` emits an HTML/Pyodide wrapper that retrieves its runtime and package from external distribution sources.

Never infer policy enforcement from a build manifest that explicitly records governance loss.

15. Higher tiers

15.1 Thirst of Gods

Thirst of Gods validates a Tier 1 AST for four structural signals:

- 1 At least one fountain with an `init` method.
- 2 At least one cascade, with every cascade inside a spillage body that has a handler.
- 3 At least one spillage handler.
- 4 At least one cleanup block.

Diagnostics are `G001` through `G004`. A program that does not satisfy the complete deity contract is not executed by the Gods runtime path.

BASH

```
thirst-of-gods check program.thirstofgods
thirst-of-gods run program.thirstofgods
thirst-of-gods transpile program.thirstofgods --target thirsty
```

15.2 Shadow Thirst

Shadow Thirst evaluates mutations with six analyzers:

- 1 Plane isolation
- 2 Determinism
- 3 Resource estimation
- 4 Purity spring
- 5 Memory evaporation
- 6 Canonical convergence

A critical analyzer failure yields **REJECT**. Only noncritical failures yield **FLAGGED**. A clean analysis yields **PROMOTE**.

Canonical convergence uses structural AST comparison, optional Z3 reasoning, and seeded execute-and-compare where the program surface is safe to evaluate. It abstains conservatively when effects prevent sound comparison.

BASH

```
shadow-thirst check mutation.thirsty
shadow-thirst visualize mutation.thirsty
```

15.3 TSCG

TSCG has nine core symbols:

Symbol	Source meaning
COG	cognition or reflection
DNT	prohibition or restriction
SHD	recommendation or guidance
INV	invariant or constant
CAP	capacity or boundary
QRM	quorum or consensus
COM	command or directive
ANC	anchor or foundation
RFX	reflex or self-reference

Extended symbols are **SAFE**, **ING**, **LED**, **MUT**, **SEL**, **QRM_LINEAR**, and **QRM_STATIC**.

Expressions use **\$\$SYMBOL**, parentheses, pipeline **->**, conjunction **^**, and disjunction **||**. Pipeline has lower precedence than the combine operators.

BASH

```
tscg parse '$COG ^ $SAFE -> $LED'
tscg canonical '$COG^$SAFE->$LED'
tscg validate '$COG ^ $SAFE'
tscg checksum '$COG ^ $SAFE'
tscg list
```

Canonicalization currently normalizes the parsed spacing and structure. **checksum** hashes the exact input text, not the canonicalized text.

15.4 TSCG-B

TSCG-B frames raw UTF-8 text:

TEXT

```
magic "TSGB"      4 bytes
version           1 byte
```

```

flags          1 byte
payload length 2 bytes, big endian
payload        N bytes, UTF-8
CRC32          4 bytes
SHA-256        32 bytes

```

The minimum frame is 44 bytes. The decoder verifies magic, version, declared length, CRC32, and SHA-256. `StreamDecoder` buffers partial input and can resynchronize on frame magic.

The module also exposes opcode helper functions, but `pack_text` and the CLI frame raw UTF-8 payload bytes. Do not describe CLI frames as base-23 opcode payloads.

16. Production, security, and release receipts

16.1 What the release evidence proves

The v0.8.6 release is anchored by:

Receipt	Recorded value
Release commit	60f226a69059b5803b0035f037d09e6f7f9c45a2
Annotated tag	v0.8.6
Tag object	d89bbbc80a165ab6340cbe7855dfd6af2b086d1a
Branch CI	run 30756636768, success
Tagged validation and PyPI	run 30756676570, success
Multi-architecture container	run 30756676576, success
Local release gate	1,463 passed, 1 skipped, 22 subtests
Local line coverage	90.60 percent, with a 90 percent CI floor
Published Python artifact	clean environment installed <code>thirsty-lang==0.8.6</code> and reproduced the context matrix
Authoritative image	ghcr.io/iamsothirsty/thirsty-lang:0.8.6
Image platforms	linux/amd64, linux/arm64
GHC manifest digest	sha256:6f3f516b8e979437dd414373afe581716b8c890dc4758b8675cbbcad9b94b13c
Preserved 0.8.5 evidence manifest	4DD3E6A4CF017E3E34A25C62A398D29A6F93D279890C4CCDF2A4528C6333F8D5

Docker Hub remains stale at 0.8.1 and is not an authoritative 0.8.6 source.

16.2 What the release evidence does not prove

The receipts establish the tested and published release state. They do not:

- make an arbitrary embedding production-ready;
- provision deployment keys, durable stores, a path guard, or an audit sink;
- prove that a governance-losing build target preserves the runtime;
- satisfy independent constitutional acceptance by themselves.

The Competence Register keeps `CR-TARL-ALLOW` and `CR-TARL-DENY` at PASS. The released repair does not by itself change `CR-CONTEXT-COHERENCE` or `CR-CONTEXT-RESOLUTION-INTEGRITY`; both remain critical FAIL pending independent acceptance and continue to block the register's listed load-bearing authority.

16.3 Continuous and release gates

The maintained workflows cover:

- Python 3.11 and 3.12;
- Ruff;

- mypy over `utf`;
- pytest with at least 90 percent coverage;
- executable shipped examples;
- production acceptance;
- wheel installation and all seven console entry points;
- tag-to-package version agreement;
- distribution build and metadata validation;
- PyPI upload;
- multi-architecture GHCR build, pull, version, and demo smoke tests.

Publication uses the GitHub [release](#) environment and a scoped PyPI API token for 0.8.6. PyPI Trusted Publishing and publish attestations were not active for this release. Optional OpenPGP artifact signing is separate from TARL Ed25519 proof signing. See [SIGNING.md](#).

17. Migration from 0.8.5 to 0.8.6

Versions through 0.8.5 must not be used for load-bearing authorization with dotted context paths. In 0.8.5, an unresolved path could collapse to ordinary Boolean false. Negation or inequality could then turn resolution failure into an illegitimate `ALLOW`.

17.1 Upgrade the artifact

BASH

```
python -m pip install --upgrade --no-cache-dir thirsty-lang==0.8.6
python -c "import importlib.metadata as m; print(m.version('thirsty-lang'))"
```

17.2 Convert caller data to the one accepted representation

Before:

JSON

```
{"user.role":"admin"}
```

After:

JSON

```
{"user":{"role":"admin"}}
```

Do not implement silent flat-to-nested conversion. Reject mixed forms. If an external boundary performs an explicit migration, record the original and canonical hashes, algorithm ID, version, and conflict status before using its output as authority.

17.3 Revalidate every policy family

Permanent coverage should include:

- flat dotted key only;
- nested object only;
- simple identifier `ALLOW`;
- simple identifier `DENY`;
- flat and nested values equal;
- flat and nested values conflicting;

- missing intermediate object;
- intermediate value of the wrong type;
- missing-path equality, inequality, negation, membership, safe-function, and Boolean comparisons;
- a real resolved Boolean false;
- schema result compared with evaluator result;
- proof hash checked against the representation actually evaluated.

Exercise the matrix through direct `SafeExpr`, policy evaluation, `TarLRuntime`, `tarl eval`, context schema validation, proof obligations, proof creation and verification, broker/governed runtime, and any embedding-specific adapter.

17.4 Reissue proof and verification expectations

0.8.6 positive proofs bind the original/canonical context, representation, normalization, conflict, and schema-validation metadata. Treat positive legacy proofs lacking coherent metadata as inadmissible for load-bearing authority.

Also account for the 0.8.6 strictness changes:

- eager validation of both Boolean operands and every quantifier element;
- rejection of empty quantifier collections as authorization evidence;
- strict finite JSON and strict expression type algebra;
- per-policy derived schemas for policy overrides;
- canonical lowercase signature encoding and complete replay identity;
- explicit original/evaluated contexts for registered sources;
- timezone-aware trusted time with no configured-clock fallback;
- proof-bound quorum promotion with freshness and replay controls;
- strict temporal cutoffs and expiry.

17.5 Preserve historical evidence

The 0.8.5 reproduction files and manifest are historical security evidence. Do not rewrite them to match 0.8.6. Add new release evidence separately.

18. Troubleshooting

`tarl` or another CLI is not found

Activate the correct virtual environment. On Windows, try the direct path:

POWERSHELL

```
.\.venv\Scripts\tarl.exe --help
```

Confirm package identity with `python -m pip show thirsty-lang` and `Get-Command tarl`.

`user.role` denies even though the JSON contains an admin

Use nested JSON:

JSON

```
{"user":{"role":"admin"}}
```

A flat key named `user.role` is a representation error, not an alternate spelling.

Missing data makes an inequality or not expression deny

This is expected. Missing, wrong-type, and conflicting values do not become false. Supply valid context or redesign the policy to require explicit presence; do not weaken the evaluator.

The schema is incomplete

Static derivation cannot infer every safe-function or collection element type. Provide an explicit `--context-schema` whose paths, types, representation, and normalization match the actual evaluated context.

tarl eval requests --now

The policy uses a temporal window, time-bound verdict, `CURRENT_*`, or `ELAPSED_SINCE`. Supply a timezone-aware ISO-8601 value such as:

TEXT

```
2026-08-02T12:00:00Z
```

An unsigned proof does not verify

Signature verification is secure by default. Register the correct signer key. Use `--allow-unsigned` only for deliberate local inspection, never to promote a positive verdict to production authority.

A registered-source proof asks for two contexts

Supply the exact original request context and exact post-injection evaluated context. This proves that only registered source fields transformed the request.

A proof verifies once and fails the second time

If a replay guard or `--replay-db` is configured, the second presentation is a replay and should fail.

Hardened execution denies despite an ALLOW rule

Check all independent requirements:

- authenticated authority token;
- trusted issuer public key;
- authority grant for the capability;
- Ed25519 proof-signer private key;
- policy and admissible schema-bound context;
- trusted time for temporal use;
- broker/path guard configuration where required;
- required audit persistence if enabled.

A raw policy `ALLOW` cannot compensate for a missing authority or proof.

No audit rows appear in hardened mode

This is expected unless the embedding attached `TarlAuditArchive` to the exact runtime. Add `set_archive(...)`; if persistence is mandatory, also add `set_require_audit(True)`.

A governed build refuses

The selected target drops the governed runtime. Keep execution under the interpreter or, only when deliberate, add `--allow-governance-loss` and inspect the emitted manifest.

TARL analysis reports that Z3 is unavailable

Install the analysis extra:

BASH

```
python -m pip install "thirsty-lang[analysis]==0.8.6"
```

LLVM object, executable, assembly, or JIT build fails

Install the required `llc`, `clang`, or `lli` tools and confirm they are on `PATH`. The emitted `.ll` file can still be inspected independently.

- - locked refuses to run

Generate or update the project lockfile with `thirsty lock`, then run `thirsty audit`. Remember that 0.8.6 implements local manifest/lock integrity, not remote package retrieval.

Strict or pure mode rejects an otherwise valid program

Strict mode requires initialized bindings. Pure mode rejects `pour` and `sip`. Select the module mode that matches the intended contract.

A governed imported file behaves differently from a core import

Governed file imports inherit the caller's governance configuration so imported top-level effects and returned functions remain gated. Do not expect the core module cache path to detach governed imports from the caller.

Docker behavior does not match the published package

Verify the exact versioned GHCR image and digest. Do not use the stale Docker Hub 0.8.1 image as evidence for 0.8.6.

19. Glossary

Term	Meaning in Thirsty-Lang 0.8.6
Authority	Identity requesting governed action; hardened authority is a verified signed claim, not a raw string.
Broker	Central admission point for governed effects and external adapters.
Canonical context	Deterministic nested finite-JSON snapshot used for evaluation and hashing.
Capability	Named action such as <code>read</code> , <code>write</code> , <code>network</code> , <code>execute</code> , <code>import</code> , or governed function entry.
Conflict	Ambiguous or contradictory context representation; never resolved by choosing a value.
Context	Request data evaluated by TARL. Caller context uses nested JSON only.
Context schema	Required path/type contract checked before policy evaluation and bound into positive proofs.
Core mode	Ordinary Tier 1 execution without governed policy admission.
DENY	Fail-closed verdict; the default.
ESCALATE	Non-authorizing verdict requiring a separately verified quorum path.
Governed mode	Runtime mode in which governed calls and mapped effects require policy and authority.
Governance loss	Build output cannot preserve governed runtime semantics; must be disclosed explicitly.
Hardened mode	Posture requiring authenticated authority and Ed25519-signed proofs at every governed gate.
MISSING	Typed resolution state indicating that a path is absent; not Boolean false.
Path guard	Canonical filesystem-root confinement used before governed file effects.
Policy	TARL source containing ordered rules and metadata.
Proof	Decision artifact binding policy, context, schema, rule, verdict, time, and optional signature.
Proof obligation	Static report of the authority, policy, schema, capability, and contract evidence a program will require.
REPRESENTATION_CONFLICT	Typed resolution state for incompatible context shapes or values.
RESOLVED	The only context lookup state that carries an expression value.
Replay guard	State that makes configured proof presentations single-use.
Registered source	Trusted explicit addition under <code>source:<name></code> whose transformation is proof-bound.
Schema fingerprint	Deterministic hash of the canonical context schema.
TARL	Thirsty's Active Resistance Language, Tier 3.
TSCG	Symbolic Constitutional Grammar, Tier 5.
TSCG-B	Integrity-checked binary TSCG text frame, Tier 6.
TYPE_ERROR	Typed resolution state indicating a path crossed an invalid intermediate value.
Trusted clock	Verified timezone-aware time provider used for temporal authority.
Verdict	TARL result: <code>ALLOW</code> , <code>DENY</code> , or <code>ESCALATE</code> .

20. Source and test traceability

This table points from the major manual claims to the active implementation and regression surfaces. Paths are relative to the repository root.

Area	Source authority	Test or operational evidence
Package version, extras, scripts, included namespaces	<code>pyproject.toml</code> ; <code>src/thirsty_lang/__init__.py</code>	package smoke tests; release workflow package-smoke job
Lexer and keywords	<code>src/utf/thirsty_lang/token.py</code> ; <code>lexer.py</code>	<code>tests/test_thirsty_lang.py</code> ; <code>tests/test_language_features.py</code> ; <code>tests/test_language_fixes.py</code>
Parser, modes, precedence, fail-closed governed parsing	<code>src/utf/thirsty_lang/parser.py</code>	<code>tests/test_threat_model_parser_fail_closed.py</code> ; language regression suites
Checker, contracts, core/governed call boundary	<code>src/utf/thirsty_lang/checker.py</code>	<code>tests/test_governance_maximal.py</code>

Area	Source authority	Test or operational evidence
Interpreter, capability gates, imports, runtime values	src/utf/thirsty_lang/interpreter.py	tests/test_gate_fail_closed.py; tests/test_threat_model_file_imports.py; tests/test_review_0_8_1.py
Standard library and sensitive map	src/utf/thirsty_lang/module_system.py	tests/test_gate_fail_closed.py; tests/test_threat_model_capability_broker.py; tests/test_proof_obligations.py
Static proof obligations and manifests	src/utf/thirsty_lang/proof_obligations.py	tests/test_proof_obligations.py; tests/test_threat_model_build_outputs.py
Main CLI and build targets	src/utf/thirsty_lang/cli.py	tests/test_cli.py; tests/test_cli_build.py; tests/test_threat_model_build_outputs.py
TARL parser and SafeExpr	src/utf/tarl/core.py	tests/test_tarl.py; tests/test_peer_review_0_8_1_tarl_regressions.py
Context representation and typed resolution	src/utf/tarl/context.py	tests/test_tarl_context_resolution_integrity.py; tests/fixtures/tarl_context_coherence/
Schema parsing and validation	src/utf/tarl/schema.py	tests/test_tarl_context_security_boundaries.py; tests/test_threat_model_context_schema.py
TARL runtime, cache, sources, proof creation	src/utf/tarl/runtime.py	tests/test_tarl_context_security_boundaries.py; tests/test_tarl_proof.py; tests/test_threat_model_failclosed.py
Proof structure and verdict lattice	src/utf/tarl/spec.py	tests/test_tarl_proof.py; tests/test_tarl_temporal.py
Proof verification	src/utf/tarl/verifier.py	tests/test_tarl_proof.py; tests/test_threat_model_proof_strictness.py; tests/test_threat_model_replay.py
Durable replay and revocation	src/utf/tarl/durable.py	tests/test_durable_state.py
Audit archive	src/utf/tarl/archive.py	tests/test_threat_model_audit_chain.py; tests/test_production_acceptance.py
Authority and deployment keys	src/utf/tarl/authority.py; keystore.py	tests/test_threat_model_authority.py; tests/test_production_acceptance.py
Trusted time	src/utf/tarl/clock.py; temporal logic in runtime.py and verifier.py	tests/test_threat_model_clock.py; tests/test_tarl_temporal.py
Broker and path confinement	src/utf/tarl/broker.py; pathguard.py	tests/test_threat_model_broker.py; tests/test_broker_unified_gate.py; tests/test_threat_model_pathguard.py
ESCALATE and quorum	src/utf/tarl/escalation.py	tests/test_threat_model_lint_quorum.py
TARL CLI and LSP	src/utf/tarl/cli.py; lsp.py	tests/test_cli_tarl_hardening.py; TARL LSP tests
Thirst of Gods	src/utf/thirst_of_gods/core.py; cli.py	tests/test_thirst_of_gods.py; tests/test_verifiers.py
Shadow Thirst	src/utf/shadow_thirst/core.py; cli.py	tests/test_shadow_thirst.py; tests/test_verifiers.py
TSCG	src/utf/tscg/core.py; cli.py	tests/test_tscg.py
TSCG-B	src/utf/tscg_b/core.py; cli.py	tests/test_tscg_b.py
Production acceptance	Runtime sources above	tests/test_production_acceptance.py; .github/workflows/smoke.yml
Python publication	pyproject.toml; .github/workflows/release.yml	run 30756676570; clean PyPI install receipt
Container publication	Dockerfile; .github/workflows/docker.yml	run 30756676576; GHCR digest receipt
Security register and adversary model	docs/STATUS.md; docs/THREAT_MODEL.md	C001-C073 test references and preserved 0.8.5 evidence manifest

21. Validation commands

For a source checkout with development extras installed:

BASH

```
ruff check src tests
mypy -p utf
python -m pytest tests/ -q --cov=utf --cov-report=term-missing --cov-fail-under=90
python -m build
python -m twine check dist/*
```

Run narrow security matrices during TARL work:

BASH

```
python -m pytest tests/test_tarl_context_resolution_integrity.py -q
python -m pytest tests/test_tarl_context_security_boundaries.py -q
python -m pytest tests/test_tarl_proof.py -q
python -m pytest tests/test_threat_model_lint_quorum.py -q
python -m pytest tests/test_production_acceptance.py -q
```

Passing repository tests is necessary release evidence, but published-artifact acceptance still requires a clean install of the exact version and execution of the release-specific matrix.

22. Where to continue

- Learn exact syntax in [GRAMMAR.md](#).
- Read semantic contracts in [LANGUAGE_SPEC.md](#).
- Check whether a claim is implemented in [STATUS.md](#).
- Review attacker paths in [THREAT_MODEL.md](#).
- Understand the broker and authority flow in [governance_model.md](#).
- Prepare an embedding with [PRODUCTION_DEPLOYMENT.md](#).
- Verify release provenance with [SHIPPING.md](#) and [SIGNING.md](#).
- Review release history in [CHANGELOG.md](#) and [CONTINUITY_MAP.md](#).

Security reports should be sent privately to FounderOfFTP@thirstysprojects.com as described in [SECURITY.md](#).

- **Document identity:** Thirsty-Lang UTF 101 (Universal Thirsty Family) for release 0.8.6
- **Canonical context representation:** `tarl.context.nested-json.v1`
- **Default governed verdict:** `DENY`

Published package boundary: `utf`, `utf.*`, and compatibility package `thirsty_lang`

PART 02 / NORMATIVE-SPECIFICATION

Language Family Specification

Maintained source: docs/LANGUAGE_SPEC.md



Version: 0.8.6 **Copyright:** 2026 Thirsty's Projects LLC (Apache 2.0)

This is the concise normative family specification. For the integrated tutorial, CLI encyclopedia, security contract, deployment guidance, and source traceability, see [THIRSTY_LANG_UTF_101.md](#).

Overview

Thirsty-Lang (Universal Thirsty Family - UTF) is a 6-tier governance-first language stack. Each tier adds progressive capabilities while maintaining default-DENY governance at every gate.

Tier 1 - Thirsty-Lang (Core)

The base language with water-metaphor syntax.

Keywords

Keyword	Token	Purpose
drink	DRINK	Variable declaration
mut	MUT	Mutable modifier
pour	POUR	Print/emit value
sip	SIP	Read input
glass	GLASS	Function declaration
return	RETURN	Return from function
thirsty	THIRSTY	If (conditional)
hydrated	HYDRATED	Else (conditional)
thirst	THIRST	Guard/when expression
quench	QUENCH	Default in guard
refill	REFILL	Loop (while/for)
times	TIMES	Repeat n times
module	MODULE	Module declaration
import	IMPORT	Import module
as	AS	Import alias
and	AND	Logical AND
or	OR	Logical OR
not	NOT	Logical NOT
true	BOOL_TRUE	Boolean true
false	BOOL_FALSE	Boolean false
none	NONE	Null value

Operators

Operator	Token	Purpose
+	PLUS	Addition / string concat
-	MINUS	Subtraction
*	STAR	Multiplication
/	SLASH	Division
%	PERCENT	Modulo
==	EQEQ	Equality
!=	NE	Not equal
<	LT	Less than
>	GT	Greater than
<=	LE	Less or equal
>=	GE	Greater or equal
=	ASSIGN	Assignment
>	PIPE	Pipe operator
->	ARROW	Return type arrow
()	LPAREN/ RPAREN	Grouping / calls
{ }	LBRACE/ RBRACE	Blocks
[]	LBRACKET/ RBRACKET	Reservoir literals and indexing (xs[i])
:	COLON	Type annotation
,	COMMA	Separator

Syntax Examples

THIRSTY

```
module hello: core

glass greet(name: String) -> String {
  return "hello, " + name + "!"
}

drink main = greet("thirsty world")
pour main
```

THIRSTY

```
module loop: core

// while form
drink mut i = 0
refill (i < 3) { pour i i = i + 1 }

// for-each form
refill (x in [1, 2, 3]) { pour x }

// C-style form - the loop counter is implicitly mutable
refill (drink x = 0; x < 10; x = x + 1) {
  pour x
}

// repeat N times
times 3 {
  pour "hydrate"
}
```

Anonymous functions (lambdas) are first-class values:

THIRSTY

```
module lambda: core

drink add = glass(a, b) { return a + b }
pour add(2, 3)
```

THIRSTY

```
module branches: core

thirsty (x > 0) {
  pour "positive"
} hydrated thirsty (x < 0) {
  pour "negative"
} hydrated {
  pour "zero"
}
```

Types

- Int - Integer values
- Float - Floating-point values
- String - String values
- Bool - Boolean values
- NoneType - Null type

CLI Commands

Command	Description
thirsty run <file>	Execute a .thirsty file
thirsty repl	Start interactive REPL
thirsty build <file> --target js	Build to JavaScript
thirsty prove <file> --policy <policy>	Emit static proof obligations without executing effects
thirsty explain-denial <file> --policy <policy>	Explain unmet static governance obligations
thirsty fmt <files>	Format .thirsty files
thirsty new <name>	Scaffold new project
thirsty govern <file>	Governance operations
thirsty add <package>	Add dependency
thirsty audit	Audit dependencies
thirsty lock	Generate lockfile
thirsty doctor	Project health check
thirsty lsp	Start the TARL language server
thirsty docs	Generate documentation

Tier 2 - Thirst of Gods

Applies a structural AST contract over the Tier 1 object-oriented, cascade, and structured-error constructs before running the ordinary interpreter.

Keywords

Keyword	Token	Purpose
fountain	FOUNTAIN	Class declaration
cascade	CASCADE	Async function
spillage	SPILLAGE	Try block
cleanup	CLEANUP	Resource cleanup
finally	FINALLY	Finalizer
error	ERROR	Error handler
throw	THROW	Throw error
this	THIS	Self reference
new	NEW	Instantiate class

Syntax Examples

THIRSTY

```
fountain Counter {
  drink count: Int = 0

  glass increment() {
    this.count = this.count + 1
    return this.count
  }
}

drink c = new Counter()
c.increment()
```

THIRSTY

```
spillage {
  drink result = risky_operation()
} error (e) {
  pour "Error: " + e
}
```

Tier 3 - T.A.R.L. (Thirsty's Active Resistance Language)

A policy-as-code engine with default-DENY governance.

Keywords

Keyword	Token	Purpose
policy	POLICY	Policy declaration
when	WHEN	Policy condition
ALLOW	ALLOW	Allow verdict
DENY	DENY	Deny verdict
ESCALATE	ESCALATE	Escalate verdict

Syntax

TARL

```
policy example:
  when user.role == "admin" => ALLOW
  when user.ip in blacklist => DENY
  when true => ESCALATE
```

TarlVerdict values: ALLOW, DENY, ESCALATE. Default-DENY applies when no rule matches.

Authoritative context contract

T.A.R.L. 0.8.6 has exactly one caller-context representation: `tarl.context.nested-json.v1`. Dotted identifiers traverse nested JSON objects:

JSON

```
{"user": {"role": "admin"}}
```

A flat key such as `{"user.role": "admin"}` is not normalized. Flat dotted keys, duplicate JSON keys, mixed flat/nested forms (even when their values are equal), non-string object keys, non-finite numbers, cycles, and values outside the finite JSON domain fail closed.

Path resolution preserves four distinct states: `RESOLVED(value)`, `MISSING`, `TYPE_ERROR`, and `REPRESENTATION_CONFLICT`. Only `RESOLVED(value)` may enter ordinary comparison, membership, negation, safe functions, or boolean algebra. Missing is not false; invalid is not false; unresolved is not evidence.

A raw evaluator may report `ALLOW` for diagnostic use. Load-bearing positive authority additionally requires a complete context schema that passed against the exact evaluated nested representation and is bound into the proof. The proof records the original and canonical/evaluated context hashes, representation identifier, normalization algorithm identifier and version, conflict status, schema fingerprint/representation/result, policy hash, matched rule and condition, verdict, trace, evaluation time, and applicable expiry.

Registered sources are the only supported context transformation. The proof binds both the original request and exact source-enriched evaluated context; verification requires both, and removing only permitted top-level `source:<name>` additions must reproduce the original request.

T.A.R.L. condition validation is eager and fail-closed. Both operands of `and/or` are evaluated, and `ALL(collection, item -> predicate)` and `ANY(collection, item -> predicate)` validate every collection element before accepting the aggregate result. Missing paths, wrong intermediate types, or incompatible predicate values therefore cannot be hidden by boolean or iteration order. An empty quantifier collection fails closed; it is not authorization evidence.

Comparisons are type-strict. Integers and floats interoperate as numbers; strings never coerce to numbers, including in ordering and equality. Numeric operands and every arithmetic result must be finite. `NaN`, infinities, and overflow to infinity are evaluation errors that deny the affected policy decision. Quantifier binder names beginning with `__tarl_` are reserved for trusted runtime state and are rejected during parse and evaluation.

Temporal metadata is strict. Invalid `valid_from/valid_until` timestamps, `for:` or `if_unresolved_after` durations, and expiry verdicts reject the policy. `on_expiry: ALLOW` is forbidden. `valid_until` is an exclusive cutoff, and a matched verdict expires at the earlier of that effective policy cutoff or its rule-level `for:` duration.

Independent verification rejects unsigned proofs by default and can enforce exact policy/context/schema bindings, Ed25519-only attribution, freshness, expiry, revocation, and durable replay prevention. A configured trusted clock must return a timezone-aware verified value; it never silently falls back to host time after failure. Quorum promotion requires an independently verified, schema-passed, signed `ESCALATE` proof plus distinct Ed25519 approvals over the complete proof digest and explicit request/time/freshness/replay bindings.

Tier 4 - Shadow Thirst

Mutation analysis and invariant verification.

Keywords

Keyword	Token	Purpose
shadow	SHADOW	Shadow block
invariant	INVARIANT	Invariant declaration
canonical	CANONICAL	Canonical form
promote	PROMOTE	Promote block
reject	REJECT	Reject block

Built-in Analyzers

The analyzers parse each `shadow` / `invariant` / `canonical` block with Thirsty-Lang's own lexer + parser and reason over the resulting **AST** (with a lexical fallback when a block does not parse):

- 1 **Plane Isolation** - Walks the shadow block for writes into `canonical_*` bindings or calls into the canonical plane
- 2 **Determinism** - Flags *calls* to non-deterministic functions (`now()`, `rand()`, `uuid()`, ...), not like-named variables
- 3 **Resource Estimation** - Estimates CPU/memory from loop, call, and allocation nodes
- 4 **Purity Spring** - Checks the invariant block for impure calls / output statements
- 5 **Memory Evaporation** - Counts allocation nodes (`new`, reservoir literals, floods)
- 6 **Canonical Convergence** - Compares shadow and canonical via structural AST equivalence (alpha-renamed shape + return arity)

Tier 5 - TSCG (Thirsty Symbolic Constraint Grammar)

Symbolic security expressions with 9 core symbols.

Symbols

Symbol	Name	Purpose
COG	Cognition	Cognitive capability
DNT	Do Not Track	Privacy constraint
SHD	Shield	Protection boundary
INV	Invariant	Invariant condition
CAP	Capability	Capability grant
QRM	Quorum	Consensus requirement
COM	Communication	Communication channel
ANC	Anchor	Trust anchor
RFX	Reflexive	Self-reference

Operators

- Pipeline: `->`
- AND-combine: `^`
- OR-combine: `||`

For boolean operands, `^` evaluates as logical **AND** (conjunction - *not* XOR, despite the glyph) and `||` evaluates as logical OR. Both operands must be `bool`: a mixed `bool/non-bool` combine is a type error (rejected statically and refused at runtime) so a malformed predicate fails closed rather than coercing. For structured operands, `^/||` keep their runtime composition behavior: dictionaries merge and reservoirs concatenate.

All expressions are SHA-256 canonicalized.

Tier 6 - TSCG-B (Thirsty Symbolic Constraint Grammar - Binary)

Binary frame protocol for TSCG expressions.

Frame Format

- Magic bytes: **TSGB** (4 bytes)
- CRC32 integrity check (4 bytes)
- SHA-256 payload verification (32 bytes)
- Automatic resynchronization for multi-frame transport

License

Copyright 2026 Thirsty's Projects LLC

Licensed under the Apache License, Version 2.0 (the "License"); you may not use this file except in compliance with the License. You may obtain a copy of the License at:

<http://www.apache.org/licenses/LICENSE-2.0>

Unless required by applicable law or agreed to in writing, software distributed under the License is distributed on an "AS IS" BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied. See the License for the specific language governing permissions and limitations under the License.

PART 03 / NORMATIVE-GRAMMAR

Tier 1 Source Grammar

Maintained source: docs/GRAMMAR.md



Edition: Thirsty-Lang v0.8.6

Scope: syntax accepted by `utf.thirsty_lang.lexer.Lexer` and `utf.thirsty_lang.parser.Parser`

Canonical handbook: [Thirsty-Lang UTF 101](#)

This is the source-authoritative EBNF-style reference for the Tier 1 parser. It describes accepted syntax, not every semantic or governance guarantee. The lexer, parser, and their regression tests govern if an older example conflicts with this document.

Several words used by higher tiers are reserved by the Tier 1 lexer. Some also have a Tier 1 parse form: `shield` and `detect` wrap a block; `sanitize` and `armor` wrap an expression; `morph` declares a callable transform; and `defend` records a strategy form. Their presence alone does not provide a higher-tier security guarantee. See the [governance model](#).

Notation

- `"..."` denotes a literal keyword or symbol.
- `( ... )` groups grammar terms.
- `[ ... ]` is optional.
- `{ ... }` repeats zero or more times.
- `A | B` is alternation.
- `/ ... /` is a note about parsing or semantics.

1. Lexical grammar

1.1 Identifiers and member access

EBNF

```
identifier_start = alphabetic | "_" ;
identifier_rest  = alphanumeric | "_" ;
identifier       = identifier_start { identifier_rest } ;

member_access    = expression "." member_name ;
```

`alphabetic` and `alphanumeric` follow Python's Unicode-aware `str.isalpha()` and `str.isalnum()` behavior. A dot is **not** part of an identifier. For example, `user.role` lexes as `identifier`, `.`, `identifier` and the parser builds a member-access path. After a dot, a reserved word whose spelling is a valid identifier may be used as the member name, such as `log.error`.

1.2 Numbers

EBNF

```

digit      = "0".."9" ;
hex_digit  = digit | "a".."f" | "A".."F" ;
oct_digit  = "0".."7" ;
binary_digit = "0" | "1" ;

decimal_int = digit { digit } ;
hex_int     = ( "0x" | "0X" ) hex_digit { hex_digit } ;
octal_int   = ( "0o" | "0O" ) oct_digit { oct_digit } ;
binary_int  = ( "0b" | "0B" ) binary_digit { binary_digit } ;
integer     = decimal_int | hex_int | octal_int | binary_int ;

exponent   = ( "e" | "E" ) [ "+" | "-" ] digit { digit } ;
float      = decimal_int "." digit { digit } [ exponent ]
           | decimal_int exponent ;

```

3. is an integer followed by a dot, not a float. A fractional form therefore requires at least one digit after the dot.

1.3 Strings and escapes

EBNF

```

string      = double_quoted_string | single_quoted_string ;
double_quoted_string = '"' { string_character | escape } '"' ;
single_quoted_string = "'" { string_character | escape } "'" ;

escape      = "\\n" | "\\t" | "\\0" | "\\\" | "\\\"" | "\\'" ;

```

The decoded escapes in v0.8.6 are newline (`\n`), tab (`\t`), NUL (`\0`), backslash (`\\`), double quote (`\"`), and single quote (`\'`). `\r` and hex escapes such as `\x41` are not implemented escape forms. For compatibility, the current lexer drops the backslash from an unknown escape and retains the following character; code should not rely on that fallback. Quoted strings may span physical lines.

1.4 Comments and whitespace

EBNF

```

line_comment = "//" { character - newline } ( newline | EOF ) ;
block_comment = "/*" { character | block_comment } "*/" ;
whitespace  = " " | "\t" | "\r" | "\n" ;

```

Block comments may nest. Whitespace, including newlines, separates tokens but does not itself produce a statement-termination token.

2. Program structure and types

EBNF

```

program      = [ module_header ] { statement } EOF ;

module_header = "module" identifier ":" module_mode ;
module_mode  = "core" | "governed" | "strict" | "pure" ;

import_stmt  = "import" string [ "as" identifier ] [ ";" ] ;

type_name    = identifier ;
parameter    = identifier [ ":" type_name ] ;
parameters   = "(" [ parameter { "," parameter } ] ")" ;

```

The module header, when present, must be first. Imports are ordinary statements and may occur wherever the parser accepts a statement.

Type annotations currently consume exactly one identifier. The checker recognizes the built-in names `Int`, `Float`, `Bool`, `String`, `Void`, `Any`, and `Error`. Function-type, tuple-type, and bracketed generic syntax are not accepted in a Tier 1 annotation by the v0.8.6 parser.

In `strict` mode, `drink` and `let` bindings require an initializer at runtime. In `pure` mode, side-effecting `pour` and `sip` operations are rejected at runtime.

3. Expressions

The productions below are ordered from lowest to highest binding power.

EBNF

```

expression      = assignment ;

assignment      = low_pipe [ "=" assignment ] ;

low_pipe        = logical_or { ( "|" | ">" ) logical_or } ;

logical_or      = logical_and { "or" logical_and } ;
logical_and     = logical_not { "and" logical_not } ;
logical_not     = { "not" } comparison ;

comparison      = additive
                  { ( "==" | "!=" | "<" | ">" | "<=" | ">=" ) additive } ;

additive        = multiplicative { ( "+" | "-" ) multiplicative } ;
multiplicative  = high_combine { ( "*" | "/" | "%" ) high_combine } ;

high_combine    = numeric_unary { ( "->" | "|" | "^" ) numeric_unary } ;
numeric_unary   = "-" numeric_unary | postfix ;

postfix         = primary
                  { call_suffix | subscript_suffix | member_suffix } ;
call_suffix     = "(" [ argument_list ] ")" ;
subscript_suffix = "[" expression "]" ;
member_suffix   = "." member_name ;
argument_list   = expression { "," expression } ;

primary         = literal
                  | identifier
                  | "this"
                  | "(" expression ")"
                  | array_literal
                  | lambda_expression
                  | guard_expression
                  | quenched_expression
                  | flow_expression
                  | new_expression
                  | cascade_expression ;

literal         = integer | float | string | "true" | "false" | "none" ;
array_literal   = "[" [ argument_list ] "]" ;

lambda_expression = "glass" parameters [ "->" type_name ] block ;

guard_expression = "thirst" expression "quench" expression ;
quenched_expression = "quenched" [ "(" [ expression ] ")" ] ;

flow_expression = ( "flood" | "drip" | "evaporate" | "condense"
                  | "sanitize" | "armor" ) expression ;

new_expression  = "new" identifier [ "(" [ argument_list ] ")" ] ;
cascade_expression = "cascade" expression ;

```

`member_name` is an identifier-shaped token; it may be a reserved word after the dot. Method calls use the same postfix chain as ordinary calls, for example `object.method(arg)[0]`.

There is no unary `+` operator. Logical negation is spelled `not`; a bare `!` is a lexer error, while `!=` is the supported inequality operator. Adjacent string literals are not an implicit concatenation form, and the parser does not implement the formerly documented `identifier { field = value }` struct literal.

Exact precedence and associativity

Binding power	Operators/forms	Associativity
1 (lowest)	assignment =	right
2	low pipe , >	left
3	or	left
4	and	left
between 4 and 5	prefix not	right/prefix
5	==, !=, <, >, <=, >=	left, one shared level
6	+, -	left
7	*, /, %	left
8	->, , ^	left
prefix at 8	unary -	right/prefix
9 (highest operators)	call (), member ., subscript []	left/postfix

This table mirrors `Parser._precedence_map()`. In particular, equality and ordering comparisons share a level, and binds more tightly than `or`, and `not a == b` means `not (a == b)`.

4. Statements and blocks

EBNF

```

statement      = variable_decl
                | for_stmt
                | pour_stmt
                | sip_stmt
                | if_stmt
                | refill_stmt
                | times_stmt
                | return_stmt
                | import_stmt
                | block
                | function_decl
                | fountain_decl
                | spillage_stmt
                | cleanup_stmt
                | throw_stmt
                | security_block
                | morph_decl
                | defend_decl
                | enum_decl
                | struct_decl
                | interface_decl
                | mutation_decl
                | symbol_stmt
                | pipe_block_stmt
                | expression_stmt ;

block          = "{" { statement } "}" ;

variable_decl  = "drink" [ "mut" ] identifier
                [ ":" type_name ] [ "=" expression ] [ ";" ]
                | "let" identifier
                [ ":" type_name ] [ "=" expression ] [ ";" ]
                | identifier "!=" expression [ ";" ] ;

for_stmt       = "for" identifier "in" expression block
                | "for" "(" identifier "in" expression ")" block ;

pour_stmt      = "pour" expression [ ";" ] ;
sip_stmt       = "sip" expression [ ";" ] ;

if_stmt        = "thirsty" "(" expression ")" block
                [ "hydrated" ( if_stmt | block ) ] ;

refill_stmt    = "refill" "(" identifier "in" expression ")" block
                | "refill" "(" expression ")" block

```



```

        | c_style_refill ;

c_style_refill  = "refill" "(" c_style_init expression ";" expression ")" block ;
c_style_init   = declaration_init | expression ";" ;
declaration_init = ( "drink" [ "mut" ] identifier
                  | "let" identifier )
                  [ ":" type_name ] [ "=" expression ] [ ";" ] ;

times_stmt     = "times" expression block ;
return_stmt    = "return" [ expression ] [ ";" ] ;
throw_stmt     = "throw" expression [ ";" ] ;

pipe_block_stmt = ( "|" | ">" ) expression ";" ;
expression_stmt = expression [ ";" ] ;

```

Most statement forms accept, but do not require, a trailing semicolon. There is no automatic semicolon insertion because newlines are discarded by the lexer; the parser instead recognizes the end of an expression or declaration from the following token. Empty `;` is not a statement.

Semicolons remain required in two places:

- 1 the condition separator in a C-style `refill` and the first separator when its initializer is an expression; and
- 2 the terminator of a statement-level pipe block.

A `drink` or `let` initializer selects the C-style `refill` form directly, so its first separator is optional in the current parser. The canonical spelling still includes both separators: `refill (drink i = 0; i < 3; i = i + 1) { ... }`.

One dispatch edge is also part of the v0.8.6 behavior: a statement beginning with `sanitize`, `armor`, `cascade`, or `new` bypasses the generic expression- statement parser. A trailing semicolon is therefore not consumed and would be read as an invalid empty statement; omit it on those four direct forms.

5. Declarations and governed forms

5.1 Functions and contracts

EBNF

```

function_decl  = "glass" identifier parameters [ "->" type_name ]
               { contract_clause } block ;

contract_clause = "requires" expression
               | "ensures" expression
               | "invariant" expression ;

```

Named and anonymous functions use `->` for return annotations. The formerly documented `: type` return form is not accepted. Contract clauses may appear in any order; canonical code uses each kind at most once.

A function with a `requires`, `ensures`, or `invariant` clause is a governed function. In governed mode its contracts are evaluated on every call, and entry also requires the configured T.A.R.L. policy and authority path. See [Runtime Enforcement](#).

5.2 Fountains

EBNF

```

fountain_decl  = "fountain" identifier "{" { fountain_member } "}" ;

fountain_member = function_decl
               | "drink" [ "mut" ] fountain_field [ ";" ]
               | fountain_field [ ";" ] ;

fountain_field = identifier [ ":" type_name ] [ "=" expression ] ;

```

Fountain inheritance, mixin lists, and interface lists after the fountain name are not supported by the v0.8.6 parser. A constructor is an ordinary method named `init`, written `glass init(...) { ... }`.

5.3 Error handling and cleanup

EBNF

```
spillage_stmt    = "spillage" block
                  { "error" [ "(" identifier ")" ] block } ;

cleanup_stmt     = "cleanup" block "finally" block ;
```

The optional identifier in an `error(name)` handler receives the thrown value.

5.4 Data and interface declarations

EBNF

```
enum_decl        = "enum" identifier
                  "{" [ identifier { "," identifier } ] "}" ;

struct_decl      = "struct" identifier "{" { struct_field [ "," ] } "}" ;
struct_field     = identifier [ ":" type_name ] ;

interface_decl   = "interface" identifier
                  "{" { interface_signature [ "," ] } "}" ;
interface_signature = identifier parameters [ "->" type_name ] ;
```

Enum variants are names only; payload-bearing variants are not supported. Interface signatures contain named parameters (with optional type annotations), an optional `->` return type, and no body. The Tier 1 parser does not support interface inheritance or an `implements` clause on fountains; this syntax does not claim either feature.

5.5 Higher-tier marker forms accepted by Tier 1

EBNF

```
morph_decl       = "morph" identifier parameters block ;

security_block    = ( "shield" | "detect" ) block ;

defend_decl      = "defend" identifier [ "(" identifier ")" ]
                  "{" { expression [ "," ] } "}" ;

mutation_decl     = "mutation" identifier "{"
                  "validated_canonical" "{"
                  { mutation_section }
                  "}"
                  "}" ;

mutation_section  = "shadow" block
                  | "invariant" block
                  | "canonical" block ;

symbol_stmt      = "symbol" identifier [ ":" ] ;
```

`morph` has parameters and a body but no return annotation. `shield` and `detect` take a block directly; neither accepts a parenthesized condition.

6. Lexically reserved words

TEXT

```
drink, let, for, strict, pure,
pour, sip, thirsty, hydrated, thirst, quench, refill, times,
glass, reservoir, well, of, flood, drip, evaporate, condense, fountain,
return, parched, quenched, empty, mut, in, import, from, as,
shield, sanitize, armor, morph, detect, defend, cascade, this, new,
public, private, await, spillage, cleanup, finally, error, throw,
policy, when, allow, deny, escalate, mutation, validated_canonical,
invariant, shadow, canonical, promote, reject, governed, requires, ensures,
enum, struct, interface, symbol, module, core, and, or, not,
true, false, none
```

This is the complete v0.8.6 [KEYWORDS](#) set. Reservation does not mean that each word starts a Tier 1 parser production. Some tokens exist for higher-tier and forward-compatibility work.

7. Source anchors

The implementation points that define this edition are:

- [src/utf/thirsty_lang/lexer.py](#) - token boundaries, number and string forms, comments, and escape decoding;
- [src/utf/thirsty_lang/token.py](#) - token kinds and reserved words;
- [src/utf/thirsty_lang/parser.py](#) - statements, declarations, expressions, precedence, and associativity;
- [tests/test_lexer.py](#), [tests/test_parser_coverage.py](#), and [tests/test_parser_precedence.py](#) - executable grammar regressions.

PART 04 / IMPLEMENTED-FEATURE-REFERENCE

Feature and Tooling Reference

Maintained source: docs/FEATURES.md



Feature-oriented reference for Thirsty-Lang 0.8.6. See [STATUS.md](#) for the authoritative capability-to-test map and [THREAT_MODEL.md](#) for security acceptance status. The [canonical Thirsty-Lang UTF 101 manual](#) is the integrated tutorial, specification, operations, and traceability reference.

Thirsty-Lang is a **governance-first language family**. It is not one language but a layered stack: a small imperative core whose execution is *governed*, a policy engine that decides what code may do, and a set of semantic verifiers that *prove* properties of code rather than pattern-matching its text.

1. Language Core (thirsty, .thirsty)

A tree-walking interpreter with a recursive-descent + Pratt parser, a static checker, a type system, and a formatter.

1.1 Declarations

Form	Meaning
drink x = expr	immutable binding
drink mut x = expr	mutable binding (reassignable via x = ...)
drink x: Int = 5	binding with a type annotation
glass f(a, b) -> Ret { ... }	function (first-class, TCO-eligible)
glass f(x) requires P ensures Q invariant I { ... }	governed function (§2)
fountain C { field: T glass init(self){...} glass m(self,...){...} }	class with fields, constructor, methods
enum, struct, interface, symbol, morph, defend	additional type / transform / strategy declarations

1.2 Statements

- Conditionals:** thirsty (c) { ... } hydrated thirsty (c) { ... } hydrated { ... } (if / else-if / else).
- Loops:** refill (item in iterable) { ... } (for-each), refill (cond) { ... } (while). Loop bodies accumulate into mutable bindings.
- I/O:** pour expr (output), sip target (input).
- Imports:** import "path" as alias.
- Errors:** spillage { ... } error { ... } (try/catch), cleanup { ... } finally { ... }, throw expr.
- Async:** cascade expr - schedules on a thread pool and **awaits**, yielding the value (not a future).
- Security blocks:** shield / sanitize / armor / morph / detect / defend.

1.3 Expressions & operators

- Literals: `int`, `float`, `string`, `bool`, `none`, `error`, `quenched` (optional), and **array/reservoir literals** [1, 2, 3] (real Python `list`).
- Arithmetic `+` `-` `*` `/` `%`; comparison `==` `!=` `<` `>` `<=` `>=`; logical `and` `or` `not`; unary `-` and keyword negation `not` (a bare `!` is invalid).
- **OOP**: member read `obj.field`, member write `obj.field = v`, method dispatch `obj.method(args)`.
- `new Class(args)`, function calls, pipe `|`, guard `thirst(e) quench(c)`, and the TSCG pipeline/combine operators.

1.4 Built-in functions

`length`, `size`, `contains`, `get`, `split`, `abs`, `min`, `max`, `push`, `pop`, `flood`, `condense`, `evaporate`, `strain`, `transmute`, `distill`, `print`, `pour`.

1.5 Modes

A module header declares the enforcement mode:

TEXT

```
module myapp: core      // ordinary execution
module myapp: governed // governance enforcement active
```

2. Governance (the differentiator)

Governance is **runtime-enforced**, **deny-by-default**, and a **hard floor**: a `GovernanceViolation` is *not* catchable by `spillage`. This is what separates Thirsty-Lang from "contracts as assertions" - a governed program cannot talk its way out of its own rules.

2.1 Design-by-contract

Clause	When checked
requires P	at call entry
ensures Q	at call exit , with <code>result</code> bound to the return value
invariant I	at both entry and exit

Contracts apply to **free functions and methods** alike.

TEXT

```
module bank: governed

glass withdraw(balance, amount)
  requires amount > 0
  requires amount <= balance
  ensures result == balance - amount
{
  return balance - amount
}
```

2.2 Capability gates

In `governed` mode, governed function calls and the explicitly mapped sensitive operations - including `import`, `I/O`, `filesystem`, `HTTP/network`, `environment`, `process`, `logging`, `testing`, and `SQLite` operations - are routed through an attached **T.A.R.L. policy runtime** (`evaluate_with_proof`). A non-`ALLOW` verdict denies the operation before it takes effect. Do not infer that every standard-library function is sensitive: `module_system.py` is the authority for the current capability map.

2.3 Cryptographic proofs

Every gated decision produces a `TarlProof` record. It is unsigned unless a runtime signing key is configured. HMAC-SHA256 provides symmetric integrity; Ed25519 provides asymmetric signer attribution and is required by hardened mode. Independent verification rejects unsigned proofs by default. A denial proof is attached to `GovernanceViolation`; durable audit storage additionally requires an attached archive.

2.4 Temporal governance

T.A.R.L. evaluates **time-windowed policies** (`valid_from` / `valid_until`, durations). Production callers must supply a verified, timezone-aware trusted clock; an explicitly configured clock that is missing, malformed, or fails verification denies without falling back to host time. The CLI requires a zoned `--now` for temporal evaluation.

2.5 Static parity (checker)

- **E053** - calling a governed function from a `core` module.
- **Hoisting** - top-level function/class names are pre-declared, so forward references and mutual recursion resolve.
- Contract predicates are checked for resolvable names.

3. Semantic Verifiers

3.1 Shadow Thirst (shadow-thirst) - safe mutation & promotion

Parses a `mutation { validated_canonical { shadow{...} invariant{...} canonical{...} } }` and runs **six AST-based analyzers** to a **PROMOTE / FLAGGED / REJECT** verdict:

- 1 **PlaneIsolation** - the shadow plane never writes canonical state.
- 2 **Determinism** - a taint dataflow (`EffectAnalysis`) that follows non-determinism (`now`, `rand`, `uuid`, ...) **through aliases to a fixpoint**, so aliasing `now` into a variable and calling it is still caught.
- 3 **ResourceEstimator** - CPU cost estimated from AST structure.
- 4 **PuritySpring** - the invariant block is side-effect free.
- 5 **MemoryEvaporation** - peak memory from allocation-producing nodes.
- 6 **CanonicalConvergence** - three layers (see below).

Layered convergence answers "does the shadow compute the same thing as the validated canonical?":

Layer	Mechanism	Guarantee
Structural	alpha-renamed AST equality	sufficient proof of equivalence
Z3 symbolic (<i>opt</i>)	translate return values to Z3 over shared inputs	proof for all inputs, or a counterexample
Execute-and-compare	run both over seeded inputs in a sandbox	observed equivalence, or the diverging input

Equal-but-differently-shaped blocks (`x + x` $\square$ `x * 2`) now **promote**; subtly different ones (`x + 1` vs `x + 2`) **reject with a witness**. The sampling layer abstains on blocks with observable effects (return-value equality is unsound there). Z3 requires `pip install thirsty-lang[analysis]`.

CLI: `shadow-thirst check <file>`, `shadow-thirst visualize <file>` (Mermaid).

3.2 Thirst of Gods (thirst-of-gods) - deity contracts

Structural AST validation of four signals, emitting diagnostics G001-G004:

- a fountain with an `init` method,

- **every** cascade **lexically inside a spillage handler** (a real error consumer - not mere co-presence),
- spillage blocks with handlers,
- cleanup blocks.

CLI: `thirst-of-gods run | check | transpile`.

4. T.A.R.L. - Policy Engine (`tarl`)

A standalone, first-match-wins policy language.

- **Verdicts:** `ALLOW` / `DENY` / `ESCALATE`, default-deny.
- **Conditions:** comparisons, set membership (`in`), arithmetic, attribute access, and temporal builtins (`CURRENT_HOUR`, `CURRENT_WEEKDAY`, ...).
- **Authoritative context:** dotted identifiers traverse nested JSON only. Flat dotted keys, duplicate keys, mixed representations, missing paths, and wrong intermediate types fail closed as distinct resolution states.
- **Schema-bound positive authority:** an evaluator `ALLOW` is diagnostic until a complete schema validated the exact evaluated representation and that binding appears in the proof.
- **Proof binding:** original and evaluated context hashes, representation, normalization identity/version, conflict status, schema result, policy hash, rule, verdict, trace, evaluation time, and expiry.
- **Temporal windows & durations**, with an optional hash-linked audit archive.
- **HMAC and Ed25519 proof signing;** secure-default verification rejects unsigned, stale, replayed, revoked, or context-mismatched proofs.
- **Registered sources and quorum:** the exact source-enriched context and proof-bound distinct Ed25519 approvals are required for their positive paths.
- **Policy composition.**
- **Z3 static analysis** (`analyze`, [`analysis`] extra): coverage gaps, shadowed/dead rules, conflicts, and **equivalence** / **refinement** proofs between two policies.

CLI: `tarl eval | parse | verify | audit | explain | test | analyze`, plus a `tarl-lsp` language server.

5. TSCG / TSCG-B - symbolic & binary tiers

- **TSCG** (`tscg`): Thirst's Symbolic Constitutional Grammar - `parse`, `canonical` (normalized form), `checksum` (SHA-256), `validate`, `list`. Canonicalization + hashing for tamper-evident symbolic expressions.
- **TSCG-B** (`tscg-b`): binary framing - `encode` / `decode` / `stream`, with CRC32 + SHA-256 integrity over each frame.

6. Tooling & Workflow

- **thirsty CLI:** `run`, `repl`, `fmt`, `new` (scaffold), `build`, `govern`, `add` / `audit` / `lock` (dependency integrity), `doctor`, `lsp`, `docs`.
- **Console scripts:** `thirsty`, `thirst-of-gods`, `tarl`, `tarl-lsp`, `tscg`, `tscg-b`, `shadow-thirst`.
- **UTF-8-safe** CLI output on Windows.
- **CI gate:** Ruff, mypy, full suite on Python 3.11/3.12 with a 90 percent coverage floor, production acceptance, every shipped example, and a separate wheel-install/all-seven-entry-points smoke job.

7. Representative use cases

- 1 **Governed agent/tool runner** - an agent emits `.thirsty`; in `governed` mode with a T.A.R.L. policy it may only import/IO what policy allows, every action yields a proof, and contracts bound its behavior.
- 2 **Provably-safe code migration** - propose a refactor as a Shadow Thirst `mutation`; promote only if Z3 proves (or sampling shows) equivalence to the validated original, else receive the diverging input.
- 3 **Time-boxed / conditional permissions** - temporal T.A.R.L. policies enforce windowed access, provable via the analyzer and auditable via the archive.
- 4 **Tamper-evident symbolic artifacts** - TSCG canonical form + checksum; TSCG-B integrity-framed binary streams.
- 5 **Contract-checked libraries** - `requires` / `ensures` / `invariant` on methods give runtime design-by-contract that `spillage` cannot bypass.

See the [whitepaper](#) for the formal model, architecture, and the soundness arguments behind the verifier layers.

PART 05 / SECURITY-CONTRACT

Governance and T.A.R.L. Runtime Model

Maintained source: docs/governance_model.md



Edition: Thirsty-Lang 0.8.6. See the [canonical Thirsty-Lang UTF 101 manual](#) for the complete end-to-end reference and deployment recipes.

Overview

Thirsty-Lang implements a **deny-by-default governance model** via T.A.R.L. (Thirsty's Active Resistance Language). Governed calls and operations in the explicit sensitive-capability map require authorization before execution.

Core Principles

- 1 **Deny by Default:** Without explicit authorization, all operations are denied.
- 2 **Policy-Driven:** Authorization rules are expressed as declarative policies, not imperative code.
- 3 **Staged Evaluation:** Policies support three verdicts:
 - **ALLOW** - Operation proceeds
 - **DENY** - Operation is blocked
 - **ESCALATE** - Operation requires manual review or higher-tier approval
- 1 **Composable Tiers:** Governance tiers stack (Tier 1 through Tier 6), with higher tiers enforcing stricter policies.

T.A.R.L. Policy Language

Syntax

TEXT

```
policy <policy_name>

when <condition> => ALLOW|DENY|ESCALATE
when <condition> => ALLOW|DENY|ESCALATE
...
```

Example Policy

TEXT

```
policy resource_access

when role == "admin" => ALLOW
when role == "user" and level >= 3 => ALLOW
when action == "delete" and resource == "critical" => ESCALATE
when action == "read" and resource == "public" => ALLOW
when action == "write" and resource == "system" => DENY
```

```
when source == "external" and port > 1024 => DENY
```

Conditions

Conditions are evaluated as sandboxed expressions supporting:

- **Comparisons:** `==`, `!=`, `<`, `>`, `<=`, `>=`
- **Logical operators:** `and`, `or`, `not`
- **Literals:** strings (`"value"`), integers (`42`), floats (`3.14`), booleans (`true/false`)
- **Context variables:** top-level identifiers and dotted paths into nested objects (`user.role` resolves `{"user":{"role":"admin"}}`)
- **Arithmetic:** Addition (`+`) within comparisons

Safe Evaluation

T.A.R.L. expressions are evaluated in a **sandboxed environment**:

- Missing identifiers and paths are a distinct `MISSING` state and terminate evaluation with a fail-closed verdict; they never become boolean `false`
- Wrong intermediate types are `TYPE_ERROR`; dotted flat/nested ambiguity is `REPRESENTATION_CONFLICT`; neither state participates in ordinary operators
- Only literal values and safe operators allowed
- Only the documented safe functions are callable; imports and mutation are unavailable
- Parallel rule evaluation with adaptive ordering (most-matched rules evaluated first)

Authoritative context representation

T.A.R.L. uses one representation: a JSON object with nested objects for dotted policy paths. For example, this policy:

TARL

```
when user.role == "admin" => ALLOW
```

is evaluated with:

JSON

```
{"user":{"role":"admin"}}
```

`{"user.role":"admin"}` is rejected; a flat dotted key is not an alias for a nested path. Supplying both forms is also rejected, whether their values agree or conflict. JSON duplicate keys and collisions between caller, authority, source, and action layers are rejected. No representation conversion occurs silently.

Missing is not false. Invalid is not false. Unresolved is not evidence. A real resolved value such as `{"feature":{"enabled":false}}` remains an ordinary boolean and can be compared to `false`.

Integration with Thirsty-Lang

Runtime Enforcement (the interpreter, today)

Governance is enforced at runtime for **governed functions** - functions that declare a precondition with `requires`:

THIRSTY

```
module bank: governed
glass withdraw(amt) requires amt > 0 {
  return amt * 2
```

```
}
```

On **every call** to a governed function, the interpreter applies a layered, default-deny decision (`Interpreter._enforce_governance`):

- 1 **In-language contract predicates** - `requires`, `ensures`, and `invariant` expressions are evaluated in the call scope. A falsy result denies the call and raises `GovernanceViolation`.
- 2 **Cross-mode guard** - a governed function invoked while the program is not in `governed` mode is denied (the runtime counterpart of checker error `E053`, "cannot call governed function from core mode").
- 3 **T.A.R.L. routing** - when a `TarLRuntime` is attached (`interpreter.attach_tarl(runtime)`) and an authority context is set, the call is routed through the policy engine. A non-`ALLOW` verdict denies. An `ALLOW` advances only when an explicit or completely derived schema passed against the evaluated representation and that result is proof-bound; an unbound or incomplete-schema `ALLOW` is replaced with a fail-closed denial. The same positive-authority gate applies to `CapabilityBroker`. A `TarLProof` certificate is recorded on `interpreter._last_proof`.

The proof binds the policy hash; original and canonical/evaluated context hashes; context representation ID; normalization algorithm ID and version; conflict status; schema fingerprint, representation, and validation result; matched rule and condition; verdict; evaluation trace/time; and applicable expiry. An identity evaluation has equal original and canonical hashes. A rejected context can produce a `DENY` proof but never an admissible positive proof. It is **unsigned by default**. Two signing modes exist: legacy **HMAC-SHA256**, a *symmetric* MAC that is forgeable by anyone holding the shared key, and **Ed25519**, an asymmetric signature whose verifier needs only the public key. Use `Ed25519` when a proof must attest to the signer rather than merely detect tampering by parties without the shared HMAC key. The `thirsty run` path emits unsigned proofs unless the embedding runtime configures a signing key.

Rule evaluation validates the whole expression surface before accepting its boolean result. Both operands of `and/or` and every predicate application in `ALL/ANY` are evaluated, so a missing or malformed value cannot be hidden behind a decisive earlier result. Comparisons are type-strict: integers and floats interoperate, but strings never coerce to numbers. Numeric operands and arithmetic results must remain finite. Empty quantifier collections fail closed rather than authorizing through vacuous truth. Quantifier binders using the reserved `__tarl__` runtime namespace are rejected.

If `evaluate(..., policy_text=...)` or `evaluate_with_proof(...)` supplies a policy different from `runtime.policy` while the runtime schema was derived, the runtime derives and binds a schema from that exact override. It denies when override derivation is incomplete or invalid; it does not reuse the base policy's derived schema. An explicitly attached schema remains the caller's authoritative binding.

- 1 **Default** - in `governed` mode, a governed function with no attached policy engine or no authority is **denied** with a proof. A call that no layer explicitly allowed is denied (deny-by-default).

A `GovernanceViolation` is a hard floor: `spillage` error handlers do **not** catch it, so governed denials cannot be swallowed by user error handling. This includes denials raised while importing a `.thirsty` module: imported modules run under the **caller's** governed runtime, so their effects are gated rather than executed in a detached, ungoverned interpreter. A `governed` module that fails to parse also fails closed - its recovered statements are discarded and execution is refused.

For verification, `ProofVerifier` rejects unsigned proofs by default. Local inspection of unsigned proofs must opt in explicitly with `ProofVerifier(require_signature=False)` or `tarl verify --allow-unsigned`. Hardened deployments can also restrict signature algorithms (for example, `Ed25519`-only) and require policy-source binding. The CLI exposes these as `tarl verify --ed25519-only --policy <policy.tarl>`. Signed proofs use one canonical lowercase hexadecimal signature encoding. Replay identity binds the complete proof semantics, signing key, algorithm, and decoded signature bytes, so a cryptographically identical re-encoding cannot obtain a second use. Authority consumers must also supply the expected original context. A registered-source `ALLOW` requires the exact source-enriched evaluated context as well; the verifier accepts only valid top-level `source:<name>` additions that leave the original context unchanged.

An escalation quorum is a separate positive-authority consumer. A `QuorumResolver` must be configured with an independent `ProofVerifier` and the exact policy source. It promotes only an `ESCALATE` decision whose proof is

independently verified, cryptographically signed, policy-bound, internally consistent, unexpired, and bound to a passed authoritative context schema. Each approval signs the SHA-256 digest of the complete proof artifact; duplicate approver identities or duplicate public-key material count once. A time-bound promoted decision retains its verified signed expiry. Every resolution requires the exact original request context, an explicit timezone-aware trusted clock, and a verifier configured with a maximum proof age and replay guard; the resolver fails closed when any is absent. Registered-source escalation proofs also require the exact evaluated context. Deployments should additionally use a shared durable replay store and configure policy revocation; approval signatures over a proof digest do not make an unbounded proof fresh, current, or single-use by themselves.

Building a **governed** module to a target that drops the governed runtime (`js`, `llvm-*`, `wasm-pyodide`) is refused by default; `thirsty build --allow-governance-loss` is required to proceed and records the loss in the emitted manifest. When `--emit-manifest` is used, the manifest records source hash, optional policy hash, required capabilities, derived or attached context schema, authority mode, proof verification mode, audit requirement, Shadow Thirst status when statically visible, and governance-loss status.

Static proof-obligation reporting

`thirsty prove` is a static, no-side-effect path:

BASH

```
thirsty prove program.thirsty --policy policy.tarl --emit-manifest
```

It lexes, parses, checks, and walks the AST, then emits a machine-readable JSON report. It does not instantiate the interpreter and does not execute program side effects. The report includes functions, imports, sensitive stdlib calls, governed calls, required TARL actions, required capabilities, context schema, authority requirements, contract obligations, proof mode, audit requirement, Shadow Thirst status, governance-loss status, diagnostics, and unresolved proof gaps. Required TARL actions include capability actions and governed function-call actions.

Context schema handling is fail-closed. If `--context-schema schema.json` is provided, that explicit schema is authoritative. Without an explicit schema, `thirsty prove` derives field names and simple kinds from TARL policy references where possible. Ambiguous references are reported as `context_schema.status = incomplete`, and `thirsty prove` exits non-zero instead of claiming the proof obligation set is complete. Explicit schema files may use a list of field objects or a compact mapping:

JSON

```
{"fields": {"user.role": "string", "risk": {"kind": "number", "required": false}}}
```

Schema field names use the same dotted-path notation as policies, and generated schemas declare `tarl.context.nested-json.v1` with no normalization. A schema that declares another representation or requests normalization is rejected. The loader also rejects duplicate/conflicting representation metadata, duplicate field names, unknown kinds, non-boolean `required`, and `on_violation: ALLOW`; schema metadata is never coerced silently.

Malformed explicit schema entries fail with validation errors instead of being treated as proof-ready.

Runtime-derived schemas follow the evaluated policy identity. A differing `policy_text` override receives a fresh derivation and proof binding; an incomplete override schema produces DENY rather than inheriting the base policy's derived schema.

`thirsty explain-denial program.thirsty --policy policy.tarl` emits a machine-readable explanation of missing policy, context, authority, and proof conditions for the same static obligation set.

From the CLI:

BASH

```
thirsty run program.thirsty --thirst-level governed \
  --authority admin --policy access.tarl
```

--authority injects the authority tag into the governance context; --policy routes governed calls through the named .tarl policy. A denial prints `governance denied: <fn>: <reason>` (with the proof verdict/hash when policy routing produced one) and exits non-zero.

Extended governance surfaces

The governed interpreter routes shipped filesystem, network, process, and other sensitive stdlib effects through `CapabilityBroker`. External adapters must call that same broker before acting; deployments remain responsible for preserving the path when they add new adapters.

Runtime Evaluation

TarlRuntime

The `TarlRuntime` class provides:

- 1 **LRU Caching** (128 entries) - Policies are cached by policy identity plus original/evaluated context binding and transformation metadata
- 2 **Parallel Evaluation** - Rules evaluated concurrently via `ThreadPoolExecutor`
- 3 **Adaptive Ordering** - Frequently-matched rules are prioritized
- 4 **Policy Hotswapping** - Policies can be updated at runtime without restart
- 5 **Optional durable audit** - `set_archive(TarlAuditArchive(...))` persists generated proofs. Call `set_require_audit(True)` when an attached archive must fail closed on write failure. With no archive attached, decisions are returned but not persisted; hardened interpreter mode does not create one.

CLI Usage

Evaluate a policy against a context:

BASH

```
tarl eval policy.tarl --context '{"role":"user","level":2}'
```

For a dotted policy path, pass nested JSON and bind the same representation when verifying its proof:

BASH

```
tarl eval policy.tarl --context '{"user":{"role":"admin"}}'
tarl verify proof.json --context '{"user":{"role":"admin"}}'
```

If the policy uses a registered source, verification preserves and checks the transformation explicitly:

BASH

```
tarl verify source-proof.json \
  --context '{"role":"admin"}' \
  --evaluated-context '{"role":"admin","source:trusted_roles":["admin"]}'
```

Temporal policy windows, time-bound verdicts, `CURRENT_*` builtins, and `ELAPSED_SINCE` require an explicitly zoned trusted evaluation time on the CLI. A configured runtime clock that cannot produce such a time fails closed without falling back to the host clock. Temporal metadata and durations are parsed strictly, expiry cannot grant `ALLOW`, and proof authority is capped at the earliest exclusive policy or rule cutoff:

BASH

```
tarl eval policy.tarl --context '{"role":"user"}' --now 2026-07-01T12:00:00Z
```

Parse and display a policy:

BASH

```
tarl parse policy.tarl
```

Security Considerations

- 1 **Context Injection** - Contexts are supplied at runtime; validate all sources
- 2 **Policy Updates** - Policies should be cryptographically signed before deployment
- 3 **Audit Logging** - Attach an archive and require persistence for deployments that depend on durable decision records; checkpoint the chain externally
- 4 **Performance** - Cache identity binds policy, context, schema, and source values; temporally constrained policies bypass the decision cache

Future Enhancements

- **Cryptographic Policy Signatures** - Sign policies to prevent tampering
- **Signed policy distribution** - Bind deployed policy sources to an external release/transparency process
- **Adapter conformance automation** - Inventory deployment-specific adapters and prove that each reaches the broker
- **External trust services** - HSM-backed key custody and independently hosted durable replay/audit infrastructure
- **Metrics & Analytics** - Track policy evaluation metrics for optimization

PART 06 / OPERATOR-REQUIREMENT

Production Deployment Checklist

Maintained source: [docs/PRODUCTION_DEPLOYMENT.md](#)



Applies to: Thirsty-Lang 0.8.6+ · **Audience:** operators embedding the governed runtime to authorize real-world, high-risk actions.

Canonical end-to-end reference: [THIRSTY_LANG_UTF_101.md](#).

The reference runtime meets its hardened-runtime acceptance bar (WHITEPAPER §8) and, as of 0.7.0, ships the operational machinery that earlier releases left to the embedder: unified broker mediation, durable cross-process replay/audit state, and deployment key management. This document is the **checklist** a deployment must satisfy before it is production-ready. The CI `production-acceptance` job and `tests/test_production_acceptance.py` enforce the core of it on every change.

Anything below that is unchecked means the deployment is **not** production-ready for governed execution.

1. Run hardened, fail-closed

- [] Run every governed entrypoint with `thirsty run --hardened`. Hardened mode requires an authenticated (signed) authority **and** Ed25519-signed proofs at every gate, or it fails closed.
- [] Always attach a policy (`--policy <file>.tarl`). A governed function or sensitive stdlib call with no policy engine + authority is **denied with a proof** - governed mode never implies authority.
- [] Treat a `GovernanceViolation` (CLI exit code 2) as a hard stop, never a warning. A returned `TarLProof` records the decision, but it is not durable audit evidence until an attached archive successfully persists it.

2. One enforcement path (broker + path guard)

- [] In-language stdlib effects and any out-of-language adapters (FFI, MCP/tool, subprocess) both flow through one `CapabilityBroker` (`Interpreter.make_broker()` / `utf.tarl.broker`). Do not add a side adapter that performs an effect without `broker.require(...)`.
- [] Set a filesystem confinement root for governed file access: `Interpreter.set_path_guard([allowed_root, ...])`. File targets are then brokered on the **canonical** path, so traversal/symlink escapes fail closed before the effect (C042).

3. Context authority and registered-source transformations

- [] Supply caller context as nested JSON under `tarl.context.nested-json.v1`. Dotted schema/policy paths traverse nested objects. Flat dotted keys, any mixed flat/nested form (equal or conflicting), duplicate keys, missing paths, and wrong intermediate types fail closed; no conversion is silent.
- [] Attach an explicit `ContextSchema`, or require `TarLRuntime.ensure_context_schema()` to derive a complete schema, before a positive verdict can authorize an effect. A raw evaluator `ALLOW` without a passed, proof-bound schema is diagnostic output, not advancement authority.

- [] Route load-bearing use through `CapabilityBroker` or the governed interpreter. Both apply the positive-context-authority gate and replace an inadmissible `ALLOW` with a fail-closed denial.
- [] Bind proof verification to the exact original request context. If a policy uses registered sources, also provide the exact evaluated context. Verification permits only valid top-level `source:<name>` additions whose removal reproduces the original context; callers may not supply reserved source fields.
- [] Keep caller contexts, registered-source values, and explicit schema files inside the strict finite JSON domain. Duplicate keys, non-finite numbers, Python-only values, coercive schema metadata, and representation conflicts fail closed.
- [] Do not rely on expression short-circuiting to skip invalid data. TARL validates both boolean operands and every quantifier element, rejects reserved `__tarl_*` binders, treats empty quantifier collections as no authorization evidence, forbids string-to-number comparison coercion, and requires finite numeric operands and arithmetic results (C065-C067).
- [] If an embedding passes a differing `policy_text` override to a runtime whose schema was derived, require a fresh complete derivation for that exact override. An incomplete override must deny; do not reuse the base policy's derived schema (C068).

4. Durable, cross-process replay & revocation state

In-memory replay/revocation state is lost on restart and not shared across workers - a replayed proof would be accepted by a second process. Use the durable stores in `utf.tarl.durable`:

- [] Wire a `DurableReplayGuard("<path>.db")` into every `ProofVerifier` (`verify --replay-db <db>` from the CLI). The store must be shared by all verifying processes (a shared volume or a single DB host).
- [] Preserve the store during upgrades. Version 0.8.6 records a complete semantic replay identity and a canonicalized legacy identity, so proofs used before upgrade remain single-use. Proof signatures must retain their one lowercase hexadecimal encoding.
- [] Maintain a `RevocationStore("<path>.db")` and hydrate verifiers from it (`verify --revocation-store <db>`). Revoke a compromised/rotated policy with `tarl revoke <policy-hash> --store <db>`; list with `tarl revoke --list`.

5. External audit checkpoints

The audit archive (`TarlAuditArchive`) is SQLite-durable and hash-chained, but a local attacker who can rewrite the DB could re-link a truncated suffix.

- [] Explicitly attach a `TarlAuditArchive` to the same `TarlRuntime` used by the interpreter or broker and call `set_require_audit(True)`. Hardened mode alone does not create or require an archive. If no archive is attached, no audit row is written; if an attached required archive fails, the decision is downgraded to DENY.
- [] On a schedule, write the chain head to a **trusted external location**: `tarl audit checkpoint --db <audit.db> --out <head.txt>` and store `head.txt` somewhere the runtime host cannot silently rewrite.
- [] Verify against it: `tarl audit verify-chain --db <audit.db> --checkpoint <head.txt>`. A head that no longer matches the checkpoint reveals suffix rewrite or truncation.

6. Trust-root key management

Three Ed25519 trust roots must be provisioned: **authority issuer, proof signer, time authority** (`utf.tarl.keystore`).

- [] Generate each key with `tarl keygen <role> --key-id <id> --out <file>`. Private key files are written `0600`; keep them off `argv` and out of logs.

- [] Load keys from files, never as hex on the command line. Use `thirsty run --sign-proofs-file <signer.key> --authority-key-file <issuer.pub>` and `tarl verify --ed25519-key-file <pub>`. The legacy `ID:HEX` flags are deprecated because `argv` is observable.
- [] **Rotation:** mint a new key with a fresh `key_id`, register its public half alongside the previous one (verifier registries are keyed by `key_id`, and `--ed25519-key-file` is repeatable), switch signing to the new key, then retire the old public key only after all in-flight artifacts signed by it have expired.
- [] Provision a `TimeAuthority` key and run temporal policies against a `TrustedClock`, not the host clock (C043).
- [] Configure each `QuorumResolver` with an independent `ProofVerifier` and the exact policy source, plus an explicit timezone-aware trusted clock. Promotion requires an `ESCALATE` proof that passes cryptographic signature, policy, context, authoritative-schema, freshness, replay, and expiry verification; counted approvers must be distinct by identity and key material, each Ed25519 approval must bind the digest of the complete proof artifact, and a time-bound promotion must retain its verified signed expiry (C069).
- [] Bind quorum resolution to the current request by supplying its exact original context; for a registered-source proof, also supply the evaluated context. Do not promote a self-consistent proof without establishing that it belongs to the request being authorized.
- [] Configure the quorum's `ProofVerifier` with a maximum proof age and a replay guard; the resolver refuses promotion without both. Back the replay guard with deployment-wide durable state and configure policy revocation as an additional deployment control. Digest-bound approvals do not by themselves make an unbounded proof fresh, current, or single-use.
- [] Treat `valid_until` as an exclusive cutoff. Runtime decisions and proofs bind the earlier of the effective policy cutoff and rule duration; independent verification and quorum promotion must receive trusted time and enforce that exact bound.

7. Secret custody (remains with the deployment)

The runtime defines the **formats and flows**; it does not choose where secrets live. The deployment owns:

- [] Storage of private key files and the durable `.db` stores in a vault, HSM, or encrypted volume with least-privilege access.
- [] Backup/restore and access auditing for those artifacts.
- [] Rotation cadence and revocation response runbooks.

8. Release gates (enforced in CI)

- [] `ruff check src tests` clean.
- [] `mypy -p utf` clean.
- [] `pytest --cov=utf --cov-fail-under=90` green.
- [] Every shipped example runs; all console scripts install and `--help` on 3.11 and 3.12.
- [] `production-acceptance` job green: a governed program runs under `--hardened` with file-based keys, an effect is **allowed with a grant** and **denied (exit 2) without one**.

Verifying a deployment end to end

BASH

```
# 1. Provision trust roots
tarl keygen authority-issuer --key-id issuer-1 --out issuer.key
tarl keygen proof-signer --key-id signer-1 --out signer.key

# 2. Run a governed program hardened, with file-based keys + a policy.
```

```
# This CLI path emits proofs but does not itself attach audit.db.
thirsty run app.thirsty --thirst-level governed --hardened \
  --policy policy.tarl \
  --authority-token token.json --authority-key-file issuer.key.pub \
  --sign-proofs-file signer.key
# -> allowed effects run; a denied effect exits 2 with a proof

# 3. In an embedding that requires durable audit, configure the exact runtime:
#   with TarlAuditArchive("audit.db") as archive:
#       runtime.set_archive(archive).set_require_audit(True)
#       interpreter.attach_tarl(runtime)
#       ... evaluate governed work while the archive is open ...

# 4. Checkpoint and verify the populated audit chain
tarl audit checkpoint --db audit.db --out head.txt
tarl audit verify-chain --db audit.db --checkpoint head.txt

# 5. Verify a proof with durable replay + revocation, rejecting reuse
tarl verify proof.json --ed25519-key-file signer.key.pub --ed25519-only \
  --context-file context.json \
  --replay-db replay.db --revocation-store revocations.db
```

For a registered-source proof, add `--evaluated-context-file evaluated-context.json`; both files are mandatory for accepting that transformed positive decision.

See also: [WHITEPAPER.md](#) §8-§9, [THREAT MODEL.md](#) §"Remaining Gaps", [governance_model.md](#), [STATUS.md](#).

PART 07 / SECURITY-ACCEPTANCE

Offensive Threat Model C001-C073

Maintained source: docs/THREAT_MODEL.md



Edition: Thirsty-Lang 0.8.6. This catalog is included in the [canonical Thirsty-Lang UTF 101 manual](#).

Purpose

This document defines the adversary model for Thirsty-Lang as a governance AI substrate: a language/runtime layer that mediates human, AI, script, service, and tool actions before side effects happen.

The defensive claim is not that Thirsty-Lang is a Python replacement. The claim is narrower and harder: under hostile conditions, a governed runtime should make unauthorized capability use structurally difficult, auditable, and fail-closed.

Security Objective

Thirsty-Lang succeeds when all sensitive action attempts pass through a single governed decision path:

- 1 Build a canonical action context.
- 2 Evaluate policy and contracts.
- 3 Produce a proof-bearing ALLOW, DENY, or ESCALATE verdict.
- 4 Execute only on ALLOW.
- 5 Refuse by default when authority, policy, context, proof, or runtime state is missing, stale, malformed, or compromised.

No claim is earned unless it is backed by code, tests, docs, or explicit roadmap status.

Existing Resistance Surface

Thirsty-Lang already has several defensive primitives that directly reduce the attack surface in this model. These are not complete hardening by themselves, but they are real substrate features rather than future aspirations.

Feature	Defensive value	Evidence
Governed mode	Separates ordinary execution from authority-checked execution	tests/test_governance_maximal.py; tests/test_gate_fail_closed.py
Default-deny capability gates	Blocks governed I/O/import and sensitive imported stdlib calls when policy or authority is missing	tests/test_gate_fail_closed.py
TARL policy engine	Moves authorization into explicit policy instead of code convention	tests/test_tarl.py; tests/test_tarl_composition.py
ALLOW / DENY / ESCALATE verdicts	Supports refusal and human/escalation workflows instead of binary allow-only logic	tests/test_tarl.py
requires / ensures / invariant contracts	Makes preconditions, postconditions, and invariants executable runtime checks	tests/test_governance_maximal.py

Feature	Defensive value	Evidence
GovernanceViolation hard floor	Prevents application error handlers from swallowing governance denials	tests/test_gate_fail_closed.py
Proof-carrying evaluation	Binds decision output to policy hash, context hash, verdict, and trace	tests/test_tarl_proof.py
Ed25519 proof signatures	Provides asymmetric verification for non-repudiable proof records	tests/test_tarl_proof.py; tests/test_cli_tarl.py
Temporal policy windows	Reduces stale authorization and emergency-policy blast radius	tests/test_tarl_temporal.py
First-match-wins policy order under adaptive execution	Prevents optimization from changing authorization semantics	tests/test_tarl.py
Safe expression evaluator	Avoids eval/import/state mutation inside policy conditions	tests/test_tarl.py
Shadow Thirst analyzers	Detects unsafe mutation promotion, determinism failures, and plane leakage	tests/test_shadow_thirst.py; tests/test_verifiers.py
TSCG/TSCG-B integrity surfaces	Provides symbolic constraints and binary-frame integrity checks	tests/test_tscg.py; tests/test_tscg_b.py
UTF-8-safe CLI output	Keeps denial/proof reporting reliable on Windows terminals	src/utf/console.py; CLI tests

The hardening work is therefore not "invent security from nothing." It is to connect these primitives into a universal capability broker, extend the gates to every side-effect adapter, and prove the result with offensive challenge tests.

Protected Assets

Asset	Why it matters	Required protection
Capability gates	They decide whether side effects happen	No bypass path around the gate
TARL policies	They encode authority	Hash-bound, versioned, signed before deployment
TarLProof records	They are the audit trail	Ed25519 for non-repudiation, replay resistance, trace integrity
Authority context	It identifies who or what is acting	Authenticated source, least privilege, no self-asserted authority
Runtime context	It drives policy decisions	Canonicalized, validated, no attacker-controlled silent defaults
Tool adapters	They bridge to real effects	Every adapter must request capability before effect
Audit archive	It preserves what happened	Append-only, tamper-evident, externally verifiable
Human escalation path	It handles ambiguous or high-risk actions	Explicit ESCALATE path, quorum or operator confirmation

Trust Boundaries

Boundary	Trusted by default?	Required rule
Thirsty source text	No	Parse, check, and govern before execution
TARL policy source	No	Verify hash/signature before use
AI-generated code	No	Govern all imports, I/O, tool calls, subprocesses, network, FFI
Human operator command	No	Authority must be bound into context by a trusted channel
CLI arguments	No	Treat as untrusted input to runtime setup
Environment variables	No	Never grant authority from env alone
Local filesystem	No	Treat policy/proof/config files as mutable attacker inputs
Network services	No	External responses are untrusted context
Plugin/MCP/tool adapters	No	Must be capability-brokered
Test results	Partially	Useful only when reproducible and mapped to a threat

Adversary Classes

ID	Adversary	Capabilities
A1	Prompt-injected AI agent	Can generate tool calls, code, policies, and explanations that look benign
A2	Malicious plugin or tool adapter	Can expose side effects outside the language runtime
A3	Compromised local script	Can call CLI/runtime APIs directly and mutate local files
A4	Insider operator	Has legitimate access but attempts unauthorized actions
A5	Policy downgrader	Can edit, swap, delete, or roll back policy files
A6	Proof forger	Can tamper with or mint audit artifacts
A7	Replay attacker	Can reuse old ALLOW proofs or cached decisions
A8	Context poisoner	Can supply misleading context fields, missing fields, or type-confused values
A9	Parser/semantic attacker	Exploits grammar ambiguity, precedence, parser recovery, or unknown identifiers
A10	Import/FFI attacker	Reaches Python, stdlib, native code, shell, or network outside gates
A11	Archive tamperer	Modifies stored proof history or deletes denials
A12	Availability attacker	Forces fail-open through errors, timeouts, resource pressure, or cache abuse
A13	Build/package attacker	Ships a different package than the audited source
A14	Human-social attacker	Uses urgency, authority language, or false mission framing to trigger unsafe ALLOW

Offensive Challenge Catalog

Each challenge is an adversarial success attempt. The expected defensive outcome is what Thirsty-Lang must do to claim resistance.

ID	Challenge	Expected defensive outcome	Current status
C001	Run <code>pour</code> in governed mode with no policy	DENY with proof	Covered by <code>tests/test_gate_fail_closed.py</code>
C002	Run callable stdout builtin <code>print(...)</code> in governed mode with no policy	DENY with proof	Covered by <code>tests/test_gate_fail_closed.py</code>
C003	Read stdin with <code>sip</code> in governed mode with no policy	DENY with proof	Covered by <code>tests/test_gate_fail_closed.py</code>
C004	Import a module in governed mode with no policy	DENY with proof	Covered by <code>tests/test_gate_fail_closed.py</code>
C005	Allow write policy, then attempt read/import	DENY non-matching capability	Covered by <code>tests/test_gate_fail_closed.py::test_write_allow_does_not_grant_read_or_import</code>
C006	Wrap a governed denial in <code>spillage</code>	Denial propagates; handler cannot swallow it	Covered by governance behavior
C007	Call governed function from core mode	DENY via cross-mode guard/static E053	Covered by status matrix tests
C008	Use contract predicate ambiguity to invert policy meaning	Parser precedence must preserve author intent	Covered by precedence tests
C009	Tamper TarlProof verdict after signing	Verification fails	Covered by proof tests
C010	Verify Ed25519 proof with wrong public key	Verification fails	Covered by proof tests
C011	Verify Ed25519 proof with no public key	Verification fails	Covered by proof tests
C012	Verify HMAC proof with wrong shared key	Verification fails	Covered by proof tests
C013	Use HMAC as non-repudiation	Documentation must reject claim	Covered by docs; not a runtime block
C014	Replace policy source after proof generation	Policy hash verification fails	Covered by proof verifier tests
C015	Submit malformed proof JSON	CLI verification exits non-zero	Covered by CLI proof tests
C016	Use expired temporal policy	DENY or configured non-ALLOW expiry verdict	Covered by temporal tests
C017	Cache a time-bound ALLOW past expiry	Runtime must not cache temporal decisions	Covered by runtime behavior
C018	Use unknown identifiers to silently allow	Unknown identifiers fail safe	Covered by TARL semantics
C019	Inject malformed context JSON through CLI	CLI exits non-zero	Covered by CLI tests
C020	Mutate policy ordering to exploit adaptive evaluation	First-match-wins must remain policy order	Covered by TARL tests
C021	Trigger evaluator exception inside rule condition	Rule fails safe and throw is observable	Covered by runtime throw stats tests
C022	Store tampered proof in archive and query without verifier	Must be documented as unverified unless verifier supplied	Covered by <code>tests/test_threat_model_audit_chain.py</code> (hash-linked chain) and <code>archive.query(verifier=...)</code>
C023	Replay old ALLOW proof for new context	Must reject by context hash and freshness policy	Covered by <code>tests/test_threat_model_replay.py</code> - <code>ProofVerifier(expected_context=...)</code> rejects context-mismatched replay
C024	Replay current ALLOW proof for same context after policy revocation	Must reject by policy version/freshness	Covered by <code>tests/test_threat_model_replay.py</code> - freshness (<code>max_age_seconds</code>), <code>revoked_policy_hashes</code> , and <code>ReplayGuard</code>
C025	Downgrade Ed25519 proof to unsigned proof	Verifiers must reject unsigned proofs unless unsigned inspection is explicit	Covered by <code>tests/test_threat_model_proof_strictness.py</code> and <code>tests/test_peer_review_081_tarl_regressions.py</code> - <code>ProofVerifier()</code> and <code>tarl.verify</code> require signatures by default; <code>require_signature=False</code> / <code>--allow-unsigned</code> is explicit
C026	Delete DENY proof from audit archive	Hash-linked append-only audit must reveal gap	Covered by <code>tests/test_threat_model_audit_chain.py</code> - deleting a record breaks the hash chain (<code>verify_chain</code>)
C027	Forge authority by setting CLI <code>--authority admin</code>	Authority must come from authenticated identity, not string input alone	Covered by <code>tests/test_threat_model_authority.py</code> - signed <code>AuthorityClaim</code> required; bare <code>--authority</code> denied in hardened mode
C028	Put authority in environment variable	Must not grant authority from env alone	Covered by <code>tests/test_threat_model_authority.py</code> - authority is <code>authority_authenticated == False</code> unless a signed claim verifies
C029	Use <code>stdlib thirst::fs</code> to write file without gate	Must route filesystem writes through capability broker	Covered by <code>tests/test_threat_model_capability_broker.py</code> and <code>tests/test_gate_fail_closed.py</code> ; every sensitive callable has an explicit action in <code>module_system.SENSITIVE_STDLIB_CAPABILITIES</code> . Deferred (medium): path-root/canonicalization policy (see C042)

ID	Challenge	Expected defensive outcome	Current status
C030	Use <code>stdlib thirst::http</code> or <code>thirst::net</code> without gate	Must route network through capability broker	Covered by <code>tests/test_threat_model_capability_broker.py</code> and <code>tests/test_gate_fail_closed.py</code> . Deferred (medium) : external adapter / real-egress coverage
C031	Use <code>stdlib thirst::process.run</code> without gate	Must route subprocess through capability broker	Covered by <code>tests/test_threat_model_capability_broker.py</code> and <code>tests/test_gate_fail_closed.py</code> . Deferred (medium) : CLI build subprocesses (<code>llc/clang/lli</code>) are a separate, non-governed surface
C032	Use <code>thirst::env.set</code> to poison later decisions	Must route env mutation through capability broker	Covered by <code>tests/test_threat_model_capability_broker.py</code> and <code>tests/test_gate_fail_closed.py</code>
C033	Use FFI/native extension to perform side effects	Must deny or broker FFI	Covered by <code>tests/test_threat_model_broker.py</code> - <code>CapabilityBroker</code> denies FFI/execute by default; no native reach in-language
C034	Use generated JS build output to skip governed runtime	Build artifacts must preserve or declare governance loss	Covered by <code>tests/test_threat_model_build_outputs.py</code> - <code>thirsty build</code> refuses governance-dropping targets for governed source by default; <code>--allow-governance-loss</code> is required, warns on stderr, and records <code>build.governance_loss</code> in the manifest
C035	Use package manager or import path confusion to load malicious module	Imports and dependency integrity must be governed	Covered by <code>tests/test_threat_model_file_imports.py</code> - imported <code>.thirsty</code> modules execute under the caller's governed runtime (policy + authority), so top-level effects and returned closures are gated, not run in a detached core interpreter. Deferred (medium) : dependency-pin/signature integrity for remote packages
C036	Use parser recovery to smuggle executable statements after an error	Parser errors must fail closed for execution	Covered by <code>tests/test_threat_model_parser_fail_closed.py</code> - a governed module with any parse error yields zero statements and the interpreter refuses to run it (DENY proof)
C037	Use resource exhaustion to force fail-open	Runtime errors must DENY, not ALLOW	Covered by <code>tests/test_threat_model_failclosed.py</code> - evaluator errors fail closed (DENY), surfaced as a non-swallowable denial
C038	Use denial-of-service to suppress audit writing	Execution should fail closed when required audit cannot persist	Covered by <code>tests/test_threat_model_failclosed.py</code> - <code>set_require_audit</code> downgrades to DENY when a required proof cannot persist
C039	Use AI-generated policy with broad <code>when true => ALLOW</code>	Policy analysis must flag broad allow and require review	Covered by <code>tests/test_threat_model_lint_quorum.py</code> - <code>lint_policy</code> flags broad/ungated ALLOW (<code>tarl lint</code>)
C040	Use prompt injection to instruct an agent to bypass Thirsty	Agent adapters must enforce broker outside model text	Covered by <code>tests/test_threat_model_broker.py</code> - agent/tool effects must call <code>CapabilityBroker.require</code> ; denied by default
C041	Use MCP/tool call directly from agent runtime	Tool adapter must call broker before tool invocation	Covered by <code>tests/test_threat_model_broker.py</code> - MCP/tool adapters broker before invocation (<code>ACTION_TOOL</code>)
C042	Use filesystem symlink/path traversal to escape allowed root	Path canonicalization and root policy required	Covered by <code>tests/test_threat_model_pathguard.py</code> - <code>PathGuard</code> confines canonical paths; traversal/symlink escape denied
C043	Use time spoofing or make a configured time authority fail so runtime falls back to the host clock	Trusted clock values must be signed and timezone-aware; a missing, malformed, naive, or failing configured clock denies without host fallback	Covered by <code>tests/test_threat_model_clock.py</code> - <code>TrustedClock</code> verifies signed zoned time and invalid configured clocks fail closed
C044	Use stale cached decision after context changes	Cache key and invalidation must bind all relevant context	Covered by <code>tests/test_tarl_composition.py::TestRuntimeRegisterSource::test_callable_source_updated_dynamically</code> and <code>test_source_does_not_leak_between_contexts</code>
C045	Use partial context omission to get safer defaults wrong	Missing required context must DENY or ESCALATE	Covered by <code>tests/test_threat_model_context_schema.py</code> - missing required field fails closed before rule evaluation
C046	Use type confusion in policy context	Context schema validation required	Covered by <code>tests/test_threat_model_context_schema.py</code> - type-confused context value fails closed (<code>ContextSchema</code>)
C047	Use policy include/composition cycle	Must raise composition error, not fail open	Covered by composition tests
C048	Use parallel evaluation race to alter first-match semantics	Policy order must win	Covered by TARL tests
C049	Use archive query without signature verification as proof of validity	CLI/docs must distinguish stored from verified	Covered by <code>tests/test_threat_model_audit_chain.py</code> and <code>tarl audit verify-chain</code> ; <code>query(verifier=...)</code> distinguishes stored from verified

ID	Challenge	Expected defensive outcome	Current status
C050	Use social pressure language to force high-risk ALLOW	ESCALATE rules and proof-bound quorum required	Covered by <code>tests/test_threat_model_lint_quorum.py</code> - promotion requires a signature-verified policy/schema-bound proof and distinct digest-bound approvals
C051	Claim governed readiness for a program without enumerating effects/proof obligations	Static report must list governed calls, sensitive stdlib calls, capabilities, TARL actions, context schema, authority, contracts, and unresolved gaps before execution	Covered by <code>tests/test_proof_obligations.py</code> - <code>thirsty</code> prove emits a machine-readable obligation report without running side effects
C052	Infer an unsafe or ambiguous policy context schema	Derived schema must be marked incomplete and make <code>thirsty</code> <code>prove</code> fail closed unless an explicit schema is supplied	Covered by <code>tests/test_proof_obligations.py</code> - ambiguous field references produce <code>context_schema.status = incomplete</code> and exit non-zero
C053	Use numeric strings to bypass ordering thresholds or exploit silent string-to-number conversion	Comparisons are type-strict: integers and floats interoperate, while every string/number comparison fails closed without conversion	Covered by <code>tests/test_peer_review_0_8_1_tarl_regressions.py</code> and <code>tests/test_tarl_context_security_boundaries.py</code>
C054	Hide a malformed DENY guard before a broad ALLOW	Policy load and residual runtime evaluation errors must reject or DENY, never skip to a later ALLOW	Covered by <code>tests/test_peer_review_0_8_1_tarl_regressions.py</code> and <code>tests/test_tarl.py::TestThrowStats</code>
C055	Accept a forged unsigned proof as valid by default	Verification default must require a signature and trace consistency must reject verdict mismatch when trace declares a verdict	Covered by <code>tests/test_peer_review_0_8_1_tarl_regressions.py</code>
C056	Evaluate a policy window, time-bound verdict, <code>CURRENT_*</code> , or <code>ELAPSED_SINCE</code> from spoofed host time through <code>tarl eval</code>	Every time-dependent CLI evaluation must require an explicitly zoned trusted time	Covered by <code>tests/test_peer_review_0_8_1_tarl_regressions.py</code> - <code>tarl eval</code> refuses all time-dependent rules without zoned <code>--now</code>
C057	Generate auto-TARL policies keyed on a context field the runtime never supplies	Generated policies must use <code>action</code> , matching governed runtime context	Covered by <code>tests/test_peer_review_0_8_1_tarl_regressions.py</code>
C058	Supply a flat dotted key, nested object, or both so schema and evaluator observe different values	One authoritative nested representation; flat dotted keys and every mixed representation DENY explicitly	Covered in released 0.8.6 by <code>tests/test_tarl_context_resolution_integrity.py</code> ; independent Competence Register acceptance remains pending
C059	Use <code>!=</code> , <code>not</code> , membership, or a safe function to turn an unresolved path into ALLOW	MISSING and TYPE_ERROR short-circuit the rule and produce fail-closed DENY; a resolved boolean <code>false</code> remains distinct	Covered in released 0.8.6 by <code>tests/test_tarl_context_resolution_integrity.py</code> ; the confirmed 0.8.5 bypass is preserved as regression evidence
C060	Present a positive proof whose schema context differs from the evaluated context	Positive proof is inadmissible unless representation metadata and original/canonical hashes are coherent and signature-bound	Covered in released 0.8.6 by <code>tests/test_tarl_context_resolution_integrity.py</code> ; independent Competence Register acceptance remains pending
C061	Supply Python-only, cyclic, or non-finite context values, or exploit Python equality between booleans and numbers	Context and registered sources are restricted to finite JSON; incompatible values cannot enter boolean, comparison, membership, arithmetic, or safe-function algebra	Covered by <code>tests/test_tarl_context_security_boundaries.py</code>
C062	Feed an unbound evaluator ALLOW directly into a broker or governed effect	Load-bearing consumers require an admissible positive proof with a passed explicit or complete derived schema; otherwise they replace ALLOW with DENY	Covered by <code>tests/test_tarl_context_security_boundaries.py</code> and <code>tests/test_tarl_proof.py</code>
C063	Inject a reserved source field or forge a source-enriched context that also changes caller data	Registered-source injection is the sole transformation; proofs preserve original/evaluated bindings and verification requires both contexts while permitting only valid top-level source additions	Covered by <code>tests/test_tarl_context_security_boundaries.py</code> and <code>tests/test_tarl_proof.py</code>
C064	Launder an explicit schema through duplicate/conflicting representation metadata, coercive fields, unknown kinds, or <code>on_violation: ALLOW</code>	Schema JSON and metadata are parsed strictly; unsupported, ambiguous, duplicate, or fail-open declarations are rejected	Covered by <code>tests/test_tarl_context_security_boundaries.py</code>
C065	Hide a missing, malformed, or type-invalid value behind a decisive boolean operand, an earlier decisive quantifier element, or vacuous truth over an empty collection	Both boolean operands and every predicate application over the supplied collection are evaluated before the combined result is accepted; any resolution/type failure or empty quantifier collection denies the decision	Covered by <code>tests/test_tarl_context_security_boundaries.py</code>
C066	Supply NaN, infinity, or overflow through numeric values/arithmetic, or send a numeric-looking string where the policy expects a number	Numeric operands and arithmetic outputs must be finite, and strings never coerce to numbers; violations raise a fail-closed evaluation error	Covered by <code>tests/test_tarl_context_security_boundaries.py</code>
C067	Shadow trusted runtime state by naming a quantifier binder <code>__tarl_trusted_now</code> or another internal identifier	The <code>__tarl_*</code> binder namespace is reserved and rejected at parse and evaluation boundaries	Covered by <code>tests/test_tarl_context_security_boundaries.py</code>
C068	Derive a schema for one policy, then evaluate a different <code>policy_text</code> override under the stale base schema	Derived-schema origin is tracked; each differing override receives a fresh complete derivation and proof binding, or fails closed when derivation is incomplete	Covered by <code>tests/test_tarl_context_security_boundaries.py</code>

ID	Challenge	Expected defensive outcome	Current status
C069	Promote a fabricated, unsigned, tampered, wrong-policy, schema-unbound, differently approved, stale, replayed, expired, expiry-stripped, or request-unbound ESCALATE artifact	Quorum requires the exact request context, trusted aware time, freshness and replay enforcement, and independent signature verification against the exact policy/rule and a passed authoritative schema; each distinct approver signs the complete proof digest, and time-bound promotion preserves the verified signed expiry	Covered by tests/test_threat_model_lint_quorum.py
C070	Re-encode the same valid signature with uppercase or embedded whitespace to obtain a second replay identity	Proof signatures have one exact lowercase hexadecimal encoding; replay identity binds the complete canonical proof, key, algorithm, and decoded signature bytes, with durable legacy-ID compatibility	Covered by tests/test_tarl_proof.py::TestProofVerifier::test_ed25519_signature_encoding_cannot_bypass_replay_guard and tests/test_durable_state.py
C071	Reuse or promote a fresh proof after the governing policy window ended, or let a rule duration outlive that window	Every matched verdict and proof expires at the earliest exclusive policy or rule cutoff; verifier and quorum independently derive and enforce the same bound	Covered by tests/test_tarl_proof.py::TestProofExpiryBinding and tests/test_threat_model_lint_quorum.py::test_quorum_cannot_promote_after_governing_policy_cutoff
C072	Misspell a temporal date, rule duration, succession duration, or expiry verdict so the restriction is silently discarded	The parser rejects malformed temporal directives and durations; direct invalid policy models fail closed	Covered by tests/test_tarl_temporal.py::TestTemporalWindowParsing::test_malformed_temporal_metadata_is_rejected and TestRuleDurationParsing::test_malformed_rule_duration_is_rejected
C073	Configure on_expiry: ALLOW or present an unmatched positive proof to an authority consumer	Expiry cannot grant ALLOW, and positive authority requires a structurally coherent matched-rule trace plus passed context/schema binding	Covered by tests/test_broker_unified_gate.py::test_invalid_on_expiry_allow_cannot_authorize_a_brokered_effect and tests/test_tarl_proof.py::TestContextSchemaProofBinding::test_unmatched_allow_proof_is_never_context_authority

Mandatory Invariants

These invariants define the substrate. Violating any of them downgrades Thirsty-Lang from "active resistance runtime" to "policy library."

- 1 **No effect before verdict.** A side effect cannot happen before ALLOW.
- 2 **No policy means DENY.** Missing policy never grants authority.
- 3 **No authority means DENY.** Missing authority never grants authority.
- 4 **No proof means no audit claim.** Execution without proof cannot be called governed execution.
- 5 **No unsigned proof in hardened mode.** Hardened deployments require Ed25519.
- 6 **No stale proof as authority.** Proofs describe decisions; they are not reusable permission tokens unless a freshness policy says so.
- 7 **No adapter side doors.** All tool, file, network, subprocess, import, FFI, database, and model-action paths go through the broker.
- 8 **No catch-and-continue for governance denials.** Application error handling must not swallow a denial.
- 9 **No silent downgrade.** If signing, policy, audit, or identity verification is required and unavailable, the runtime fails closed.
- 10 **No unverifiable readiness claim.** Security claims cite tests or are labeled roadmap.
- 11 **Missing is not false. Invalid is not false. Unresolved is not evidence.** `MISSING`, `TYPE_ERROR`, and `REPRESENTATION_CONFLICT` never participate in comparison, negation, membership, functions, or boolean algebra; they short-circuit to DENY or ESCALATE with an explicit reason.
- 12 **One context representation.** Dotted policy names are paths into nested objects. A JSON key containing a dot is not an alternative spelling. Flat dotted keys, equal mixed forms, contradictory mixed forms, duplicate keys, and reserved-layer collisions fail closed; no conversion occurs silently.
- 13 **Positive proof context coherence.** A positive policy verdict is inadmissible as advancement authority unless the exact context representation used for schema validation is the representation evaluated by the policy engine and bound into the proof. The proof binds the original context hash, canonical context hash, representation identifier, normalization algorithm identifier and version, and collision/conflict status. Identity evaluation requires the original and canonical hashes to match.
- 14 **Strict JSON and type algebra.** Contexts and registered sources contain only JSON objects, arrays, strings, booleans, finite numbers, and null. Python-specific values and incompatible cross-kind coercions fail closed; resolved boolean `false` is never equal to numeric zero.
- 15 **Load-bearing consumer gate.** Plain evaluation may report `ALLOW`, but a broker or governed runtime advances only when the positive proof carries a passed schema binding for the evaluated representation.
- 16 **One explicit source transformation.** Callers cannot provide `source: *`. Registered sources may add valid top-level source fields only, and a verifier must bind both the original and evaluated contexts before accepting a transformed positive proof.
- 17 **Strict schema metadata.** Representation fields, path model, normalization, violation verdict, field kinds, required flags, and field names are validated without duplicate-key acceptance or coercion.
- 18 **No hidden invalid operand.** Boolean operators evaluate both operands, and quantifiers evaluate their predicate for every supplied element before accepting the aggregate result. Resolution/type failure anywhere in that evaluated surface, or an empty quantifier collection, fails closed.
- 19 **Type-strict finite numeric evaluation.** Integers and floats may interoperate, but strings never coerce to numbers. Numeric operands and arithmetic outputs must be finite; `NaN`, infinity, and overflow to infinity never participate in a verdict condition.
- 20 **Reserved evaluator bindings.** A policy cannot bind a quantifier variable in the `__tarl__` namespace used for trusted internal evaluation state.

- 21 **Policy/schema identity.** A derived schema belongs to the exact policy from which it was derived. A different policy override receives a fresh complete derivation and proof binding or is denied.
- 22 **Proof-bound quorum.** ESCALATE promotion requires a cryptographically signed proof independently verified against the exact policy, exact rule, passed schema, exact original request context, explicit trusted aware time, maximum proof age, and replay guard. Registered-source proofs also bind the exact evaluated context. Approvals bind the complete proof digest and are distinct by identity and key material. Time-bound promotion preserves its verified signed expiry.

Offensive Test Suites

Implemented this hardening pass:

Suite	Purpose
tests/test_threat_model_capability_broker.py	Asserts an import-only policy grants no fs/http/net/process/env/log/test/sqlite side effect (C029-C032)
tests/test_threat_model_broker.py	FFI/native, subprocess, and MCP/agent-tool effects must call <code>CapabilityBroker.require</code> ; denied by default (C033, C040-C041)
tests/test_threat_model_file_imports.py	Imported <code>.thirsty</code> modules run under the caller's governed gate, not a detached core interpreter (C035)
tests/test_threat_model_proof_strictness.py	Strict verification rejects unsigned/HMAC/wrong-key/tampered proofs and missing policy source (C025)
tests/test_threat_model_replay.py	Rejects replayed proofs by context hash, freshness window, and revoked policy hash (C023-C024)
tests/test_threat_model_authority.py	Authority must come from a signed <code>AuthorityClaim</code> ; bare <code>--authority/env</code> grants nothing in hardened mode (C027-C028)
tests/test_threat_model_context_schema.py	Missing or type-confused context fields fail closed before rule evaluation (C045-C046)
tests/test_threat_model_audit_chain.py	Hash-linked append-only audit; edits, deletions, and reordering break <code>verify_chain</code> (C022/C026/C049)
tests/test_threat_model_pathguard.py	<code>PathGuard</code> confines canonical (symlink-resolved) paths; traversal/symlink escape denied (C042)
tests/test_threat_model_clock.py	Temporal windows evaluate against signed, zoned <code>TrustedClock</code> time; invalid configured clocks deny without host fallback (C043)
tests/test_threat_model_failclosed.py	Evaluator errors and failed required-audit writes surface as non-swallowable denials (C037-C038)
tests/test_threat_model_lint_quorum.py	<code>lint_policy</code> flags broad/ungated ALLOW; <code>QuorumResolver</code> requires current-request binding, explicit trusted time, freshness/replay enforcement, and an independently verified signed policy/rule/schema-bound ESCALATE proof with distinct complete-digest approvals and preserved signed expiry (C039/C050/C069)
tests/test_threat_model_build_outputs.py	Governance-dropping build targets are refused for governed source unless explicitly opted in and disclosed (C034)
tests/test_threat_model_parser_fail_closed.py	Governed parse errors fail closed: no executable statements survive recovery (C036)
tests/test_proof_obligations.py	Static proof-obligation extraction, derived schema, build-manifest proof metadata, denial explanation, no-side-effect prove path, and proof/audit regression coverage (C051-C052)
tests/test_peer_review_0_8_1_tarl_regressions.py	TARL adversarial peer-review regressions for comparison typing, fail-closed evaluation, secure verifier defaults, trusted CLI time, and auto-TARL action context (C053-C057)
tests/test_tarl_context_resolution_integrity.py	Preserved context matrix across <code>SafeExpr</code> , evaluator, runtime, CLI, schema/proof obligations, proof creation/verification, and governed runtime; missing/type/conflict bypass regressions (C058-C060)
tests/test_tarl_context_security_boundaries.py	Strict JSON/type algebra, eager expression integrity, reserved binders, load-bearing schema authority, registered-source transformation, strict explicit-schema metadata, and per-override derivation (C061-C068)

The earlier "still required" suites (proof replay, policy downgrade, context poisoning, archive tamper, agent tools, resource failure) were implemented under the consolidated filenames above: replay -> `_replay.py`, downgrade -> `_proof_strictness.py`, context poisoning -> `_context_schema.py`, archive tamper -> `_audit_chain.py`, agent tools -> `_broker.py`, resource failure -> `_failclosed.py`. No offensive suite remains deferred; see Remaining Gaps for breadth/operational items.

Current Defensive Evidence

The following surfaces are implemented and tested today:

- Governed `pour`, `sip`, `import`, callable `print(...)`, and sensitive imported stdlib calls fail closed without policy authority.
- Governed denials carry `TarlProof`.
- Contract ALLOW and DENY decisions carry proof records.
- `GovernanceViolation` is not swallowed by `spillage`.
- Ed25519 proof signing and public-key verification are implemented.
- HMAC proof signing remains supported but documented as symmetric and not non-repudiable.
- Policy hash, trace consistency, malformed proof, and signature verification checks are covered by tests.
- Temporal policy windows and first-match-wins semantics are covered by tests.
- Imported `.thirsty` modules execute under the caller's governed runtime, so their top-level effects and returned closures are gated (not run detached).
- Proof verification requires a signature by default, can restrict the signature family to Ed25519 (`ProofVerifier / tarl verify --ed25519-only`), and only accepts unsigned proofs when `require_signature=False` or `--allow-unsigned` is explicit. Signature permissiveness does not admit a positive legacy proof that lacks coherent context-representation binding.
- The active source uses nested objects as the sole context representation. Dotted policy references resolve as paths; flat dotted keys and mixed forms fail closed before any rule can match.
- Runtime proofs bind both original and evaluated context hashes plus the representation and transformation identity. The verifier rejects positive proofs whose binding metadata is missing, contradictory, or unsupported.
- `CapabilityBroker` and the governed interpreter call the runtime's schema assurance path and independently reject a positive proof that is not context-authority admissible. Plain evaluation without a schema remains available, but its ALLOW cannot authorize an effect through those consumers.
- Registered-source verification requires the expected original and evaluated contexts and proves that removing only valid top-level `source:<name>` fields reproduces the original request exactly.
- Boolean evaluation checks both operands, and ALL/ANY evaluates every supplied element's predicate before accepting the aggregate result. Numeric comparisons are type-strict, numeric operands/arithmetic outputs must be finite, empty quantifier collections fail closed, and binders cannot use the reserved `__tarl_*` namespace.
- A derived context schema is bound to the policy that produced it. Differing `policy_text` overrides receive a fresh complete derivation and proof binding; incomplete override derivation denies.
- Quorum promotion independently verifies the exact policy-bound, exact-rule, schema-passed, signed ESCALATE proof before counting distinct approvals. Each approval signs the digest of the complete proof artifact, and a time-bound promotion preserves its verified expiry.
- Governance-dropping build targets are refused for governed source unless `--allow-governance-loss` is given, which warns and records the loss in the emitted manifest.

- Governed modules that fail to parse fail closed: no recovered statement executes, and the interpreter raises a denial.
- `thirsty prove program.thirsty --policy policy.tarl` parses and checks source and emits a machine-readable proof-obligation report without executing program side effects. The report lists functions, imports, sensitive stdlib calls, governed calls, required TARL actions, derived or attached context schema, authority requirements, contract obligations, proof mode, audit requirement, Shadow Thirst status, governance-loss status, and unresolved proof gaps.
- `thirsty explain-denial program.thirsty --policy policy.tarl` reports which policy, context, authority, or proof condition is missing for the static obligation set.

Remaining Gaps

Release 0.8.6 covers C058-C073 and replaces the vulnerable 0.8.5 evaluator. The tagged commit passed the branch, tagged-release, PyPI, and multi-architecture container workflows; a fresh PyPI installation reproduced the permanent context matrix. The Competence Register still keeps context coherence and resolution integrity at critical FAIL until an independent constitutional acceptance records that the released artifact satisfies the gate. Other remaining work is:

- 1 **Adapter rollout.** Shipped governed stdlib effects and the reference external adapters use `CapabilityBroker`; deployments must preserve that path for every additional adapter they introduce.
- 2 **Durable-state placement.** SQLite-backed replay/revocation stores and audit checkpoints are implemented, but deployments must place them on shared, access-controlled storage and export checkpoints to an independently trusted location.
- 3 **Trust-root custody.** Key formats, generation, loading, and rotation flows are implemented; deployments still own private-key custody, distribution of trusted public keys, and rotation/revocation operations.
- 4 **Independent acceptance.** Review the released 0.8.6 artifacts and recorded matrix evidence under an authority independent of the implementing agent; only that acceptance may unblock load-bearing positive verdict reliance.

Proof-Obligation Behavior Contracts

- Plain evaluation can run without a schema, but its positive result is not advancement authority. Load-bearing broker and governed-runtime consumers require a passed schema binding and attempt only complete schema derivation. `thirsty prove` derives simple field kinds from TARL policy references.
- Ambiguous context references remain incomplete and fail closed; use an explicit schema file for policy expressions the static derivation cannot prove.
- `thirsty prove` is intentionally static: it produces the proof-obligation report before execution and does not replace runtime TARL verdict enforcement, proof verification, broker mediation, or audit append behavior.

Acceptance Bar For Hardened Runtime

Thirsty-Lang can claim hardened governance-substrate status when:

- 1 Every challenge in the catalog is passing with a test or documented out of scope. **Met in released 0.8.6** - C001-C073 have coverage; independent Competence Register acceptance for C058-C073 remains open.
- 2 All side-effect adapters are mediated by the same broker. **Met (mechanism)** - `utf.tarl.broker.CapabilityBroker`; deployment adapter rollout tracked in Remaining Gaps #1.
- 3 Hardened mode requires Ed25519 proof signatures. **Met** - `Interpreter.set_hardened()` fails closed without authenticated authority and Ed25519-signed proofs (`tests/test_threat_model_authority.py`).
- 4 Policy and context schemas use the representation evaluated by the engine. **Met in released 0.8.6** - `utf.tarl.context`, `utf.tarl.schema`, and `tests/test_tarl_context_resolution_integrity.py`; the

load-bearing consumer gate is covered by `tests/test_tarl_context_security_boundaries.py`; independent Competence Register acceptance remains open.

- 5 Audit persistence is hash-linked and tamper-evident. **Met** - `TarlAuditArchive.verify_chain` (`tests/test_threat_model_audit_chain.py`).
- 6 Replay and downgrade attacks are rejected. **Met** - `tests/test_threat_model_replay.py`, `tests/test_threat_model_proof_strictness.py`.
- 7 The full offensive challenge suite passes locally and in CI. **Met for 0.8.6** - the local gate reported 1,463 passed, 1 skipped, 22 subtests, and 90.60% coverage; branch CI run `30756636768` and tagged release validation run `30756676570` completed successfully.

PART 08 / STATUS-REGISTER

Feature Status and Bootstrap Competence Register

Maintained source: docs/STATUS.md



Edition: Thirsty-Lang 0.8.6. The integrated navigation and reference layer is [THIRSTY LANG UTF 101.md](#); security acceptance details remain authoritative in [THREAT_MODEL.md](#).

Every capability below is marked **Real** (implemented and enforced, with a test that proves it) or **Roadmap** (reserved surface, not yet enforced). The test reference is the authority - if a row says Real, the cited test fails when the behaviour regresses. Run the whole matrix with:

TEXT

```
pytest tests/ -q
```

The optional Z3 layer is exercised only when the `analysis` extra is installed (`pip install thirsty-lang[analysis]`); its tests `importorskip("z3")` otherwise.

Bootstrap Competence Register

This register distinguishes verdict execution from context-contract integrity. The PASS entries are evidenced by the preserved 0.8.5 package reproduction. The repairs shipped in 0.8.6 at commit [60f226a69059b5803b0035f037d09e6f7f9c45a2](#); the branch CI, PyPI release, and multi-architecture GHCR workflows all passed, and a fresh PyPI installation reproduced the permanent ALLOW/DENY matrix. The critical FAIL entries remain in place because release evidence is not the same thing as independent constitutional acceptance. Until that acceptance is recorded, they continue to block load-bearing positive authority. Regression authority is provided by `tests/test_tarl_context_resolution_integrity.py`, `tests/test_tarl_context_security_boundaries.py`, and `tests/test_threat_model_lint_quorum.py`.

YAML

```

- competence_id: CR-TARL-ALLOW
  title: Positive policy verdict executes
  status: PASS
  evidence:
    - nested dotted-path case returned ALLOW
    - simple identifier case returned ALLOW

- competence_id: CR-TARL-DENY
  title: Negative and default-deny paths execute
  status: PASS
  evidence:
    - flat dotted-key mismatch returned DENY
    - guest control returned DENY

- competence_id: CR-CONTEXT-COHERENCE
  title: Schema, CLI, evaluator, proofs, and documentation share one context model
  status: FAIL
  severity: critical
  repair_status: released_in_0_8_6_pending_independent_acceptance
  blocks:
    - policy_engine_load_bearing_authority
    - trusted_state_promotion
    - signed_positive_verdict_reliance

- competence_id: CR-CONTEXT-RESOLUTION-INTEGRITY
  title: Context resolution preserves missing and invalid states
  
```



```
status: FAIL
severity: critical
security_impact: confirmed_authorization_bypass
repair_status: released_in_0_8_6_pending_independent_acceptance
blocks:
  - all_dotted_path_policy_authority
  - competence_advancement_authorization
  - trusted_state_promotion
  - positive_verdict_proof_reliance
```

Preserved reproduction evidence manifest SHA-256:
4DD3E6A4CF017E3E34A25C62A398D29A6F93D279890C4CCDF2A4528C6333F8D5.

Language core (Thirsty-Lang)

Capability	Status	Test reference
Array / reservoir literals [1, 2, 3] evaluate to a real list	Real	tests/test_thirsty_lang.py; tests/test_examples.py
flood / evaporate / new over reservoirs and fountains	Real	tests/test_thirsty_lang.py; tests/test_examples.py
Variable reassignment (x = ...) mutates the binding	Real	tests/test_thirsty_lang.py
OOP: member read/write, method dispatch (obj.f(), obj.x = ...)	Real	tests/test_thirsty_lang.py; tests/test_examples.py (gods.thirstofgods)
cascade awaits and yields a value (not a Future)	Real	tests/test_examples.py
spillage / error / cleanup / finally control flow	Real	tests/test_examples.py; tests/test_thirst_of_gods.py
refill (x in xs) loop accumulation	Real	tests/test_examples.py
UTF-8-safe CLI output on Windows (cp1252)	Real	src/utf/console.py (enable_utf8), shared by all CLIs
Every shipped example parses, type-checks, and runs clean	Real	tests/test_examples.py
let (immutable binding), for ... in keyword loop, := (define mutable)	Real	tests/test_language_features.py
strict (requires initialization) / pure (no I/O) module modes	Real	tests/test_language_features.py
Self- and mutual recursion (correct arity, no false E030)	Real	tests/test_language_fixes.py
this keyword, member assignment, fountain field default initializers	Real	tests/test_language_fixes.py
Closures capture their lexical (defining) scope	Real	tests/test_language_fixes.py
> pipe operator (and bare); error (name) binds the thrown value	Real	tests/test_language_fixes.py
times N { ... } repeat loop	Real	tests/test_new_language_features.py
C-style refill(init; cond; step) loop (counter implicitly mutable)	Real	tests/test_new_language_features.py
Anonymous functions / lambdas glass(params) { ... } as values	Real	tests/test_new_language_features.py
Subscript indexing xs[i] (reservoir/dict/string), read + assign, bounds-checked	Real	tests/test_review_0_8_1.py
Un-annotated functions return Any (value-use + recursion type-check)	Real	tests/test_review_0_8_1.py
Combine ^ require both operands bool; mixed bool/non-bool rejected (no fail-open)	Real	tests/test_review_0_8_1.py

Governance (maximal)

Capability	Status	Test reference
requires precondition on governed functions	Real	tests/test_governance_maximal.py::TestContracts
ensures postcondition (result bound after the body)	Real	tests/test_governance_maximal.py::TestContracts::test_ensures_*
invariant checked at entry and exit	Real	tests/test_governance_maximal.py::TestContracts::test_invariant_entry_and_exit
Contracts on methods (design-by-contract, any mode)	Real	tests/test_governance_maximal.py::TestContracts::test_method_contract_any_mode
Capability gates: imports + I/O routed through TARL, deny-by-default	Real	tests/test_governance_maximal.py::TestCapabilityGates
Sensitive imported stdlib calls require their own capability verdict after import	Real	tests/test_gate_fail_closed.py::test_import_allow_does_not_grant_sensitive_stdlib_calls
Denials carry a TarlProof; proofs are unsigned unless runtime signing is configured	Real	tests/test_governance_maximal.py::TestCapabilityGates::test_write_denied_with_proof; tests/test_gate_fail_closed.py
Temporal windows govern a call (allow/deny)	Real	tests/test_governance_maximal.py::TestTemporal
Static E053 for a governed call from core mode	Real	tests/test_governance_maximal.py::TestStatic::test_e053_governed_call_from_core
Forward-reference / mutual-recursion hoisting	Real	tests/test_governance_maximal.py::TestStatic::test_forward_reference_resolves
Offensive threat model and challenge catalog	Real	docs/THREAT_MODEL.md
Imported <code>.thirsty</code> modules run under the caller's governed gate (not detached core)	Real	tests/test_threat_model_file_imports.py
Governed module with a parse error fails closed (no statements execute)	Real	tests/test_threat_model_parser_fail_closed.py
Secure-by-default proof verification (signature required unless unsigned mode is explicit; Ed25519-only / policy source options)	Real	tests/test_threat_model_proof_strictness.py; tests/test_peer_review_0_8_1_tarl_regressions.py
Governed build refuses governance-dropping targets unless explicitly disclosed	Real	tests/test_threat_model_build_outputs.py
Import-only policy grants no sensitive stdlib side effect	Real	tests/test_threat_model_capability_broker.py
Static proof-obligation manifest (<code>thirsty prove</code>) without executing side effects	Real	tests/test_proof_obligations.py
Derived context schema from TARL policy references, fail-closed when incomplete	Real	tests/test_proof_obligations.py; tests/test_threat_model_context_schema.py
Denial explanation for missing policy/context/authority/proof conditions	Real	tests/test_proof_obligations.py
Build manifest records source/policy hashes, capabilities, schema, proof/audit requirements, Shadow status, and governance-loss status	Real	tests/test_proof_obligations.py; tests/test_threat_model_build_outputs.py
TARL numeric comparison is type-strict: int/float interoperate; strings never coerce to numbers	Real in released 0.8.6; independent acceptance pending	tests/test_peer_review_0_8_1_tarl_regressions.py; tests/test_tarl_context_security_boundaries.py
TARL policy load and runtime evaluation reject or deny malformed/unsafe rules instead of falling through	Real	tests/test_peer_review_0_8_1_tarl_regressions.py; tests/test_tarl.py::TestThrowStats
Runtime and <code>tarl eval</code> require valid zoned trusted time for configured clocks, policy windows, time-bound verdicts, <code>CURRENT_*</code> , and <code>ELAPSED_SINCE</code> ; invalid configured clocks never fall back to host time	Real	tests/test_threat_model_clock.py; tests/test_peer_review_0_8_1_tarl_regressions.py
<code>thirsty govern --auto-tarl</code> emits policies keyed on runtime action context	Real	tests/test_peer_review_0_8_1_tarl_regressions.py

Hardened runtime (offensive catalog C022-C073)

Capability	Status	Test reference
Authenticated authority provenance; hardened mode requires signed authority + Ed25519 proofs	Real	tests/test_threat_model_authority.py
Context schema validation fails closed on missing/type-confused fields	Real	tests/test_threat_model_context_schema.py
One authoritative nested context representation across schema, CLI, evaluator, proof, and governed runtime	Real in released 0.8.6; independent acceptance pending	tests/test_tarl_context_resolution_integrity.py
Missing, malformed, and conflicting context states short-circuit to DENY and never enter boolean algebra	Real in released 0.8.6; independent acceptance pending	tests/test_tarl_context_resolution_integrity.py
Positive proofs bind original/canonical hashes, representation, algorithm/version, and conflict status	Real in released 0.8.6; independent acceptance pending	tests/test_tarl_context_resolution_integrity.py
Context and registered sources use strict finite JSON; incompatible expression kinds cannot coerce into ALLOW	Real in released 0.8.6; independent acceptance pending	tests/test_tarl_context_security_boundaries.py
Broker and governed runtime require a passed, proof-bound explicit or complete derived schema before ALLOW can advance authority	Real in released 0.8.6; independent acceptance pending	tests/test_tarl_context_security_boundaries.py
Registered-source proofs preserve original/evaluated contexts and verification permits only valid source-field additions	Real in released 0.8.6; independent acceptance pending	tests/test_tarl_context_security_boundaries.py; tests/test_tarl_proof.py
Explicit schema representation and field metadata are parsed strictly without duplicate-key acceptance or coercion	Real in released 0.8.6; independent acceptance pending	tests/test_tarl_context_security_boundaries.py
Boolean operands and every supplied quantifier element are validated eagerly, and empty quantifier collections fail closed, so decisive or vacuous results cannot hide resolution/type failures	Real in released 0.8.6; independent acceptance pending	tests/test_tarl_context_security_boundaries.py
Numeric operands and arithmetic results must remain finite; strings never coerce to numbers	Real in released 0.8.6; independent acceptance pending	tests/test_tarl_context_security_boundaries.py
Quantifier binders cannot shadow the reserved <code>__tarl_*</code> evaluator namespace	Real in released 0.8.6; independent acceptance pending	tests/test_tarl_context_security_boundaries.py
When the runtime schema is derived, a differing <code>policy_text</code> override receives its own complete derived schema and proof binding, or fails closed; an explicitly attached schema remains caller-authoritative	Real in released 0.8.6; independent acceptance pending	tests/test_tarl_context_security_boundaries.py
Quorum promotion requires the exact request context, trusted aware time, freshness and replay enforcement, plus an independently verified, signed, exact-policy/rule, schema-passed ESCALATE proof with distinct approvals over its complete digest and preserved signed expiry	Real in released 0.8.6; independent acceptance pending	tests/test_threat_model_lint_quorum.py
Replay/freshness/revocation rejection in the proof verifier	Real	tests/test_threat_model_replay.py
Canonical proof signatures and complete semantic replay identity prevent alternate-encoding reuse	Real in released 0.8.6; independent acceptance pending	tests/test_tarl_proof.py; tests/test_durable_state.py
Temporal directives parse strictly, expiry cannot grant ALLOW, and proof/quorum authority is capped at the earliest exclusive policy or rule cutoff	Real in released 0.8.6; independent acceptance pending	tests/test_tarl_temporal.py; tests/test_tarl_proof.py; tests/test_threat_model_lint_quorum.py; tests/test_broker_unified_gate.py
Tamper-evident hash-linked audit chain (<code>verify_chain</code>)	Real	tests/test_threat_model_audit_chain.py
Universal capability broker (FFI/native + agent/MCP tools)	Real	tests/test_threat_model_broker.py
Fail-closed under evaluator error / required-audit failure	Real	tests/test_threat_model_failclosed.py
Filesystem path confinement (traversal/symlink escape)	Real	tests/test_threat_model_pathguard.py
Broad-ALLOW policy lint + proof-bound signed ESCALATE quorum	Real in released 0.8.6; independent acceptance pending	tests/test_threat_model_lint_quorum.py
Trusted signed-time source for temporal policy	Real	tests/test_threat_model_clock.py
CLI: <code>tarl lint</code> , <code>tarl audit verify-chain</code> , <code>strict tarl verify</code> flags	Real	tests/test_cli_tarl_hardenning.py

Semantic verifiers

Capability	Status	Test reference
Convergence: structural (alpha-renamed AST) pre-check	Real	tests/test_verifiers.py::TestConvergence::test_structural_alpha_equivalent_promotes
Convergence: Z3 symbolic proof + counterexample (arith subset)	Real (opt)	tests/test_verifiers.py::TestConvergenceZ3
Convergence: execute-and-compare over seeded inputs	Real	tests/test_verifiers.py::TestConvergence::test_execute_and_compare_finds_diverging_input
Convergence abstains on blocks with observable effects	Real	tests/test_verifiers.py::TestConvergence::test_execute_and_compare_abstains_on_effects
Determinism: taint dataflow follows non-determinism through aliases	Real	tests/test_verifiers.py::TestEffectPass
Thirst of Gods: each cascade linked to an enclosing spillage handler	Real	tests/test_verifiers.py::TestCascadeLinking; tests/test_thirst_of_gods.py
Shadow Thirst 6-analyzer promote/reject flow (AST-based)	Real	tests/test_shadow_thirst.py

T.A.R.L. policy engine

Capability	Status	Test reference
Policy parsing, first-match-wins evaluation, verdicts	Real	tests/test_tarl.py
HMAC-signed proof certificates	Real	tests/test_tarl_proof.py
Ed25519-signed proof certificates	Real	tests/test_tarl_proof.py
Temporal windows (<code>valid_from/valid_until</code> , durations)	Real	tests/test_tarl_temporal.py
Policy composition	Real	tests/test_tarl_composition.py
Z3 static analysis (coverage / shadows / conflicts / equiv / refines)	Real (opt)	tests/test_tarl_analyzer.py

Stability

Capability	Status	Test reference
Lint gate (ruff) clean under the project config	Real	CI lint-and-test job; ruff check src tests
Full suite on 3.11 + 3.12 with a coverage floor	Real	CI lint-and-test job (--cov-fail-under=90)
Examples executed through their CLIs in CI	Real	CI lint-and-test job
Package builds and imports cleanly	Real	CI package-smoke job

PART 09 / SECURITY-POLICY

Security Policy

Maintained source: SECURITY.md



Supported Versions

Thirsty-Lang is currently in alpha (v0.8.6). Security updates are applied to the latest release only.

Versions through 0.8.5 must not be used for load-bearing authorization with dotted context paths. Upgrade to 0.8.6 for fail-closed missing, malformed, and conflicting context handling. The repaired release also fails closed on hidden invalid boolean/quantifier values, empty quantifier collections, non-finite numeric results, stale per-policy derived schemas, and inadmissible quorum proofs. Quorum promotion additionally requires the exact request context, trusted verification time, proof freshness, and replay enforcement. Proof signatures have one canonical lowercase hexadecimal encoding, and replay identity binds the complete proof semantics plus decoded signature bytes. Configured trusted clocks and CLI trusted-time values must be timezone-aware; an invalid clock fails closed without host-clock fallback. Malformed temporal directives are rejected, expiry can never grant `ALLOW`, and proof authority is capped at the earliest exclusive policy or rule cutoff. The 0.8.6 package, branch CI, release workflow, GHCR build, and fresh-install regression matrix passed. Independent constitutional acceptance remains required before the Competence Register can unblock load-bearing positive authority.

Version	Supported
0.8.6	:white_check_mark:
0.8.5 and earlier	:X:

Reporting a Vulnerability

Thirsty-Lang takes security seriously. The project is designed around governance-first principles where security is a first-class concern at every tier.

If you discover a security vulnerability in Thirsty-Lang, please report it privately before disclosing it publicly.

How to Report

Do not report security vulnerabilities through public channels.

Send a detailed report to FounderOfTP@thirstysprojects.com.

What to Include

To help us respond quickly, please include:

- Type of vulnerability
- Full reproduction steps
- Affected tiers and components
- Potential impact
- Any suggested mitigations (if known)

Response Timeline

- **Acknowledgment:** Within 48 hours of receiving your report
- **Investigation:** Within 5 business days
- **Fix and Release:** Timeline depends on severity and complexity
- **Public Disclosure:** After a fix is released, typically within 30 days

Disclosure Policy

We follow a coordinated disclosure process:

- 1 Reporter submits vulnerability details
- 2 We acknowledge receipt within 48 hours
- 3 We investigate and develop a fix
- 4 A security release is prepared and published
- 5 The vulnerability is publicly disclosed after the fix is available

Security-Relevant Components

The following shipped components have security implications and receive priority attention:

- **T.A.R.L. (Tier 3):** Policy enforcement engine with default-DENY
- **Shadow Thirst (Tier 4):** Mutation analysis and invariant verification
- **TSCG/TSCG-B (Tiers 5-6):** Symbolic constraint grammar and binary protocol
- **Capability broker and governed interpreter:** load-bearing positive verdict admission and sensitive effect mediation
- **Authority, verifier, durable stores, and audit archive:** signature, replay, revocation, time, and persistence controls

The top-level [governance/](#) tree and [src/psia](#) are repository-local surfaces; they are not included in the published 0.8.6 Python package and are not part of its installed security boundary.

Offensive Threat Model

The adversary model and challenge catalog are maintained in [docs/THREAT_MODEL.md](#). Security claims should map to a challenge ID, a passing test, or a roadmap gap. In hardened use, a missing policy, missing authority, missing proof, failed signature verification, or stale proof must fail closed. Durable audit persistence is an explicit embedding control: attach `TarLAuditArchive` and call `set_require_audit(True)`. If an attached required archive cannot persist the proof, the runtime denies; `--hardened` alone does not create an audit sink.

Default DENY

Thirsty-Lang's core security principle is **default DENY at every governance gate**. A governed action cannot advance through the capability broker without the policy, context, authority, and proof required by that gate. The symbolic and binary tiers provide analysis or integrity services; they should not be mistaken for independent authorization gates.

Security Best Practices

- Always run untrusted code in governed mode
- Review T.A.R.L. policies before deployment

- Verify Shadow Thirst promotion results
 - Attach a durable audit archive, enable required persistence, checkpoint it, and verify its hash chain
 - Follow [docs/PRODUCTION_DEPLOYMENT.md](#) and the [canonical Thirsty-Lang UTF 101 manual](#)
-

Thirsty's Projects LLC

PART 10 / OPERATOR-GUIDE

Container Operations

Maintained source: DOCKER.md

SOURCE



CHECK



POLICY



VERDICT



PROOF



EFFECT

Current Published Image

The authoritative container for Thirsty-Lang 0.8.6 is:

TEXT

```
ghcr.io/iamsothirsty/thirsty-lang:0.8.6
```

It is a multi-architecture image for `linux/amd64` and `linux/arm64`. The published manifest is anchored by this digest:

TEXT

```
sha256:6f3f516b8e979437dd414373afe581716b8c890dc4758b8675cbbcad9b94b13c
```

The v0.8.6 Docker workflow completed successfully in GitHub Actions run 30756676576.

Docker Hub is not a current distribution channel. Its `latest` and `0.8.1` tags are historical and unmaintained. Do not use `thirstyofthirsty/thirsty-lang` when validating or deploying 0.8.6.

Prefer the immutable version tag for normal use. For the strongest artifact pin, use the manifest digest:

BASH

```
docker pull ghcr.io/iamsothirsty/thirsty-lang:0.8.6
docker pull ghcr.io/iamsothirsty/thirsty-lang@sha256:6f3f516b8e979437dd414373afe581716b8c890dc4758b8675cbbcad9b94b13c
```

Quick Start

Verify the CLI and run the demo

BASH

```
docker run --rm ghcr.io/iamsothirsty/thirsty-lang:0.8.6 --version
docker run --rm ghcr.io/iamsothirsty/thirsty-lang:0.8.6 run --demo
```

Start an interactive REPL

BASH

```
docker run -it --rm ghcr.io/iamsothirsty/thirsty-lang:0.8.6 repl
```

Run a script from the current directory

macOS/Linux:

BASH

```
docker run --rm -v "$(pwd):/work" \
  ghcr.io/iamsothirsty/thirsty-lang:0.8.6 \
  run /work/myfile.thirsty
```

PowerShell:

POWERSHELL

```
docker run --rm -v "${PWD}:/work" `
ghcr.io/iamsothirsty/thirsty-lang:0.8.6 `
run /work/myfile.thirsty
```

Local Development

Build a local image without publishing it:

BASH

```
docker build -t thirsty-lang:dev .
```

Run the full test service:

BASH

```
docker compose run --rm test
```

Open the development shell with the repository source mounted:

BASH

```
docker compose run --rm dev
```

Format the project or run its health check:

BASH

```
docker compose run --rm fmt
docker compose run --rm doctor
```

Docker Compose Services

`docker-compose.yml` defines exactly eight services:

Service	Purpose
<code>repl</code>	Interactive Thirsty-Lang REPL
<code>test</code>	Full Python test suite
<code>thirsty</code>	Execute the <code>thirsty</code> CLI
<code>build-js</code>	Compile a Thirsty-Lang source file to JavaScript
<code>dev</code>	Development shell with mounted source
<code>doctor</code>	Project health check
<code>fmt</code>	Source formatter
<code>lsp</code>	TARL language server on port 9898

List the services from the active Compose configuration:

BASH

```
docker compose config --services
```

Run any service with:

BASH

```
docker compose run --rm <service>
```

Image Contents

The runtime image is based on `python:3.11-slim`, runs as the non-root `thirsty` user with UID 1000, and uses the `thirsty` CLI as its entrypoint. It includes all seven installed command-line entry points:

- `thirsty` - core and governed runtime, build, proof, audit, and tooling

- `thirst-of-gods` - divine contract validation
- `tarl` - policy evaluation, proof verification, and audit tools
- `shadow-thirst` - mutation and invariant analysis
- `tscg` - symbolic constraint grammar tools
- `tscg-b` - binary frame protocol tools
- `tarl-lsp` - TARL Language Server Protocol server

Governance Example

Mount the program, policy, and context schema into the container and invoke the governed runtime explicitly:

BASH

```
docker run --rm -v "$(pwd):/work" \
  ghcr.io/iamsothirsty/thirsty-lang:0.8.6 \
  run /work/program.thirsty \
  --thirst-level governed \
  --policy /work/policy.tarl \
  --context-schema /work/context.schema.json \
  --authority myapp
```

For a load-bearing deployment, also follow the context, proof, key, freshness, replay, and revocation requirements documented in [README.md](#), [docs/THREAT_MODEL.md](#), and [docs/STATUS.md](#).

Release Publishing

The Docker workflow is defined in [.github/workflows/docker.yml](#) and supports:

- `v*` tags for a coordinated PyPI and container release;
- `docker-v*` tags for a Docker-only publication; and
- manual workflow dispatch for an explicitly supplied version.

Every published workflow build targets `linux/amd64` and `linux/arm64`. The Dockerfile runs the test suite before constructing the runtime stage. The workflow then pulls the GHCR image and runs both `--version` and `run --demo`.

For a normal versioned release:

BASH

```
git tag -a v<version> -m "Thirsty-Lang <version>"
git push tp refs/tags/v<version>
```

For a Docker-only rebuild of an already released source state:

BASH

```
git tag -a docker-v<version> -m "Thirsty-Lang Docker <version>"
git push tp refs/tags/docker-v<version>
```

Do not publish from an unverified working tree. Complete the checklist in [SHIPPING.md](#) before creating either tag.

Docker Hub publication is conditional on repository secrets. It is not part of the verified 0.8.6 distribution and should remain undocumented as a current consumer endpoint until its versioned and `latest` tags are rebuilt and independently verified.

Local Helper Scripts

macOS/Linux:

BASH

```
bash docker-quick.sh build
bash docker-quick.sh run --demo
bash docker-quick.sh repl
bash docker-quick.sh test
bash docker-quick.sh dev
bash docker-quick.sh fmt
bash docker-quick.sh doctor
bash docker-quick.sh version
```

Windows:

BAT

```
docker-quick.bat build
docker-quick.bat run --demo
docker-quick.bat repl
docker-quick.bat test
docker-quick.bat dev
docker-quick.bat fmt
docker-quick.bat doctor
docker-quick.bat version
```

The helper scripts also expose a local cleanup command. Review the targeted Docker objects before using it on a machine with other active image work.

Troubleshooting

Image not found

Confirm the GHCR path and explicit version:

BASH

```
docker pull ghcr.io/iamsothirsty/thirsty-lang:0.8.6
```

Do not substitute the stale Docker Hub image.

Build fails while running tests

The Dockerfile has a test gate in its builder stage. A failing test prevents the runtime image from being built. Run the same suite with:

BASH

```
docker compose run --rm test
```

Mounted files are not writable

The runtime image intentionally runs as `thirsty:1000`. Ensure the mounted directory permits access by that UID, or use the development service appropriate for your host.

Confirm the published architectures

BASH

```
docker buildx imagetools inspect ghcr.io/iamsothirsty/thirsty-lang:0.8.6
```

The manifest must list both `linux/amd64` and `linux/arm64` before it is described as the current multi-architecture release.

PART 11 / CONTRIBUTOR-GUIDE

Contributor and Validation Workflow

Maintained source: CONTRIBUTING.md



Thank you for your interest in contributing to Thirsty-Lang. This guide tracks the checks enforced by the repository at version 0.8.6. Start with the [canonical Thirsty-Lang UTF 101 manual](#) for the complete language, governance, security, and operations map.

Code of Conduct

All contributors are expected to uphold a respectful, inclusive, and constructive environment. Harassment, discrimination, or any form of unprofessional behavior will not be tolerated.

When contributing, please:

- Be respectful and considerate of others
- Provide constructive feedback
- Focus on what is best for the community and the project
- Show empathy towards other community members

How to Contribute

Reporting Issues

If you find a bug or have a feature request, please email FounderOfTP@thirstysprojects.com with full details.

When reporting, please include:

- A clear description of the issue
- Steps to reproduce
- Expected behavior vs actual behavior
- Version information (output of `thirsty --version`)
- Any relevant error messages or stack traces

Submitting Changes

- 1 Fork the repository
- 2 Create a feature branch (`git checkout -b feature/amazing-feature`)
- 3 Make your changes
- 4 Run the test suite: `python -m pytest tests/ -v`
- 5 Commit your changes with clear commit messages
- 6 Push to your fork
- 7 Open a Pull Request

Development Setup

BASH

```
# Clone the repository
git clone https://github.com/IAMSoThirsty/Thirsty-lang.git
cd Thirsty-lang

# Create virtual environment
python3.11 -m venv .venv
source .venv/bin/activate # PowerShell: .\venv\Scripts\Activate.ps1

# Install code, analysis, and documentation dependencies
python -m pip install --upgrade pip
python -m pip install -e ".[dev,analysis,docs]"

# Verify installation
thirsty --version
thirst-of-gods --help
tarl --help
tarl-lsp --help
shadow-thirst --help
tscg --help
tscg-b --help
```

Development Workflow

Run the same code gates used by CI before committing:

BASH

```
# Format code, if your change touched Python
black src/ tests/

# Lint without silently rewriting the tree
ruff check src tests

# Type-check the installed package surface
mypy -p utf

# Run the release-equivalent suite and coverage floor
python -m pytest tests/ -q --cov=utf --cov-report=term-missing --cov-fail-under=90
```

The repository ships `.pre-commit-config.yaml`. Install the development extra and run every hook before delivery:

BASH

```
python -m pip install -e ".[dev]"
pre-commit run --all-files
```

Local hooks supplement the authoritative gates in `.github/workflows/smoke.yml` and `.github/workflows/release.yml`; they do not replace CI evidence.

For documentation or PDF changes, also run:

BASH

```
python scripts/build_thirsty_lang_utf_101.py
python scripts/verify_thirsty_lang_utf_101.py output/pdf/Thirsty-Lang-UTF-101.pdf
python -m pytest tests/test_thirsty_lang_utf_101_pdf.py -q
```

Code Standards

- Python 3.11+ required
- Keep runtime dependencies explicit and minimal; `cryptography>=41.0` is a required dependency for the shipped proof and authority surfaces
- All water-metaphor keywords implemented exactly per spec
- Every AST node must carry span tracking
- Tests are required for all new features
- Default = DENY at every governance gate

Pull Request Guidelines

- Keep pull requests focused on a single concern
- Add or update tests as needed
- Update documentation if the API or behavior changes
- Ensure all tests pass before requesting review
- If documentation changes a normative security or governance claim, update the canonical manual and its generated PDF in the same change

Governance

Thirsty-Lang follows a default-DENY governance model. All changes are reviewed for:

- Security implications
- Backward compatibility
- Specification compliance
- Test coverage adequacy

Questions?

If you have questions about contributing, please email FounderOfTP@thirstysprojects.com.

Thirsty's Projects LLC

PART 12 / RELEASE-GUIDE

Release Authentication and Provenance

Maintained source: docs/SIGNING.md



Edition: Thirsty-Lang 0.8.6 (2026-08-02)

This guide separates three mechanisms that protect different things:

Mechanism	Purpose	0.8.6 status
PyPI API token	Authenticates the upload account	Active
PyPI Trusted Publishing and publish attestations	Binds an uploaded file to a supported CI identity through OIDC	Not active for 0.8.6
TARL HMAC or Ed25519 signatures	Bind policy-verdict proofs inside the language runtime	Active; unrelated to package publication
Detached OpenPGP signature	Optional out-of-band signature distributed somewhere other than PyPI	Not produced by the release workflow

The current `.github/workflows/release.yml` validates the tagged commit, builds the wheel and source distribution, checks metadata, and publishes through the pinned PyPA action. Authentication uses the project-scoped PyPI API token in the GitHub `release` environment secret named `IAMSOTHIRSTY`. The repository does not contain that secret.

Current 0.8.6 publication contract

The release workflow enforces:

- 1 The Git tag version matches `pyproject.toml`.
- 2 Ruff and mypy pass.
- 3 The full test suite passes with at least 90 percent coverage.
- 4 The package builds and `twine check` accepts every distribution.
- 5 The pinned publisher action uploads with `__token__` credentials.

This is authenticated package publication, but it is not PyPI Trusted Publishing. Consequently the 0.8.6 files do not carry the Trusted Publisher publish-attestation claim described by PyPI.

Recommended future migration: Trusted Publishing

PyPI Trusted Publishing exchanges a GitHub Actions OIDC identity for a short-lived upload token. It removes the long-lived project API token from the workflow and enables PyPI's supported publish-attestation path.

Migration requires an external PyPI project configuration that exactly matches:

- owner: `IAMSoThirsty`
- repository: `Thirsty-lang`
- workflow: `release.yml`

- environment: `release`

After that external configuration is verified, remove the explicit `user` and `password` inputs from the PyPA publishing step and grant `id-token: write` to the publish job. Do not make only one side of this change: a workflow/PyPI identity mismatch fails publication.

The official PyPA publishing action generates and uploads supported attestations by default when it publishes through Trusted Publishing. A token- authenticated upload must not be described as having that OIDC provenance.

Optional detached OpenPGP signatures

OpenPGP signatures can still be useful when the project operates a separate, documented distribution channel for both the artifact and signature. They are not a substitute for PyPI Trusted Publishing.

Current Twine warns that PyPI and TestPyPI silently ignore attached PGP signatures. Do not use `twine upload --sign` or upload `.asc` files as evidence that PyPI users received or verified a signature.

If an out-of-band channel is intentionally established:

BASH

```
python -m build
gpg --detach-sign --armor dist/thirsty_lang-0.8.6-py3-none-any.whl
gpg --verify \
  dist/thirsty_lang-0.8.6-py3-none-any.whl.asc \
  dist/thirsty_lang-0.8.6-py3-none-any.whl
```

Use a dedicated protected release key, keep its passphrase and private material outside the repository, publish the complete fingerprint through an authenticated channel, document rotation and revocation, and test the consumer verification procedure. Never recommend an unprotected private release key.

TARL proof signing is separate

TARL proof signatures establish authority over an evaluated policy decision, not provenance for a wheel file. See the [canonical Thirsty-Lang UTF 101 manual](#) and [governance model.md](#) for proof canonicalization, HMAC compatibility, Ed25519 attribution, key identifiers, replay identity, expiry, and revocation.

References

- [PyPI Trusted Publishing](#)
- [PyPI: Producing attestations](#)
- [Twine upload implementation and PGP warning](#)
- [GNU Privacy Handbook](#)

PART 13 / ARCHITECTURE-RATIONALE

Architecture Whitepaper

Maintained source: docs/WHITEPAPER.md



Version: 0.8.6 · **Stack:** Universal Thirsty Family (UTF) **Copyright:** 2026 Thirsty's Projects LLC · **License:** Apache-2.0 **Status of claims:** every capability below is marked *Real* (implemented and covered by a test) or *Roadmap / Deferred* (reserved surface, not yet enforced). The test suite is the authority; see [STATUS.md](#) and [THREAT_MODEL.md](#). The integrated tutorial and reference is [THIRSTY_LANG_UTF_101.md](#).

Abstract

Most programming languages treat *authorization* as an application concern bolted on after the fact: a function runs, and somewhere inside it a check may or may not happen before it writes a file, opens a socket, or shells out. When code is increasingly **generated by AI agents**, that model fails closed nowhere - a plausibly-worded function can reach the filesystem, the network, or a subprocess with no gate in its path.

Thirsty-Lang inverts the default. In **governed** mode, a side effect cannot occur before an explicit **ALLOW** verdict from a separate policy engine (T.A.R.L.), and every governed decision emits a **proof record** binding the policy, the context, and the verdict. The record becomes tamper-evident when HMAC- or Ed25519-signed; hardened mode requires Ed25519. Missing policy, missing authority, a failed signature check, or an unavailable gate all resolve to **DENY**. Governance is the substrate, not a library call.

This paper describes the language, its governance model, the T.A.R.L. policy engine and proof system, the offensive threat model that scopes its guarantees, and the engineering state of the 0.8.6 release.

1. Motivation and threat setting

Thirsty-Lang targets a specific adversary class: **untrusted code and untrusted policy**, including AI-generated programs, prompt-injected agents, malicious plugins/tool adapters, and operators attempting unauthorized actions. The full adversary catalog (A1-A14) and a 73-entry offensive challenge catalog (C001-C073) are maintained in [THREAT_MODEL.md](#).

The design goal is not "mathematically unbreakable." It is: **no known reproducible critical/high bypass of the governed runtime remains, and every remaining gap is recorded with severity and a next step.** The threat catalog is the ledger that keeps that claim honest.

Mandatory invariants

These define the substrate. Violating any one downgrades Thirsty-Lang from an "active resistance runtime" to a "policy library":

- 1 **No effect before verdict.** A side effect cannot happen before ALLOW.
- 2 **No policy means DENY.** Missing policy never grants authority.
- 3 **No authority means DENY.** Missing authority never grants authority.
- 4 **No proof means no audit claim.** Execution without proof is not governed execution.
- 5 **No unsigned proof in hardened mode.** Hardened deployments require Ed25519.

- 6 **No stale proof as authority.** Proofs describe decisions; they are not reusable tokens.
- 7 **No adapter side doors.** Tool, file, network, subprocess, import, FFI, DB, and model-action paths go through the broker.
- 8 **No catch-and-continue for governance denials.** Application error handling cannot swallow a denial.
- 9 **No silent downgrade.** If signing, policy, audit, or identity verification is required and unavailable, fail closed.
- 10 **No unverifiable readiness claim.** Security claims cite tests or are labeled roadmap.

2. Architecture: the Universal Thirsty Family

UTF is a six-tier stack. Each tier adds capability while preserving default-DENY governance at every gate. The Python reference implementation ships them as one package (`utf`) with seven console entry points.

Tier	Name	Role	CLI
1	Thirsty-Lang (Core)	Water-metaphor language: lexer, parser, tree-walking interpreter, type checker	<code>thirsty</code>
2	Thirst of Gods	Async/effect-linked execution surface (<i>cascade/spillage</i>)	<code>thirst-of-gods</code>
3	T.A.R.L.	Thirsty's Active Resistance Language - the policy engine + proof system	<code>tarl</code> , <code>tarl-lsp</code>
4	Shadow Thirst	Mutation governance: prove a candidate computes the same thing as a validated canonical	<code>shadow-thirst</code>
5	TSCG	Thirsty Symbolic Constraint Grammar - symbolic integrity constraints	<code>tscg</code>
6	TSCG-B	TSCG-Binary - binary-frame integrity	<code>tscg-b</code>

Runtime: Python ≥ 3.11 , one required dependency (`cryptography`, for Ed25519/ HMAC proof signatures). The Z3 SMT layer used for static policy analysis and symbolic convergence is an **optional** extra (`pip install thirsty-lang[analysis]`); the package installs and runs without it, and z3-backed features degrade to an "install the extra" message rather than failing.

3. The language (Tier 1)

Thirsty-Lang uses a water metaphor: `drink` declares a binding, `pour` emits, `sip` reads, `glass` declares a function, `fountain` a class, `reservoir` a list, `thirsty/hydrated` is if/else, `refill` loops. A module declares its governance **mode** in its header:

THIRSTY

```
module billing: governed

glass charge(amount: Int) requires amount > 0 ensures result >= 0 {
  return amount
}

drink total = charge(42)
pour total
```

Module modes:

- **core** - ordinary execution; capability gates are inert.
- **governed** - every gated capability (I/O, import, sensitive stdlib call) must be authorized by a policy verdict; deny-by-default.
- **strict** - governed plus mandatory initialization of declarations.
- **pure** - no I/O permitted (`pour/sip` rejected).

The interpreter supports first-class functions, closures, OOP (fields, methods, `new`), reservoirs/maps, `cascade` (await-and-yield), `spillage/error/cleanup` control flow, pipelines (`|`, `->`), and tail-call optimization at higher

optimization levels. A 16-namespace standard library (`thirst::fs`, `http`, `net`, `process`, `env`, `sqlite`, `crypto`, `json`, ...) plus 16 global builtins is available through `import`.

Design-by-contract is built in: `requires` (precondition), `ensures` (postcondition, with `result` bound), and `invariant` (checked at entry **and** exit) are executable runtime checks on governed functions, and contract predicates on methods hold in any mode.

4. The governance model (Tier 3 in action)

4.1 Capability gates, fail-closed

In governed mode, every side-effecting operation routes through `_gate_capability(action, target)` before it runs. The gate is **fail-closed**:

- If no policy engine and authority are attached, the capability is **denied** with a proof - governed mode never implies authority.
- Otherwise the action/target/authority context is evaluated by T.A.R.L.; only an explicit **ALLOW** lets the effect proceed. Deny-by-default (no matching rule -> DENY) comes from the engine itself.

Gated capabilities include `pour/print` (write stdout), `sip` (read stdin), `import`, and **every sensitive imported-stdlib call**. The capability/action map for the standard library is a single source of truth (`module_system.SENSITIVE_STDLIB_CAPABILITIES`): each sensitive callable carries an explicit `read / write / network / execute` action, so allowing `action == "import"` grants no filesystem write, network call, subprocess, environment mutation, logging, or SQLite side effect (challenges C029-C032).

4.2 Imports execute under the caller's governance

An imported `.thirsty` module runs under the **caller's** governed runtime - sharing its policy engine and authority and forced into governed mode - rather than in a detached, ungoverned interpreter. Top-level effects in the imported module are gated at import time, and the function closures it exports remain gated when later invoked from governed code. A denial during import surfaces as a `GovernanceViolation` and is **not** swallowed by `spillage` (challenge C035).

4.3 The hard floor

`GovernanceViolation` is a control-flow floor: `spillage` and other error handlers must not catch it, so a governed denial cannot be absorbed by application error handling (challenge C006). Cross-mode calls (invoking a governed function from `core`) are rejected statically (E053) and at runtime.

4.4 Parser fail-closed

A `governed` module with **any** parse error fails closed: the parser discards all recovered statements (no executable program survives), and the interpreter refuses to run it, raising a denial with a proof. Parser error-recovery therefore cannot smuggle executable statements past a malformed boundary (challenge C036). Non-governed modules keep ordinary error recovery.

5. T.A.R.L. - the policy engine and proof system

T.A.R.L. (Thirsty's Active Resistance Language) is a standalone policy layer.

5.1 Policy language and evaluation

Policies are ordered rule lists with **first-match-wins** semantics and three verdicts - **ALLOW**, **DENY**, **ESCALATE** (for human/quorum workflows):

TARL

policy access

```
when role == "admin" => ALLOW
when action == "read" and resource IN source:allowlist => ALLOW
when true => DENY           # explicit default-deny floor
```

The runtime evaluates rules over a context dict, with a safe expression evaluator (no `eval/import/state` mutation in conditions), an LRU decision cache, and adaptive rule ordering that never changes first-match semantics. A dynamic **source registry** binds live data providers to `source:<name>` references. Contexts and sources are restricted to finite JSON values. Missing paths, wrong intermediate types, representation conflicts, incompatible expression kinds, and evaluator exceptions terminate the affected decision fail closed; they do not become ordinary false values or fall through to a later ALLOW. Boolean operators validate both operands, and quantifiers validate every supplied element rather than stopping at an earlier decisive value. Empty quantifier collections are not authorization evidence and fail closed. Numeric comparisons are type-strict: integers and floats interoperate, but strings never coerce to numbers. Numeric operands and arithmetic results must be finite. The `__tarl__` quantifier binder namespace is reserved for trusted runtime state.

5.2 Temporal governance

Rules and policies carry time windows (`valid_from/valid_until`, durations like `for: 4h`, `if_unresolved_after ... revert_to ...`). Temporally-constrained decisions are never cached, so an expiring ALLOW cannot be replayed from cache.

5.3 Composition

Policies compose (`EXTENDS/RESTRICTS`) and group into policy sets with set operators (`UNION/INTERSECT/MAJORITY`). Composition cycles raise an error rather than failing open.

5.4 Proof-carrying decisions

Every governed boundary decision - policy verdict, contract verdict, or fail-closed denial - emits a `TarlProof` certificate binding:

- `policy_hash` - SHA-256 of the policy source,
- hashes of the original request and exact canonical/evaluated context, plus the representation ID, normalization algorithm ID/version, and conflict status,
- for policy proofs, the schema fingerprint, representation, and validation status (a positive proof requires passed),
- the matched rule index and condition, the verdict, and a per-rule **trace** up to the first match,
- an evaluation timestamp, applicable expiry, and an optional signature.

Runtime proof generation can be unsigned when no signing key is configured, but independent verification rejects unsigned proofs by default. Two signing modes exist:

- **HMAC-SHA256** - a *symmetric* MAC: detects tampering, but anyone holding the shared key (including the verifier) can forge it. Documented as **not** non-repudiable.
- **Ed25519** - an *asymmetric* signature; the verifier needs only the public key. Use this when a proof must attest to the signer.

Derived schema bindings follow policy identity. If evaluation supplies a different `policy_text` override, a runtime-derived schema is regenerated from that exact policy and bound into its proof. An incomplete override derivation denies instead of inheriting the base policy's schema.

5.5 Independent verification - secure default, explicit unsigned mode

`ProofVerifier` checks signature validity, policy-hash match (when the source is supplied), trace consistency, context/schema coherence, and expected-context bindings - with no runtime or policy engine required. A registered-source ALLOW requires both the original and evaluated contexts, and only valid top-level `source:<name>` additions may distinguish them. A coherent raw evaluator ALLOW without a passed schema remains inadmissible to `CapabilityBroker` and the governed interpreter.

`QuorumResolver` applies the same positive-authority standard to escalation. It requires an independently verified, cryptographically signed `ESCALATE` proof bound to the exact policy/rule and a passed authoritative schema before it can promote. Resolution also requires the exact original request context, an explicit timezone-aware trusted clock, and a verifier with a maximum proof age and replay guard; registered-source proofs additionally require the exact evaluated context. Each approval is Ed25519-signed over the SHA-256 digest of the complete proof; duplicate identities or public-key material count once, and time-bound promotion preserves the verified signed expiry.

The default requires a signature. Local unsigned-proof inspection must opt in explicitly:

PYTHON

```
ProofVerifier(
    require_signature=True,           # default; reject unsigned proofs
    allowed_signature_algorithms={"ed25519"}, # reject HMAC / downgrade
    require_policy_source=True,       # require policy binding
)

ProofVerifier(require_signature=False) # explicit unsigned inspection
```

The CLI exposes this as `tarl verify --ed25519-only --policy <policy.tarl>`. Unsigned local inspection is explicit with `tarl verify --allow-unsigned`. Verification rejects unsigned-by-default, wrong-key, tampered-field, HMAC-as-Ed25519, trace-verdict mismatch, and missing-policy-source proofs when that binding is required (challenge C025/C055).

5.6 Static analysis (optional Z3 layer)

With the `analysis` extra, `tarl analyze` proves policy properties via Z3: **coverage** (contexts that fall through to default-DENY), **shadows** (dead rules), **conflicts** (overlapping rules with different verdicts), **equiv** (two policies decide identically for all contexts), and **refines** (one policy never allows what another would deny). Because z3's bindings share a non-thread-safe global context, every analysis entry point is serialized through a process-wide lock with z3 garbage collected inside it - so concurrent use (e.g. the LSP running analysis on a background thread) cannot corrupt the z3 heap.

5.7 Audit, explanation, and tooling

T.A.R.L. ships an append-oriented **audit archive** (proofs stored on evaluation, queryable by verdict/time), an **explainer** (`tarl explain` - why a context got a verdict, including skipped rules), a **test runner** (`.tarl_test` suites), and a **language server** (`tarl-lsp`, JSON-RPC over stdio or TCP) providing syntax diagnostics plus z3-backed coverage/shadow hints.

6. Mutation governance (Tier 4) and integrity (Tiers 5-6)

Shadow Thirst answers one question about a proposed change: does a *shadow* block compute the same thing as the *validated canonical* block? Three layers answer it, cheapest-and-strongest first:

- 1 **Structural** - alpha-renamed AST equality (a sound sufficient condition).
- 2 **Z3 symbolic** (optional) - translate each block's returned value to a Z3 term over shared symbolic inputs; **UNSAT** proves equivalence for all inputs, **SAT** yields a concrete diverging counterexample.
- 3 **Execute-and-compare** - run both in a sandboxed interpreter over seeded inputs and compare; abstains when blocks have observable effects.

A six-analyzer pipeline (plane isolation, determinism, resource estimation, purity, memory evaporation, canonical convergence) drives a promote/reject verdict. **TSCG/TSCG-B** provide symbolic-constraint and binary-frame integrity surfaces.

7. Build outputs and governance loss

Thirsty-Lang can transpile to JavaScript, LLVM IR (and via the toolchain to object/exe/asm/JIT), and a Pyodide (WASM) HTML bundle. None of these targets carry the governed runtime, so a governed module compiled to them **loses policy enforcement**. The build therefore **refuses** a governance-dropping target for governed source by default. `thirsty build --allow-governance-loss` is required to proceed; it warns on stderr and records `build.governance_loss = true` in the emitted manifest (challenge C034). Governance loss is never silent.

8. Security posture: covered vs. deferred

The threat catalog (C001-C073) is the source of truth. As of 0.8.6:

Every challenge C001-C073 has **released test coverage** in 0.8.6. The repaired wheel, source distribution, and container were published and passed their release gates; C058-C073 remain blocked only at the separate independent Competence Register acceptance gate. Highlights:

- **Fail-closed core** - governed I/O/import/print without policy (C001-C004); non-matching capability denial (C005); denial unswallowable by `spillage` (C006); cross-mode guard (C007); evaluator/audit failure -> DENY (C037-C038).
- **Proof integrity & freshness** - tamper/wrong-key/no-key/policy-swap detection (C009-C014); downgrade rejection (C025); context-bound replay, freshness windows, policy revocation, single-use replay guard, and canonical signature encoding with semantic replay identity (C023-C024, C070).
- **Authority provenance** - Ed25519-signed `AuthorityClaims`; a bare `--authority` string grants nothing in hardened mode (C027-C028); hardened mode also mandates Ed25519-signed proofs.
- **Tamper-evident audit** - hash-linked archive whose `verify_chain` detects edits, deletions, and reordering (C022, C026, C049).
- **Universal broker** - one `CapabilityBroker` mediates FFI/native (C033) and agent/MCP tool adapters (C040-C041), denied by default.
- **Context & path safety** - schema and evaluator share one authoritative nested JSON representation; missing, malformed, conflicting, or mixed dotted-path context is preserved as a typed failure; finite JSON and strict type algebra prohibit Python-specific coercion (C045-C046, C058-C061, C064). Eager boolean/quantifier validation, finite arithmetic, reserved evaluator binders, and per-policy override derivation close hidden-expression and stale-schema paths (C065-C068). `PathGuard` confines canonical paths against traversal and symlink escape (C042).
- **Positive authority & sources** - brokered and governed effects require a passed schema binding before ALLOW can advance; registered-source proofs bind original/evaluated contexts and permit no hidden caller-data rewrite (C060, C062-C063).
- **Time & escalation** - `TrustedClock` decides temporal windows on verified explicitly zoned signed time and an invalid configured clock fails closed without host fallback (C043); `tarl eval` refuses policy windows, time-bound verdicts, `CURRENT_*`, and `ELAPSED_SINCE` without explicit zoned `--now` (C056); `QuorumResolver` upgrades ESCALATE only from an independently verified, cryptographically signed, current-request/policy/rule/schema-bound proof checked under explicit trusted time, freshness, and replay controls, with distinct digest-bound approvals while preserving verified signed expiry (C050, C069); matched authority expires at the earliest exclusive policy or rule cutoff, malformed temporal restrictions are rejected, and expiry cannot grant ALLOW (C071-C073); the linter flags broad/ungated ALLOW (C039).
- **Adversarial peer-review fixes** - TARL comparison is type-strict and denies every string/number comparison without conversion (C053), malformed or throwing rules reject/deny instead of falling through (C054), unsigned forged proofs are not valid by default and verdict-bearing traces are checked (C055), and `thirsty govern --auto-tarl` emits policies keyed on runtime `action` (C057).

Operational readiness (0.8.6). The three items previously left to the embedder are now implemented in-tree: the in-language capability gate routes through the same `CapabilityBroker` as the out-of-language adapters (one enforcement path, with optional `PathGuard` confinement on file targets); `utf.tarl.durable` provides a SQLite-backed `DurableReplayGuard` and `RevocationStore` plus an external audit-chain checkpoint; and `utf.tarl.keystore` / `tarl keygen` provide a defined on-disk key format, file-based loading, and a rotation path. What genuinely remains with the embedder is *secret custody* - the durable stores and key files live wherever the deployment puts them (vault, HSM, encrypted volume), and automatic schema derivation remains conservative and fails closed when ambiguous. See `docs/PRODUCTION_DEPLOYMENT.md` for the deployment checklist and `THREAT_MODEL.md` §"Remaining Gaps".

9. Engineering state (0.8.6)

Property	State
Test suite	1,463 passed, 1 skipped, and 22 subtests at the 0.8.6 local release gate; CI covers Python 3.11/3.12 and the C001-C073 offensive catalog
Line coverage	90.60% at the local 0.8.6 release gate; 90% CI floor enforced
Lint	<code>ruff check src tests</code> clean
Types	<code>mypy -p utf</code> clean across all modules; ships a PEP 561 <code>py.typed</code> marker; <code>mypy</code> is a CI gate
Packaging	builds an sdist + wheel; clean-venv install runs all 7 console scripts and shipped examples
Platform	Windows/macOS/Linux; UTF-8-safe CLI output on Windows code pages

The reference implementation is an interpreter. The governance substrate meets its own hardened-runtime acceptance bar (§8), and as of 0.8.6 the operational items that were previously deferred - adapter routing through the broker, durable cross-process replay/audit state, and deployment key management - are implemented in-tree and exercised by a CI `production-acceptance` job. Treat load-bearing context authority as **pending independent constitutional acceptance**. The repaired 0.8.6 PyPI artifact already passed the preserved package matrix; only an independent authority may change the critical Competence Register entries. A deployment must still provision secret custody for durable stores and trust-root key files per `docs/PRODUCTION_DEPLOYMENT.md`.

10. Getting started

BASH

```
pip install thirsty-lang          # core
pip install thirsty-lang[analysis] # + Z3 static analysis & symbolic convergence

thirsty run program.thirsty --thirst-level governed \
  --authority admin --policy access.tarl

tarl eval access.tarl -c '{"role":"admin"}'
tarl eval temporal.tarl -c '{"role":"admin"}' --now 2026-07-01T12:00:00Z
tarl analyze coverage access.tarl
tarl verify proof.json --ed25519-only --ed25519-key id:HEX
```

A denied governed call prints `governance denied: <fn>: <reason>` with the proof verdict and policy hash, and exits non-zero.

11. Conclusion

Thirsty-Lang makes authorization a property of the runtime rather than a convention of the code: in governed mode no effect precedes an explicit ALLOW, every decision carries a verifiable proof, and every "secure" claim is pinned to a test or labeled as roadmap. In 0.8.6 the offensive catalog (C001-C073) has released-artifact coverage, while C058-C073 remain blocked pending independent Competence Register acceptance. The operational substrate - authority provenance, tamper-evident audit, replay/downgrade rejection, universal adapter brokering, durable cross-process state, and deployment key management - is implemented in-tree and gated in CI. What remains for a deployment is secret custody for the durable stores and trust-root keys, per [docs/PRODUCTION_DEPLOYMENT.md](#).

Canonical manual: [THIRSTY_LANG_UTF_101.md](#)

References: [LANGUAGE_SPEC.md](#) · [GRAMMAR.md](#) · [governance_model.md](#) · [THREAT_MODEL.md](#) · [SIGNING.md](#) · [STATUS.md](#)

PART 14 / HISTORICAL-EVIDENCE

Operational Continuity and Release Evidence

Maintained source: docs/operations/CONTINUITY_MAP.md



Workstream: **Proof-Carrying Effects Language + TARL Security Corrections** Updated: 2026-08-02
Branch: `master` Workspace: `thirsty_lang_exploration_0754`

2026-08-02 v0.8.6 Release Closure and Documentation Reconciliation

- Release commit: `60f226a69059b5803b0035f037d09e6f7f9c45a2`.
- Annotated tag: `v0.8.6`; tag object `d89bbbc80a165ab6340cbe7855dfd6af2b086d1a`, peeling to the release commit.
- Branch CI run `30756636768`, tagged validation/PyPI run `30756676570`, and multi-architecture container run `30756676576` completed successfully.
- Local release gate: 1,463 passed, 1 skipped, 22 subtests, 90.60 percent coverage. A clean environment installed the published PyPI 0.8.6 package and reproduced the permanent context matrix.
- GHCR 0.8.6 publishes `linux/amd64` and `linux/arm64` under digest `sha256:6f3f516b8e979437dd414373afe581716b8c890dc4758b8675cbbcad9b94b13c`. Docker Hub remains stale at 0.8.1 and is not an authoritative 0.8.6 source.
- The preserved 0.8.5 reproduction remains anchored by evidence-manifest hash `4DD3E6A4CF017E3E34A25C62A398D29A6F93D279890C4CCDF2A4528C6333F8D5`.
- `CR-TARL-ALLOW` and `CR-TARL-DENY` remain PASS. The released repair does not itself change `CR-CONTEXT-COHERENCE` or `CR-CONTEXT-RESOLUTION-INTEGRITY`; both remain critical FAIL pending independent constitutional acceptance.
- Documentation is being reconciled to the released behavior and assembled into `docs/THIRSTY_LANG_UTF_101.md` plus the generated single-file canonical PDF.

2026-08-02 Context Resolution Integrity Repair

- An isolated install of published `thirsty-lang==0.8.5` preserved the canonical four-case context matrix. Nested `user.role` and simple `role` ALLOW paths work; flat dotted input and guest control return DENY. Evidence manifest: `4DD3E6A4CF017E3E34A25C62A398D29A6F93D279890C4CCDF2A4528C6333F8D5`.
- Adversarial evaluation confirmed a critical authorization bypass: unresolved dotted paths were converted to `False`, so negation and inequality could produce ALLOW. Published 0.8.5 is not admissible for load-bearing positive verdict authority.
- Active source now uses typed resolution states (`RESOLVED`, `MISSING`, `TYPE_ERROR`, `REPRESENTATION_CONFLICT`) and one nested context representation. Flat dotted keys, all mixed forms, duplicate keys, and trusted-layer collisions fail closed.

- Proofs bind original and canonical context hashes, representation identity, transformation algorithm/version, and conflict status. Positive legacy proofs without coherent context metadata are inadmissible even when unsigned inspection mode is selected.
- Permanent fixtures and tests are in `tests/fixtures/tarL_context_coherence/` and `tests/test_tarL_context_resolution_integrity.py`. The matrix runs through `SafeExpr`, `evaluate_policy`, `TarLRuntime`, `tarL eval`, `schema/proof obligations`, `proof creation/verification`, and governed Thirsty-Lang routing.
- Post-audit expression hardening now evaluates both boolean operands and every supplied quantifier element, treats an empty quantifier collection as no authorization evidence, forbids string-to-number comparison coercion, rejects non-finite numeric operands/results, and reserves `__tarL_*` quantifier binders for trusted runtime state.
- Trusted-time hardening rejects naive signed assertions and CLI times. Once a runtime clock is configured, a missing, malformed, or failing value denies without falling back to host time. CLI `--now` is required for windows, time-bound verdicts, `CURRENT_*`, and `ELAPSED_SINCE`.
- Derived schemas now follow exact policy identity. A differing `policy_text` override receives a fresh complete derivation and proof binding; incomplete override derivation fails closed rather than inheriting the base policy's derived schema.
- Quorum promotion now requires independent cryptographic verification of the signed exact-policy/rule, schema-passed ESCALATE proof before distinct Ed25519 approvals over the complete proof digest are counted. The resolver also requires the exact request context, an explicit timezone-aware trusted clock, verifier freshness and replay enforcement, and the exact evaluated context for registered-source proofs. Time-bound promotion retains its verified signed expiry. Regression coverage is in `tests/test_threat_model_lint_quorum.py`.
- Threat-model cases C065-C073 permanently record the eager-expression, finite-result, reserved-binder, per-policy-schema, proof-bound-quorum, canonical replay-identity, and strict temporal-authority protections. Active-source coverage does not itself satisfy package-level independent acceptance.
- Competence Register state is in `docs/STATUS.md`. Context coherence and resolution integrity remain critical FAIL pending independent constitutional acceptance; the repaired 0.8.6 package and release gates have passed.

2026-07-29 Sync and Release Fix Workstream

Workstream: **Upstream README Sync + PyPI Trusted Publisher Fix** Branch: `iamsothirsty-sync-and-fix-release`

What Was Read

- Local HEAD at `d64e7ff` (Release 0.8.5: repair PyPI publishing).
- GitHub master ahead by 7 commits (`d35d179` through `2317dcc`); all README documentation/cosmetic changes via PR #22.
- Remote tag `v0.8.5 = 1da4a9735d06811ec82a10554ef5695cbc601df4` matches local tag exactly. Release identity/plumbing was verified at that time; the later 2026-08-02 context-resolution finding supersedes any broader claim that the public evaluator implementation was correct.
- `.github/workflows/release.yml` had `permissions: id-token: write` and used `pypa/gh-action-pypi-publish` but had **no environment: field on the job**.
- GitHub repo environments API: only `copilot` environment exists; no `release` or `IAmSoThirsty` environment was previously configured on 2026-07-29 start. By end of session: `release` environment exists (user-created), containing secret `IAMSOTHIRSTY` (PyPI API token). `IAmSoThirsty` environment was auto-created by GitHub during the failed OIDC run and is now unused.

- GitHub Actions run log for 29914952475 (v0.8.5 release, failed): `invalid-publisher: valid token, but no corresponding publisher and environment: MISSING`. This is the definitive root cause.

What Was Changed

- `.github/workflows/release.yml`: corrected `environment: release` (was `IAmSoThirsty`). Added `user: __token__` and `password: ${ secrets.IAMSOTHIRSTY }` to the publish step. Auth method is token-based (PyPI API token stored in the `release` environment secret `IAMSOTHIRSTY`); OIDC trusted publishing is not used despite `id-token: write` remaining in the top-level permissions.
- `README.md`: fast-forwarded from upstream master via `git merge FETCH_HEAD`. 132 insertions / 94 deletions - cosmetic hero-banner and badge additions only. No logic or API change.
- `docs/operations/CONTINUITY_MAP.md`: this update.

Verified

- `git merge FETCH_HEAD` completed as fast-forward. Working tree clean.
- `release.yml` confirmed correct after edit (all existing steps preserved; `environment: release` + token credentials added; confirmed `release` environment exists with secret `IAMSOTHIRSTY`).

Not Verified / Requires User Action

- 1 **PyPI environment name - Resolved.** Auth method is token-based. GitHub Actions environment is `release`. PyPI API token is stored as secret `IAMSOTHIRSTY` in the `release` environment. Workflow passes `user: __token__` and `password: ${ secrets.IAMSOTHIRSTY }`.
- 2 **GitHub environment creation - Resolved.** `release` environment exists (created 2026-07-29T14:09:14Z).
- 3 **Re-tag and push:** The v0.8.5 tag already exists on PyPI's side with failed publish. To re-publish, the user must either push a new tag (e.g. v0.8.5.post1 if PyPI allows it) or delete/re-create the GitHub release and re-tag if the PyPI release slot is still empty. Check PyPI project page first.
- 4 **Commit and push this fix:** This session's changes are uncommitted. The user must commit `.github/workflows/release.yml` and `README.md` and push to master (or merge this branch), then tag the next release.

Recommended Next Steps

- 1 ~~Verify PyPI trusted publisher environment name - Resolved: token-based auth.~~
- 2 ~~Create IAmSoThirsty GitHub environment - Resolved: `release` environment exists.~~
- 3 Commit: `git add .github/workflows/release.yml README.md docs/operations/CONTINUITY_MAP.md`
- 4 Push to master and create a new release tag (or re-trigger v0.8.5 if PyPI slot is still vacant).

Open Issues

- PyPI still shows stale metadata for `thirsty-lang`; v0.8.5 never published.
- No packaging validation ran (no dist artifacts in worktree; existing tests not re-run since no source code changed).

Baseline

- Starting branch: `master`.
- Starting git state: clean before this work began.
- Baseline continuity document read before edits.

- Existing governance surfaces inspected: parser/checker/interpreter CLI build, T.A.R.L. policy/proof/audit APIs, capability broker, Shadow Thirst, docs, and threat-model tests.
- Current problems rule applied: discovered issues are either fixed or listed under Open Issues instead of dismissed as pre-existing.
- Dirty state rule applied: current modified/untracked files are from this workstream.

Implemented

- `thirsty prove program.thirsty --policy policy.tarl --emit-manifest` emits a machine-readable static proof-obligation report without executing program side effects.
- Proof-obligation reports include functions, imports, sensitive stdlib calls, governed calls, required TARL actions, required capabilities, context schema, authority requirements, contract obligations, proof mode, audit requirement, build governance-loss status, Shadow status when statically visible, diagnostics, and unresolved proof gaps.
- Required TARL actions include capability actions and governed function-call actions.
- TARL policy references can derive simple context schema fields. Explicit JSON context schemas remain authoritative. Ambiguous or incomplete derived schemas fail closed in `thirsty prove`.
- Explicit JSON context schemas accept both list form and mapping form (`{"fields": {"name": "kind"}}`) and reject malformed entries with clear validation errors.
- Contract obligations preserve string literals in recorded annotations, so `requires path != ""` and `ensures result == "ok"` remain faithful in proof reports and manifests.
- `thirsty explain-denial program.thirsty --policy policy.tarl` emits a machine-readable explanation of missing policy, context, authority, and proof conditions.
- Build manifests now record source hash, optional policy hash, required capabilities, context schema, authority mode, proof verification mode, audit requirement, Shadow status when available, and governance-loss status.
- Checker-level effect warnings are available in prove/strict-style checking paths without changing normal checker behavior.
- TARL ordering comparisons compare numeric-looking strings numerically and fail closed on unorderable values instead of falling through.
- TARL policy load validates rule expressions; residual runtime evaluation errors produce DENY rather than skipping to a later rule.
- `ProofVerifier` and `tarl verify` reject unsigned proofs by default. Unsigned proof inspection must opt in explicitly with `require_signature=False` or `--allow-unsigned`.
- `tarl eval` refuses temporal policy windows and `CURRENT_*` builtins unless `--now` supplies a trusted evaluation time.
- `thirsty govern --auto-tarl` emits rules keyed on `action`, matching the governed runtime context.
- Package metadata is bumped to 0.8.2 for a non-colliding patch release tag.

Files Modified

- `src/utf/thirsty_lang/proof_obligations.py` - proof-obligation model, extraction, derived schema, denial explanation, and effect-warning helpers.
- `src/utf/thirsty_lang/cli.py` - `prove`, `explain-denial`, and policy/context-aware manifest output.
- `src/utf/thirsty_lang/checker.py` - optional proof-effect warning diagnostics.
- `src/utf/thirsty_lang/parser.py` - preserves string literals when storing contract annotation text.

- `tests/test_proof_obligations.py` - minimum proof tests for extraction, schema derivation, fail-closed schema behavior, sensitive stdlib obligations, build manifest proof metadata, denial explanation, no-side-effect prove behavior, explicit schema mapping shape, contract string-literal preservation, and replay/audit/proof regression.
- `README.md` - user-facing CLI additions.
- `docs/STATUS.md` - implemented/tested status rows for proof obligations.
- `docs/THREAT_MODEL.md` - C051/C052 challenge rows and current evidence.
- `docs/governance_model.md` - static proof-obligation model and CLI behavior.
- `docs/operations/CONTINUITY_MAP.md` - this continuity update.
- `src/utf/tarl/core.py` - type-safe comparisons, expression validation, trusted-time threading, and fail-closed `evaluate_policy`.
- `src/utf/tarl/runtime.py` - fail-closed runtime rule errors and verdict-aware proof traces.
- `src/utf/tarl/verifier.py` - secure-by-default signature requirement and trace verdict checks.
- `src/utf/tarl/cli.py` - `tarl eval --now` and `tarl verify --allow-unsigned`.
- `src/utf/tarl/spec.py` - proof documentation aligned with secure verifier defaults.
- `tests/test_peer_review_0_8_1_tarl_regressions.py` - adversarial regression coverage for the five adversarial TARL review findings.
- `pyproject.toml`, `src/utf/thirsty_lang/__init__.py`, `Dockerfile`, `docs/WHITEPAPER.md`, `docs/LANGUAGE_SPEC.md`, and `docs/SIGNING.md` - 0.8.2 release metadata and stale-doc corrections.

Tested

- `python -m pytest tests/test_cli_build.py::test_build_emit_manifest -q` passed: 1 passed.
- `python -m pytest tests/test_proof_obligations.py -q` passed: 10 passed.
- `python -m pytest tests/test_cli_build.py tests/test_threat_model_build_outputs.py -q` passed: 23 passed.
- `python -m pytest tests/test_thirsty_lang_governance.py tests/test_tarl_proof.py tests/test_threat_model_context_schema.py -q` passed: 110 passed.
- `ruff check src tests` passed.
- `mypy -p utf` passed: no issues in 52 source files.
- `python -m pytest tests/ -q --cov=utf --cov-fail-under=90 --basetemp .tmp_pytest\basetemp` passed: 1233 passed, 1 skipped, 7 subtests passed, total coverage 90.77%.
- `python -m build --wheel --no-isolation --verbose` passed and built the prior patch wheel before the 0.8.2 correction work.
- `python -m build --sdist --no-isolation --verbose` passed and built the prior patch sdist before the 0.8.2 correction work.
- `python -m build --verbose` passed and built the prior patch sdist and wheel before the 0.8.2 correction work.
- Fresh venv installed the prior patch wheel and verified installed `thirsty` CLI support from a `C:\Temp` venv for `prove`, `explain-denial`, and `build --emit-manifest --policy --context-schema`, including no side effects during `prove`, explicit schema mapping, preserved contract strings, source and policy hashes, and governance-loss status.
- `python -m pytest tests/test_peer_review_0_8_1_tarl_regressions.py tests/test_cli_tarl.py -q` passed: 51 passed.

- `ruff check src tests` passed after the 0.8.2 corrections.
- `mypy -p utf` passed after the 0.8.2 corrections: no issues in 52 source files.
- `python -m pytest tests/ -q --cov=utf --cov-fail-under=90 --basetemp .tmp_pytest\basetemp` passed after the final CLI correction: 1234 passed, 1 skipped, 7 subtests passed, total coverage 90.75%.
- `python -m build --verbose` passed and built `thirsty_lang-0.8.2.tar.gz` and `thirsty_lang-0.8.2-py3-none-any.whl`.
- `python -m twine check dist\thirsty_lang-0.8.2.tar.gz dist\thirsty_lang-0.8.2-py3-none-any.whl` passed for both artifacts.
- Fresh venv installed `dist\thirsty_lang-0.8.2-py3-none-any.whl` from `C:\Temp\thirsty-0.8.2-smoke-final` and verified imports/version, all console script help commands, numeric-string DENY, malformed-policy clean CLI error, temporal `--now`, unsigned-proof default rejection plus explicit unsigned mode, and auto-TARL `action` generation.

Not Yet Verified

- None for this workstream's acceptance gates.

Behavior Contracts

- `thirsty prove` is a static proof-obligation extractor. It reports obligations and unresolved gaps before execution. Runtime TARL verdict enforcement, proof verification, broker mediation, and audit append behavior remain runtime responsibilities.
- Derived context schema is fail-closed by design. Policy expressions that cannot be statically resolved produce an incomplete schema and require an explicit schema file before `thirsty prove` can claim schema completeness.
- TARL proof verification is secure by default; unsigned proof verification is an explicit compatibility/inspection mode.
- Temporal CLI evaluation requires explicit trusted time when policy windows or temporal builtins are present.
- Shadow convergence status is reported when it is present in the statically extracted source metadata. `thirsty prove` does not run a separate Shadow analysis pass.

Open Issues

- Git commit, GitHub push, and release tag push remain pending for the 0.8.2 correction work.

Resolved Validation Notes

- First `python -m build` attempt timed out after 10 minutes. A later exact isolated verbose build passed after build dependencies were cached.
- Installed-wheel smoke initially exposed explicit schema mapping handling and contract string-literal preservation bugs. Both were fixed and covered by regression tests plus fresh installed-wheel smoke verification.
- A venv launcher under the repo `.tmp` path was blocked by Windows Application Control. The same built wheel installed into a fresh `C:\Temp` venv ran the generated `thirsty.exe` launcher and passed the full proof/build smoke. This is resolved for acceptance by verifying the real console launcher from an allowed fresh venv path.

Future Work

- Extend obligation extraction to richer policy expression inference if the TARL grammar grows structured schema declarations.
- Promote selected checker effect warnings from diagnostic-only to strict-mode errors if a future release defines that compatibility boundary.
- Add richer Shadow Thirst convergence integration when a stable static analysis hook is exposed for ordinary `.thirsty` sources.

Safe to Continue

Yes. Targeted tests, full coverage gate, lint, type check, wheel build, sdist build, isolated verbose package build, and fresh installed-wheel CLI smoke are green.

COLOPHON / REPRODUCIBLE SOURCE MANIFEST

Document Provenance

Aggregate source SHA-256: 05193bc67744ed39b6f9c7457ed16b615fb436b7d58977bf74ae9516ecb674d8. The aggregate hashes each repository-relative path, a NUL separator, its exact bytes, and a final NUL in sorted path order. It includes the composition manifest and PDF builder, but not the generated PDF, avoiding a self-referential checksum.

Built deterministically for Thirsty-Lang 0.8.6; document revision 2026-08-02; fixed source epoch 1785686400. Final PDF metadata carries the same digest.

Source	SHA-256
CONTRIBUTING.md	1e1bc3574f0665a40a46be03baea8fc7458ce0ba588afe243c3be57c08e62202
DOCKER.md	0390f46675c575d69bcacce41e7cdc04d636ec95639588ac3b676456032d41bf
SECURITY.md	79907c165d5712914d186dd6a9d419870fb75da57b331b57216a9ce61d970ce4
docs/FEATURES.md	41444995d39df67546a90cd139ae10ae3ca68cc3a555b457a817890f898c2098
docs/GRAMMAR.md	8b25985d455f4d65fa8f12827b46a056831a11b367cd6ada6b3cb07c5d707805
docs/LANGUAGE_SPEC.md	9857ab7e011cd2fa25f01d9beeafe99aa488828fdec34cc9163574dcd607a8a4
docs/PRODUCTION_DEPLOYMENT.md	61b063cd23d9b41685b331df7cf384986bb981a979aefb251ed7428523352047
docs/SIGNING.md	c4fc9f423ca7256635c4c7faed5f6f86e52624b68343952a0b33b147b8329c3c
docs/STATUS.md	28c5c58edc6b2b57020ae4cb6070ecc2e89b3679574b0f37c53d8898a4487a51
docs/THIRSTY_LANG_UTF_101.md	0eeb49833217eb0d047829892aea47d12357714b6e036a073038cea36c4f9f27
docs/THREAT_MODEL.md	610dce6a0d4ac34e83fae80fe03effa3a5b995195d94b6b657cb3f64b29f5bab
docs/WHITEPAPER.md	4a8d07dc2fe1a67ece475079331c95d2ce31619829a0997f9155039cd1f42b9b
docs/governance_model.md	38eb83485fcf9ff33ecf28ce932a833a02fb6f79e620aa08edabc4ba8bc3a91b
docs/operations/CONTINUITY_MAP.md	9d1a8137e66cd548aa2d8466a03b66c20f3db3d582fe2339f543632f7cebce6e
docs/thirsty_lang_utf_101.toml	9d6c6a71d7c47df2b5c199d3d24cf6875444351d3a59b15ed5221fc03dde
scripts/build_thirsty_lang_utf_101.py	894fe3b2663335df1d839ca0e6525e5078cc1e62d2b3eda69e6e7274037aa31b

Authority and limits

This edition unifies maintained project documentation. Active source and deterministic tests decide implemented behavior when prose and code conflict. The Bootstrap Competence Register remains the authority for its own independent acceptance states; PDF generation does not promote a failed competence.

Copyright 2026 Thirsty's Projects LLC. Distributed under Apache-2.0.